

1 Appendix H: Spectral Indicators and Advisory Invariants

1.1 Scope and relation to the Core and audit layer

This appendix develops spectral indicators, advisory invariants, warning policies, and verifier-side re-entry rules for the Technical Extension layer. It does not enlarge the Core or create an external-domain theorem.

The central rule is

Advisory information may guide verification, but it does not determine a Core verdict.

The repaired window-first persistence semantics of Appendix A, the Type IV discipline of Appendix D, the non-compensating status calculus of Appendix F, and the manifest schema of Appendix G remain controlling.

Definition 1.1 (Advisory sovereignty principle). Advisory data may guide where, when, and how verifier-side checks are performed. They do not determine, compensate for, repair, or mutate a Core verdict.

Definition 1.2 (No external-domain upgrade principle). No spectral indicator, potential, warning, ranking, AI confidence value, advisory band, collapse-energy candidate, or advisory attachment proves an arithmetic, PDE, geometric, categorical, cryptographic, or other external-domain conclusion. Such conclusions require separately proved and registered Interface or Bridge Theorems.

Definition 1.3 (Immutable dependency baseline). The statements of this appendix are interpreted relative to:

- (i) AK-HPDST v19.0.0 Main Chapters 1–8;
 - (ii) Foundation Appendices A–G;
 - (iii) CR-v19.0.0-r3;
 - (iv) CM-v19.0.0-r2 and TB-v19.0.0-r1, only at their registered strengths.
- Only the registered mathematical strengths of these dependencies are imported here.

1.2 Standing repaired Core regime

Fix a field k , work in the constructible one-parameter regime

$$\text{Pers}_k^{\text{cons}} \subset [(\mathbb{R}, \leq), \text{Vect}_k],$$

and use finite-type filtered objects in $\text{FiltCh}(k)$ when a filtered realization is invoked. For monitored degree i , right-open window W , and threshold $\tau > 0$, set

$$B_{i,W}(F) := W_W \mathbf{P}_i(F), \quad \text{Eval}_{i,W,\tau}(F) := T_\tau^{\text{bar}}(B_{i,W}(F)).$$

Windowing precedes hard deletion. Threshold clearance is measured on $B_{i,W}(F)$ before deletion.

The verifier-side conditions relevant to this appendix include, when invoked,

$$B\text{-Gate}^+, \quad \text{Eval}_{1,W,\tau}(F) = 0, \quad (\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0), \quad \text{val}_B(\mathfrak{d}_{\text{met}}) < \text{gap}_\tau(\mathcal{B}).$$

No advisory expression is a substitute for any of these typed records.

Definition 1.4 (Typed advisory status). Every advisory datum has exactly one lifecycle status among

proposed, computed, reproducible, stale, superseded, invalid, undefined.

These are advisory-lifecycle statuses, not Core atomic statuses. In particular, computed and reproducible do not mean pass.

Proposition 1.5 (Unknown or unlabeled advisory output is non-verdict). *An advisory output with missing identity, scope, generator, normalization, status, or provenance is non-verdict data and cannot be consumed by any verifier-side clause.*

Proof. The missing fields prevent typed reconstruction and claim-use preflight. By the v19 weaker-safe rule, the output remains outside verifier-side evidence rather than being silently promoted. $\square$

1.3 Typed advisory objects and invariance discipline

Definition 1.6 (Advisory datum). A typed *advisory datum* is a record

$$A_{\text{adv}} = (\text{id}, \text{generator}, \text{inputs}, \text{scope}, \text{codomain}, \text{normalization}, \text{value}, \text{status}, \text{provenance}, \text{artifacts})$$

whose declared purpose is attention, warning, prioritization, navigation, calibration, or rechecking. It is not a verifier-side record merely because its value is mathematically defined.

Definition 1.7 (Advisory reading rule). The only default reading is

$$A_{\text{adv}} \rightsquigarrow \text{attention, warning, prioritization, navigation, calibration, or rechecking.}$$

The forbidden default reading is

$$A_{\text{adv}} \rightsquigarrow \text{proof, discharge, pass, reject, obstruction number, or external theorem.}$$

Definition 1.8 (Declared invariance relation). Let $\sim_{\mathcal{E}}$ be a declared equivalence or comparison relation on the input scope. An advisory datum is called $\mathcal{E}$ -invariant only when a proof or verified computation establishes

$$X \sim_{\mathcal{E}} Y \implies A_{\text{adv}}(X) = A_{\text{adv}}(Y)$$

with the same normalization, domain, codomain, and versioned generator.

Proposition 1.9 (The word invariant does not establish invariance). *Naming a score or profile an “invariant” does not establish invariance under isomorphism, interleaving, representative equivalence, window refinement, threshold change, or realization change.*

Proof. Each listed relation has distinct typed hypotheses. A name supplies none of them, so no invariance conclusion follows without the corresponding proof record. $\square$

1.4 Spectral indicators and typed integration

Definition 1.10 (Spectral indicator). A *spectral indicator* is an advisory datum derived from spectral, graded, filtered, barcode, energy-like, or threshold-sweep data. Its record must declare the monitored degree, window, threshold or sweep domain, source profile, normalization, and output codomain.

Definition 1.11 (Measurable spectral profile). For a declared filtered object F and window W , a spectral profile is an advisory map

$$\text{Spec}_{F,W} : D \longrightarrow [0, +\infty]$$

on a declared measurable scale domain $D \subseteq (0, +\infty)$. The record states whether the map is Borel measurable, whether it is finite almost everywhere, and whether the intended band integral is finite, extended-valued, or undefined.

Definition 1.12 (Band-limited spectral indicator). Let $B = [r_0, r_1] \subset D$. If $\text{Spec}_{F,W}$ is measurable on B , define the extended band indicator

$$\text{Sl}_B(F, W) := \int_B \text{Spec}_{F,W}(r) dr \in [0, +\infty].$$

A finite-valued claim additionally requires integrability. For a discrete declared mesh $\{r_j\}$, define

$$\text{Sl}_B^{\text{disc}}(F, W) := \sum_{r_j \in B} \text{Spec}_{F,W}(r_j) \Delta r_j$$

only when the weighting and convergence policy are declared. Otherwise the indicator status is `undefined`.

Definition 1.13 (Repaired-profile spectral indicator). A spectral indicator is *repaired-profile based* when its source is exactly

$$\text{Eval}_{i,W,\tau}(F) = T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F))$$

or a declared family of such profiles. This phrase does not make the indicator a Core evaluation or gate clause.

Definition 1.14 (Comparison-calibrated advisory family). A family of advisory values is comparison-calibrated only when all compared records share, or are connected by a proved transport of, the same input type, scale, normalization, monitored scope, window convention, threshold convention, generator version, and output semantics.

Proposition 1.15 (Uncalibrated advisory values are not comparison evidence). *Two advisory values computed under different undeclared normalizations, scopes, windows, thresholds, generator versions, or codomains cannot be used as evidence that the underlying Core-readable objects are equal, ordered, interleaved, or close.*

Proof. A numerical comparison without a common typed semantics does not determine a relation between the source objects. The required comparison record is absent, so the stronger conclusion is unavailable. $\square$

Definition 1.16 (Sampled advisory band). A sampled advisory band is a threshold interval J together with advisory evaluations at a declared finite or countable sample set $S \subset J$. It records only the observed sample behavior unless an interpolation, monotonicity, continuity, or finite-reduction theorem is supplied.

Proposition 1.17 (Sample stability is not band certification). *Stability of an advisory indicator on $S \subset J$ does not imply advisory stability, threshold-gap safety, or Type IV cleanliness at every point of J unless a separate theorem reduces the continuum claim to the sampled data.*

Proof. Unsampled parameters may contain discontinuities, threshold crossings, or diagnostic changes. A finite or countable sample alone supplies no universal quantifier over J . $\square$

1.5 Advisory noninterference and non-compensation

Definition 1.18 (Verifier-side projection). For a complete certificate record C , let $\pi_{\text{ver}}(C)$ denote the sub-record containing only verifier-side objects, obligations, statuses, artifacts, ledgers, comparison records, diagnostics, and gate verdicts. Advisory attachments are omitted.

Theorem 1.19 (Advisory noninterference). *Let C and C' be well-typed certificate records satisfying*

$$\pi_{\text{ver}}(C) = \pi_{\text{ver}}(C').$$

Then their Core atomic status and failure route are identical, regardless of differences in advisory attachments.

Proof. By Core sovereignty and Appendix G semantics, the verdict function factors through the verifier-side projection. Equal verifier-side projections therefore produce equal verdicts and failure routes. Advisory fields are outside the function's input. $\square$

Corollary 1.20 (Attachment insertion and removal are verdict-neutral). *Adding, deleting, reordering, or re-computing advisory attachments without changing π_{ver} cannot turn *undefined* or *reject* into *pass*, nor can it turn *pass* into *reject*.*

Theorem 1.21 (Advisory non-compensation). *No finite or infinite collection of favorable advisory values can compensate for a failed or undefined required verifier-side obligation. In particular, advisory stability, low potential, high confidence, or zero advisory energy cannot repair missing eligibility, a missing tower artifact, failed threshold-gap evidence, a nonzero diagnostic, or a failed budget inequality.*

Proof. The v19 readiness calculus is conjunctive on required obligations and forbids pass by cancellation. Advisory values are not members of the required verifier-side obligation family unless separately reconstructed as new verified records. Hence they cannot compensate for a blocker. $\square$

1.6 Verifier-side re-entry calculus

Definition 1.22 (Re-entry candidate). A re-entry candidate is a proposal to use advisory input in the construction of a new verifier-side record. The advisory datum itself does not change type. The candidate specifies:

- (i) the target verifier-side record type;
- (ii) exact source and target scope;
- (iii) a theorem, verified computation, or artifact-backed construction;
- (iv) all hypotheses and non-scalar obligations;
- (v) the resulting immutable record identifier;
- (vi) one status among candidate, verified, rejected, or undefined.

Definition 1.23 (Re-entry witness). A re-entry witness is the theorem or artifact package proving that the newly constructed record satisfies the target verifier-side schema and semantics. A witness must identify its authority scope, environment, inputs, outputs, artifact hashes, and failure behavior.

Theorem 1.24 (No declaration-only re-entry). *Declaring an advisory datum to be verifier-side, renaming it as a gate, diagnostic, representative, or ledger component, or copying it into a verifier-side field does not establish a valid re-entry.*

Proof. A verifier-side record is typed by its mathematical construction, hypotheses, proof or artifact evidence, and schema conformance. A declaration or field move supplies none of these. Therefore the target record remains undefined until a re-entry witness is verified. $\square$

Proposition 1.25 (Re-entry is immutable and non-mutating). *A successful re-entry creates a new content-addressed verifier-side record linked to the advisory source by provenance. It does not mutate the source advisory datum or retroactively alter a prior verdict.*

Proof. Content-addressed identity and the no-retroactive-mutation rule require new records for changed types or evidence. Provenance links preserve ancestry without identifying the two semantic objects. $\square$

1.7 Budget conversion and metric safety

Definition 1.26 (Certified advisory-to-budget conversion). Let $(V, \oplus, 0_V, \preceq)$ be the declared metric-ledger quantale. A certified advisory-to-budget conversion consists of a new element $\delta_{\text{adv}} \in V$, together with:

- (i) immutable identifiers for the actual pre-threshold profiles B, C ;
- (ii) a scope-matched conversion theorem or verified computation;
- (iii) a declared scalarization $\text{val}_B : V \rightarrow [0, +\infty]$;
- (iv) reconstructible evidence that

$$d_B(B, C) \leq \text{val}_B(\delta_{\text{adv}});$$

- (v) all crop, window-edge, normalization, and approximation contributions required by the bound;
- (vi) inclusion of δ_{adv} in the declared metric ledger.

A numerical score-to-ledger map without item (iv) is advisory calibration, not certified budget conversion.

Theorem 1.27 (Soundness boundary for budget conversion). *A budget-converted advisory quantity may affect the Core budget condition only through the newly certified ledger element δ_{adv} and only on the scope for which the discrepancy bound is proved. The original advisory value remains non-verdict data.*

Proof. The Core budget consumes typed ledger elements whose scalarizations bound actual discrepancies. Definition 1.26 supplies such an element and evidence. No other semantic authority is transferred from the advisory source. $\square$

Proposition 1.28 (Zero advisory score does not imply zero metric defect). *The equality $A_{\text{adv}} = 0$ does not imply $\delta_{\text{adv}} = 0_V$, $d_B(B, C) = 0$, exact comparison, or positive threshold reserve unless a theorem proves the relevant implication.*

Proof. The advisory codomain and the metric-ledger quantale are distinct typed spaces. Equality to zero in the first has no default transport to the second. $\square$

1.8 Gate-adjacent quantities and warning semantics

Definition 1.29 (Gate-adjacent advisory quantity). An advisory datum is gate-adjacent when it is designed to correlate with possible residual persistence, categorical obstruction, Type IV defect, comparison instability, or budget sensitivity. Gate adjacency is a purpose label, not a logical relation.

Proposition 1.30 (Gate-adjacent is not gate-equivalent). *A gate-adjacent advisory datum is not equivalent to the corresponding $B\text{-Gate}^+$ clause unless an explicit theorem proves the equivalence on the declared scope and a valid re-entry witness constructs the verifier-side record.*

Proof. Correlation or predictive usefulness is weaker than logical equivalence. The additional theorem and re-entry witness are exactly the missing implications and typed construction. $\square$

Definition 1.31 (Spectral warning policy). A warning policy declares the indicator, scope, trigger condition, recommended response, severity semantics, false-positive/false-negative interpretation when known, and explicit statement that the warning is not a Core verdict.

Proposition 1.32 (Warning triggers inspection, not verdict). *A warning may trigger recomputation, refinement, resampling, representative review, tower-artifact review, threshold-gap verification, or escalation to formal proof. It cannot itself assign *pass*, *reject*, or a numbered failure type.*

Proof. Warnings are proposer-side routing signals. Atomic status and failure typing require the corresponding verifier-side object and evidence. $\square$

1.9 Type IV advisory proxies and stable bands

Definition 1.33 (Type IV advisory proxy). A Type IV advisory proxy is an advisory datum intended to correlate with a possible terminal-generic kernel or cokernel defect. It does not itself define either defect profile.

Definition 1.34 (Actual Type IV diagnostic). For an invoked monitored triple (i, W, τ) , an actual persistence morphism

$$\phi_{i,W,\tau} : M_{i,W,\tau} \longrightarrow N_{i,W,\tau}$$

is required. If source, target, kernel, and cokernel are terminal-tame on the same declared terminal scope, define

$$\mu_{i,W,\tau} := \text{tgdim}_W(\ker \phi_{i,W,\tau}), \quad \nu_{i,W,\tau} := \text{tgdim}_W(\text{coker } \phi_{i,W,\tau}).$$

The specialized Type IV status is exactly one of

`not_invoked, undefined, certified_zero, obstructed.`

No numerical pair is imputed in the first two cases.

Theorem 1.35 (Type IV proxy is not Type IV diagnosis). *A Type IV advisory proxy does not determine $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$ or $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) \neq (0, 0)$. Such a determination requires the actual comparison morphism, terminal-tameness evidence, defect profiles, and terminal-generic computation from Definition 1.34, or a separately proved exact transport theorem to those data.*

Proof. The Type IV diagnostic is defined from the kernel and cokernel of a typed morphism. A proxy provides neither the morphism nor those defects by default. Therefore no diagnostic equality follows. $\square$

Proposition 1.36 (Missing Type IV artifact cannot be repaired by a proxy). *If Type IV is invoked but $\phi_{i,W,\tau}$ or terminal-tameness evidence is missing, the Type IV clause is `undefined`. No proxy value, including zero, changes that status.*

Proof. The missing artifact is a non-scalar proof obligation. Appendix F and the Type IV contract prohibit scalar or advisory manufacture of missing evidence. $\square$

Definition 1.37 (Advisory and Core stable bands). An advisory stable band is an interval on which declared advisory indicators satisfy an advisory stability policy. A Core Type IV stable band is an interval on which actual monitored Type IV diagnostics are certified zero throughout the band, supported by a universal artifact family or a theorem reducing the band claim to finite data.

Theorem 1.38 (Advisory stable band is not Core stable band). *An advisory stable band, including one supported by dense or extensive sampling, is not a Core Type IV stable band unless the actual diagnostic vanishing is verified throughout the band by the required artifact or reduction theorem.*

Proof. Advisory stability and diagnostic vanishing are different predicates on different typed data. Sampling does not supply the missing universal diagnostic statement without a reduction theorem. $\square$

1.10 Navigation quantities, AI outputs, and reproducibility

Definition 1.39 (Navigation potential). A navigation potential is an advisory map

$$\Phi : X \longrightarrow \mathbb{R} \cup \{+\infty\}$$

on a declared search or parameter space, used only to rank, prioritize, or route candidate regions. Its orientation, normalization, tie policy, and incomparable cases must be declared.

Theorem 1.40 (AI confidence, ranking, and potential are non-verdict). *No AI confidence value, model probability, ranking, recommendation, low value of Φ , high value of Φ , or ensemble agreement establishes mathematical validity, verifier-side readiness, or Core status.*

Proof. These quantities are proposer-side estimates or orderings. They are not repaired evaluations, comparison records, representatives, eligibility records, Type IV artifacts, ledger bounds, or gate verdicts. $\square$

Definition 1.41 (Reproducible advisory computation). An advisory computation is reproducible only when the record supplies immutable inputs, generator or model identity, version and configuration, randomness and seed policy when applicable, normalization, environment, output artifact, and replay procedure. Reproducibility establishes reconstruction of the declared output, not mathematical truth of an interpretation.

Proposition 1.42 (Stale advisory outputs are non-consumable). *If any content-addressed input, generator version, normalization, or declared scope changes, the prior advisory output is stale for the changed run and cannot be silently reused as current guidance or re-entry input.*

Proof. The changed dependency produces a different computation identity. Reuse without a new record would violate artifact identity and repair ancestry. $\square$

1.11 Advisory attachments and manifest schema

Definition 1.43 (Advisory attachment record). An advisory attachment to a certificate record contains at least:

- (i) attachment ID and parent certificate ID;
- (ii) advisory datum ID and lifecycle status;
- (iii) input scope, monitored degree, window, threshold or sweep domain;
- (iv) generator, normalization, codomain, and value;
- (v) artifact and provenance references;
- (vi) advisory interpretation and prohibited stronger reading;
- (vii) any re-entry candidate ID, kept distinct from the attachment;
- (viii) an explicit `non_verdict` flag unless a separate verified re-entry record exists.

Definition 1.44 (Human-readable and machine-readable equivalence policy). The mathematical appendix body is the source of theorem statements, proofs, and internal labels. The complete human-readable Claim Register is published as a separate audit sidecar, together with canonical JSON and CSV serializations. The sidecar forms must contain the same UID set and the same typing, source-binding, comparison-policy, permitted-use, forbidden-reading, failure-route, ancestry, and review fields. A mismatch blocks promotion, but the register is not reproduced inside the mathematical appendix.

Proposition 1.45 (Advisory attachments are non-verdict by default). *An advisory attachment does not affect pass, reject, undefined, or failure type unless a separate verified re-entry record changes the verifier-side projection.*

Proof. This is the attachment form of advisory noninterference, Theorem 1.19. $\square$

1.12 Optional quantitative collapse indicators

Definition 1.46 (Residual bar-mass indicator). For a finite repaired profile $\text{Eval}_{i,W,\tau}(F)$, a residual bar-mass indicator is an advisory statistic of the surviving barcode, such as a declared count, total finite length, weighted length, moment, or distributional functional. Infinite bars and divergent sums require an explicit convention; absent one, the value is undefined.

Definition 1.47 (Collapse-energy candidate). A collapse-energy candidate is an advisory functional

$$\mathcal{E}_{i,W,\tau}^{\text{adv}}(F)$$

with declared domain, normalization, range, treatment of infinite bars, and threshold-sweep semantics. The term “energy” does not imply positivity, coercivity, lower semicontinuity, Lyapunov monotonicity, or equivalence to a Core obstruction unless separately proved.

Theorem 1.48 (Collapse energy is not B-Gate⁺). *No collapse-energy candidate replaces the exact repaired topological clause, eligible categorical clause, Type IV artifact and diagnostic clause, comparison-mode obligations, or strict metric budget condition of B-Gate⁺.*

Proof. B-Gate⁺ is a typed conjunction of verifier-side obligations. A single advisory scalar is not definitionally equal to that conjunction, and no such equivalence is asserted here. $\square$

1.13 Appendix H package theorem

Theorem 1.49 (Appendix H advisory-layer package). *Within the Technical Extension layer, the following package is available.*

- (i) *Advisory objects are typed, versioned, and non-verdict by default.*
- (ii) *The term invariant requires a declared invariance theorem.*
- (iii) *Band integrals require measurable profiles and declared finite or extended-value semantics.*
- (iv) *Uncalibrated advisory values are not comparison evidence.*
- (v) *Sample stability does not certify a continuum band.*
- (vi) *Advisory attachments do not alter a verdict when the verifier-side projection is fixed.*
- (vii) *Advisory values cannot compensate for failed or undefined required obligations.*
- (viii) *Re-entry constructs a new verifier-side record and requires a theorem or artifact witness.*
- (ix) *Budget conversion requires an actual scope-matched discrepancy upper bound.*
- (x) *Gate-adjacent quantities and warning flags remain non-verdict.*
- (xi) *Type IV proxies do not replace an actual comparison morphism or terminal-generic diagnostics.*
- (xii) *Advisory stable bands do not become Core stable bands without universal diagnostic evidence.*
- (xiii) *AI confidence, ranking, and navigation potentials are proposer-side only.*
- (xiv) *Reproducibility proves replay of an output, not mathematical truth.*
- (xv) *Residual mass and collapse-energy candidates remain advisory unless separately promoted by theorem-backed re-entry.*

Proof. Typing and invariance are supplied by Definitions 1.4–1.14. Measurable integration and sampled-band limits are given by Definitions 1.11–1.16 and Proposition 1.17. Noninterference and non-compensation are Theorems 1.19 and 1.21. Re-entry and metric conversion are controlled by Theorem 1.24, Proposition 1.25, and Theorem 1.27. Type IV and stable-band separation follow from Theorems 1.35 and 1.38; AI and reproducibility boundaries follow from Theorem 1.40 and Proposition 1.42. $\square$

1.14 Explicit exclusions

The following are not asserted.

- No advisory indicator is a Core gate, gate verdict, or failure type.
- No advisory score replaces $\text{Eval}_{1,w,\tau}(F) = 0$, an eligible Ext^1 -clause, an actual Type IV artifact, or the strict metric budget condition.
- No declaration, field rename, confidence score, or favorable ranking performs verifier-side re-entry.
- No zero advisory value implies zero metric defect, exact equality, Type IV cleanliness, or positive threshold reserve.
- No sampled threshold sweep proves a whole-band statement without a reduction theorem.
- No Type IV proxy is the actual computation of $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}})$.
- No numerical pair is assigned when Type IV is `not_invoked` or `undefined`.
- No normative Latin u is used for the Type IV cokernel diagnostic; ν is canonical.
- No global exactness or global non-expansiveness of hard positive-threshold deletion is assumed.
- No global strict filtered collapse functor is assumed.
- No AI-generated output, human preference, or platform ranking changes a verifier-side verdict.
- No technical-extension theorem silently enlarges the Core.

1.15 Provenance and status

Definition 1.50 (Artifact closure). The integrated mathematical appendix body and the separately released audit sidecars form one release package. The sidecars carry machine registers, reproducibility records, and repair metadata; they are not mathematical proof and are not embedded in the appendix body.

This appendix depends on:

- (i) Chapters 1–3 and 7–8;
- (ii) Appendix A for repaired threshold-gap semantics;
- (iii) Appendices D and F for Type IV and status semantics;
- (iv) Appendix G for manifest and proof-object requirements.

Its role is to provide the advisory and spectral-indicator discipline without enlarging the Core or asserting an external-domain theorem.

Technical Extension definitions: Definitions 1.1, 1.2, 1.3, 1.4, 1.6, 1.7, 1.8, 1.10, 1.11, 1.12, 1.13, 1.14, 1.16, 1.18, 1.22, 1.23, 1.26, 1.29, 1.31, 1.33, 1.34, 1.37, 1.39, 1.41, 1.43, 1.44, 1.46, 1.47, 1.50.

Technical Extension results: Theorems 1.19, 1.21, 1.24, 1.27, 1.35, 1.38, 1.40, 1.48, 1.49. Propositions 1.5, 1.9, 1.15, 1.17, 1.25, 1.28, 1.30, 1.32, 1.36, 1.42, 1.45. Corollaries 1.20.

Explicit exclusions: Section 1.14.

Final status. Document role: Technical Extension appendix. Evidence status: mixed proved, conditional where explicitly stated, and operational. Formal CR-level promotion remains pending until the complete Technical Extension sequence is audited.

2 Appendix I: Grid-to-Continuum and Discretization Stability

2.1 Scope and relation to Appendices A, G, and H

This appendix governs the use of grids, meshes, samples, numerical fields, truncations, and discretized observables when they approximate Core-readable persistence data. It does not assert that a discrete computation is automatically a continuum theorem.

The central rule is

Discretized evidence enters a target claim only through a typed realization, a certified transfer, and a complete error ledger.

Appendix A controls threshold-gap transfer, Appendix G controls audit-readable artifacts, and [Appendix H](#) controls advisory noninterference and theorem-backed re-entry.

Definition 2.1 (Discretization sovereignty principle). Discretized data may propose, approximate, or support verifier-side records. They do not determine, compensate for, mutate, or retroactively repair a Core verdict unless a separately verified construction produces the required verifier-side record.

Definition 2.2 (No computation-only continuum upgrade principle). A grid, mesh, sample, solver output, computed barcode, refinement plot, or numerical confidence value does not prove a statement about a declared continuum target merely by being accurate-looking, stable-looking, or reproducible. A continuum conclusion requires an identified target, a target realization, and an exact or metric-certified transfer witness at the claimed scope.

Definition 2.3 (Immutable dependency baseline). The statements of this appendix are interpreted relative to:

- (i) AK-HPDST v19.0.0 Main Chapters 1–8;
- (ii) Foundation Appendices A–G;
- (iii) CR-v19.0.0-r3;
- (iv) [Appendix H](#) and the promoted CM/TB records, only at their registered strengths.

Only the registered mathematical strengths of these dependencies are imported here.

Definition 2.4 (Comparison policy and runtime-mode calculus). A target-directed comparison is consumed in exactly one of three runtime modes: *exact*, *metric-gap-certified*, or *not compared*. Exact mode requires an evidenced identity or isomorphism of the actual compared objects. Metric-gap-certified mode requires an actual discrepancy bound together with every threshold-gap and crop hypothesis used by the transfer theorem. Not-compared mode licenses statements only about the identified source or surrogate and cannot discharge a target-comparison obligation. The machine-readable register records the corresponding claim-level policy; that schema vocabulary is maintained in the audit sidecar.

Proposition 2.5 (Missing target or witness blocks continuum interpretation). *If the continuum target, its realization, or the selected comparison witness is missing or undefined, every continuum-level conclusion is undefined. A discrete-only result may remain valid for the identified discrete realization.*

Proof. The missing item is a typed dependency of the continuum statement. Core readiness is conjunctive and does not allow advisory or scalar compensation for a missing non-scalar obligation. $\square$

2.2 Typed targets, discretizations, and realizations

Definition 2.6 (Continuum target record). A continuum target record identifies an object X , the mathematical category in which it lives, the claimed parameter and spatial scope, and the exact statement intended to be transferred from discretized evidence. The symbol X alone is not a target record.

Definition 2.7 (Discretized object record). A discretized object record identifies X_h , its scale or resolution parameter h , the construction method, boundary and truncation conventions, and immutable input artifacts. Different meshes, seeds, tie policies, or solver tolerances define different records.

Definition 2.8 (Directed discretization family). A discretization family is a directed system $\{X_h\}_{h \in H}$ together with a declared refinement order. The notation $h \rightarrow 0$ is used only when a cofinal refinement map and its meaning are specified. A finite list of meshes is not by itself a convergence object.

Definition 2.9 (Discrete realization record). A discrete realization record supplies a declared map

$$\mathfrak{R}_h(X_h) \in \text{FiltCh}(k) \quad \text{or} \quad \mathfrak{R}_h(X_h) \in \text{Pers}_k^{\text{cons}},$$

together with all filtration, coefficient, boundary, tie, and constructibility conventions needed to form repaired profiles.

Definition 2.10 (Continuum realization record). A continuum realization record supplies

$$\mathfrak{R}(X) \in \text{FiltCh}(k) \quad \text{or} \quad \mathfrak{R}(X) \in \text{Pers}_k^{\text{cons}},$$

or a theorem-backed constructible replacement on the monitored scope. If only a replacement is available, the replacement error is a separate ledger component.

Definition 2.11 (Target identity and surrogate-only mode). A run is in surrogate-only mode when $\mathfrak{R}_h(X_h)$ is defined but no verified comparison to $\mathfrak{R}(X)$ is supplied. In that mode, all verdicts are explicitly about the surrogate. A basename, visual similarity, or shared problem label does not identify the surrogate with the continuum target.

Proposition 2.12 (Raw numerical output is not a Core object). *A matrix, array, point cloud, image, mesh file, or solver output is not directly a repaired persistence profile or a B-Gate⁺ record. It becomes Core-readable only through a declared realization and the corresponding evaluation artifacts.*

Proof. The raw object lacks the typed filtered or persistence structure, evaluation scope, and artifact schema required by Chapters 1–3 and Appendix G. □

2.3 Error ledger, comparison evidence, and numerical modes

Definition 2.13 (Pre-threshold discretization profiles). For monitored (i, W) , define the actual pre-threshold profiles

$$B_h := W_W \mathbf{P}_i(\mathfrak{R}_h(X_h)), \quad B := W_W \mathbf{P}_i(\mathfrak{R}(X)).$$

Every metric transfer statement in this appendix is bound to the immutable identities of these profiles, not merely to the names X_h and X .

Definition 2.14 (Discretization comparison record). A discretization comparison record contains: the source and target profile identities; comparison mode; window and degree; threshold when used; an exact witness or a bound

$$d_B(B_h, B) \leq \varepsilon_h;$$

the metric convention for infinite bars; the source theorem or verified computation; crop and model contributions; and failure behavior. In metric-gap-certified mode it also contains the two-sided closed-band evidence for the same ε_h .

Definition 2.15 (Typed discretization ledger). Let $(V, \oplus, 0_V, \preceq)$ be the declared metric-ledger quantale. A discretization ledger may include typed components

$$\mathfrak{d}_{\text{model}}, \mathfrak{d}_{\text{samp}}, \mathfrak{d}_{\text{num}}, \mathfrak{d}_{\text{crop}}, \mathfrak{d}_{\text{trunc}}, \mathfrak{d}_{\text{round}} \in V,$$

but only when each component has an actual scope-matched discrepancy witness. Their certified aggregate is denoted $\mathfrak{d}_{\text{disc}}$.

Definition 2.16 (Dependency-aware aggregation and no double counting). A ledger aggregation record contains a dependency graph identifying which components are independent bounds, nested bounds, or alternative bounds. The aggregate $\mathfrak{d}_{\text{disc}}$ must be justified by an aggregation theorem. The same discrepancy may not be counted twice merely because it appears under two labels.

Definition 2.17 (Scalarized discrepancy witness). A scalarized witness consists of $\mathfrak{d}_{\text{disc}} \in V$, a declared scalarization $\text{val}_B : V \rightarrow [0, +\infty]$, and evidence that

$$d_B(B_h, B) \leq \varepsilon_h := \text{val}_B(\mathfrak{d}_{\text{disc}}).$$

A scalar number without the profile identities and supporting theorem or artifact is not a witness.

Proposition 2.18 (Actual-profile binding is mandatory). *A discrepancy estimate computed for different windows, thresholds, coefficient fields, filtrations, normalization policies, or source artifacts cannot certify the comparison of B_h and B unless a proved transport theorem connects the records.*

Proof. The estimate and the claimed comparison otherwise have different typed inputs. Reusing the number would be an identity collision or a silent default theorem. $\square$

Proposition 2.19 (Estimator-only values do not enter the ledger). *An empirical error estimate, confidence interval without a coverage theorem, residual plot, or learned predictor remains advisory until a reliability theorem or verified computation proves the required discrepancy upper bound on the declared scope.*

Proof. A scalar estimate cannot manufacture the non-scalar comparison evidence required by the ledger and comparison record. $\square$

Definition 2.20 (Numerical evidence modes). A numerical artifact is labeled exactly one of:

- (i) `exact`: symbolic, rational, integer, or otherwise exact computation with replay evidence;
- (ii) `interval_certified`: outward-rounded or theorem-backed enclosure proving the claimed bound;
- (iii) `heuristic`: floating or learned computation without a certified enclosure.

Only the first two may discharge a numerical theorem obligation, and only at the registered strength.

2.4 Crop-aware and threshold-gap-safe transfer

Proposition 2.21 (No unconditional window non-amplification). *This appendix does not assert that passage from source-level persistence data to a bounded window is globally non-amplifying under every shift, endpoint, crop, or extension convention. A transfer theorem must compare the actual windowed profiles or ledger the crop contribution.*

Proof. The v19 contract records window-edge and crop obligations separately and rejects unconditional crop-collapse commutation. Therefore a source-level bound alone is insufficient without the declared window theorem. $\square$

Definition 2.22 (Crop-aware window comparison). A crop-aware comparison either proves the bottleneck bound directly for B_h and B , or supplies source-level evidence together with a crop component $\mathbf{d}_{\text{crop}}$ and an aggregation theorem yielding the bound for the actual windowed profiles.

Proposition 2.23 (No global non-expansiveness of hard deletion). *There is no unconditional rule*

$$d_B(T_\tau^{\text{bar}}(B_h), T_\tau^{\text{bar}}(B)) \leq d_B(B_h, B)$$

for arbitrary barcodes at positive threshold. Any discretization transfer through hard deletion must use exact mode or the v19 threshold-gap hypotheses.

Proof. This is the registered rejection of global non-expansiveness, specialized to discretization comparisons. $\square$

Theorem 2.24 (Gap-certified discretization transfer). *Let B_h, B be finite barcodes, possibly containing infinite bars, under the registered bottleneck convention. Let $\tau > 0$, $\varepsilon_h \geq 0$, assume*

$$d_B(B_h, B) \leq \varepsilon_h,$$

and assume no finite bar of either profile has length in

$$[\tau - \varepsilon_h, \tau + \varepsilon_h].$$

Then

$$d_B(T_\tau^{\text{bar}}(B_h), T_\tau^{\text{bar}}(B)) \leq \varepsilon_h.$$

Proof. Apply the sharp two-sided threshold-gap-safe stability theorem of Chapter 2 and Appendix A to the actual pre-threshold profiles. $\square$

Proposition 2.25 (Closed threshold band is mandatory at the certified radius). *A clearance check at radius smaller than ε_h , or on only one of B_h, B , does not certify Theorem 2.24. Exact equality may still be recorded in exact mode, but it is not a positive-radius robustness certificate.*

Proof. The source theorem is explicitly two-sided and uses the closed band at the same discrepancy radius. Weakening either condition changes the theorem. $\square$

Corollary 2.26 (Post-threshold metric transfer). *Under Theorem 2.24,*

$$d_B(\text{Eval}_{i,w,\tau}(\mathfrak{R}_h(X_h)), \text{Eval}_{i,w,\tau}(\mathfrak{R}(X))) \leq \varepsilon_h.$$

In the declared constructible one-parameter regime, the same bound may be written with interleaving distance only under the registered barcode-isometry convention.

Proposition 2.27 (Exact discretization comparison). *If the actual pre-threshold profiles are evidenced equal or isomorphic under the declared barcode convention, deterministic threshold deletion gives equal repaired profiles. This is exact comparison mode and does not assert positive-radius robustness.*

Proof. Apply the same deterministic deletion rule to the identified profiles. $\square$

2.5 Gate verdict transfer and the half-threshold reserve

Definition 2.28 (Discrete-only verdict). A discrete-only verdict asserts a gate or diagnostic statement solely for $\mathfrak{R}_h(X_h)$. It does not contain the phrase “for X ”, “continuum”, or an equivalent target-level reading unless a transfer witness is attached.

Proposition 2.29 (Continuum verdict requires a transfer witness). *A continuum pass or reject cannot be inferred from a discrete verdict solely from small ε_h or adjusted-budget pass. The exact logical property being transferred must have its own theorem.*

Proof. Metric proximity is a quantitative relation, whereas vanishing, categorical eligibility, and diagnostic status are typed predicates. No general transport follows without an appropriate theorem. $\square$

Theorem 2.30 (Threshold-separated vanishing transfer). *Let B_h, B be finite barcodes, possibly containing infinite bars, and set*

$$E_h := T_\tau^{\text{bar}}(B_h), \quad E := T_\tau^{\text{bar}}(B).$$

Assume $d_B(E_h, E) \leq \varepsilon_h$ and $2\varepsilon_h < \tau$. Then

$$E_h = 0 \iff E = 0.$$

Proof. Suppose $E_h = 0$. Any finite bar of E has length strictly greater than τ , so its cost to the diagonal is greater than $\tau/2 > \varepsilon_h$. An infinite bar has infinite cost to the diagonal under the registered convention. Hence no bar of E can be matched to the empty barcode at cost at most ε_h , unless $E = 0$. The converse is symmetric. $\square$

Proposition 2.31 (Robust nonvanishing witness). *For finite barcodes B_h, B , if $d_B(E_h, E) \leq \varepsilon_h$ and E_h contains an infinite bar or a finite bar of length greater than $2\varepsilon_h$, then $E \neq 0$. The symmetric statement also holds.*

Proof. Such a bar cannot be matched to the diagonal within cost ε_h , so every bottleneck matching at that cost must match it to a bar on the other side. $\square$

Corollary 2.32 (Zero/nonzero classification equivalence below half-threshold). *Under Corollary 2.26 and $2\varepsilon_h < \tau$, the discretized and continuum repaired profiles have the same zero/nonzero classification.*

Proposition 2.33 (Budget pass alone does not transfer vanishing). *The inequality*

$$\text{val}_B(\mathfrak{d}_{\text{disc}}) < \text{gap}_\tau(\{B_h, B\})$$

licenses metric transfer through hard deletion, but does not by itself imply $E_h = 0 \iff E = 0$. The half-threshold reserve or another vanishing-transfer theorem is additionally required.

Proof. A nonzero bar of length at most $2\varepsilon_h$ may be matched to the diagonal. Metric closeness alone therefore need not preserve emptiness. $\square$

Theorem 2.34 (Conditional target-side B-Gate transfer). *Assume: (i) an exact or gap-certified comparison record; (ii) the hypotheses of Corollary 2.32 for the topological clause; (iii) every invoked target-side categorical obligation is independently verified or transported by a registered bridge theorem; (iv) the actual continuum Type IV artifact and diagnostics satisfy their contract; and (v) the complete scalarized ledger passes. Then a discrete topological pass may participate in a target-side B-Gate⁺ pass. Omitting any invoked clause leaves the target verdict undefined or rejected according to its failure route.*

Proof. The topological clause transfers by Corollary 2.32; all other clauses pass by hypothesis. Apply the conjunctive B-Gate⁺ contract and non-compensation rule. $\square$

Proposition 2.35 (Persistence proximity does not transfer Ext discharge). *A bottleneck or interleaving bound between repaired persistence profiles does not by itself identify admissible representatives or transfer an Ext¹-vanishing statement. Such a transfer requires the registered representative and bridge hypotheses.*

Proof. The categorical clause depends on non-scalar realization and eligibility data not manufactured by a persistence distance. $\square$

2.6 Convergence, reconstruction, and eventual readiness

Definition 2.36 (Pointwise repaired-profile convergence). For fixed (i, W, τ) , a family converges to the target when there are certified comparison records with radii $\varepsilon_h \rightarrow 0$ for the actual repaired profiles. This definition is pointwise in the declared parameters.

Definition 2.37 (Uniform monitored-family convergence). For a declared monitored family $\mathcal{M}$ of degrees, windows, and thresholds, convergence is uniform when a certified rate $\rho(h) \rightarrow 0$ bounds every comparison in $\mathcal{M}$. Pointwise certificates do not imply a uniform certificate on an infinite family.

Proposition 2.38 (Discrete Cauchy evidence is not target convergence). *Pairwise convergence of B_h to one another establishes at most a Cauchy property in the declared metric. It does not identify a limit with $B = W_W \mathbf{P}_i(\mathfrak{R}(X))$, nor prove that such a target realization exists.*

Proof. Completeness, existence of a limit object, and identification with the declared target are separate obligations. $\square$

Definition 2.39 (Reconstruction witness). A reconstruction witness consists of: a theorem giving existence of a limit in a declared complete ambient category or metric space; a construction of that limit; and an exact or certified identification of the limit with the continuum realization on the monitored scope.

Definition 2.40 (Certified convergence rate). A certified rate is a function $\rho(h) \rightarrow 0$ together with replayable evidence that the actual pre-threshold or repaired-profile discrepancy is bounded by $\rho(h)$ on the stated scope. A fitted decay curve without a reliability theorem is advisory only.

Theorem 2.41 (Eventual two-sided gap readiness from target clearance). *Let B be a finite barcode and define*

$$g := \text{clear}_\tau(B) := \inf\{|\ell(I) - \tau| : I \text{ a finite bar of } B\},$$

with $g = +\infty$ if B has no finite bars. Assume $g > 0$ and $d_B(B_h, B) \leq \varepsilon_h \rightarrow 0$. For every sufficiently small h satisfying

$$3\varepsilon_h < \min\{g, \tau\},$$

neither B nor B_h has a finite bar in $[\tau - \varepsilon_h, \tau + \varepsilon_h]$. Hence the pair is eventually eligible for Theorem 2.24.

Proof. The target exclusion follows from $\varepsilon_h < g$. Suppose a finite bar J of B_h lies in the stated band. It cannot match the diagonal at cost ε_h , because $\ell(J) \geq \tau - \varepsilon_h > 2\varepsilon_h$. Thus it matches a finite target bar I , and endpoint displacement gives $|\ell(I) - \ell(J)| \leq 2\varepsilon_h$. Therefore $|\ell(I) - \tau| \leq 3\varepsilon_h < g$, a contradiction. $\square$

Proposition 2.42 (Refinement can eventually restore scalar budget admissibility). *Assume a fixed target gap $G > 0$, a base scalar bound $b < G$, a certified rate $\varepsilon_h \rightarrow 0$, and an aggregation witness*

$$\text{val}_B(\mathfrak{d}_{\text{base}} \oplus \mathfrak{d}_{\text{disc}}(h)) \leq b + \varepsilon_h.$$

Then the adjusted scalar budget is below G for sufficiently small h .

Proof. Choose h so that $\varepsilon_h < G - b$. The displayed inequality then gives the strict budget condition. $\square$

2.7 Sampling, aliasing, and numerical certification

Definition 2.43 (Sampling operator record). A sampling record identifies the operator $S_h : X \mapsto X_h$, sample locations or cells, randomness and seed policy, boundary behavior, antialiasing or quadrature rule, and the target norm or metric in which a reliability statement is claimed.

Definition 2.44 (Reliability-certified a posteriori estimator). An estimator η_h is reliability-certified when a theorem or verified computation proves, for the actual records,

$$d_B(B_h, B) \leq \eta_h$$

or proves a source-level bound that is validly aggregated into that profile bound. Effectivity or lower bounds are optional; reliability is mandatory for ledger use.

Proposition 2.45 (Aliasing warning is non-verdict). *A detected or suspected aliasing pattern is a warning, not a discrepancy bound, gate verdict, or failure type. It becomes verifier-side evidence only through a separately certified estimator, refinement theorem, or exact reconstruction record.*

Proof. The warning does not supply the typed upper bound or target comparison required by the verifier. $\square$

Definition 2.46 (Under-resolved status). A run is `under_resolved` when a required reliability bound is unavailable, the certified radius violates a threshold or budget reserve, or the chosen resolution cannot support the claimed monitored scope. This is a Technical Extension lifecycle status, not a Core atomic status. It blocks the target-level claim and is not silently converted to pass or reject.

Definition 2.47 (Floating-point and filtration-tie policy). A numerical persistence artifact records number format, rounding mode, tolerance, comparison policy for nearly equal filtration values, ordering of exact ties, treatment of bars with length exactly τ , and whether interval enclosures cover all branching decisions. Changing any item creates a new artifact identity.

Definition 2.48 (Adaptive refinement stopping rule). An adaptive stopping rule identifies the certified quantity being tested, its reliability theorem, the target reserve, the stopping inequality, maximum refinement, and failure behavior. A rule based only on consecutive outputs appearing similar is advisory.

Proposition 2.49 (Observed refinement plateau is advisory only). *Stability of computed barcodes or scores over finitely many refinements does not prove convergence, error below tolerance, or absence of unresolved features without a reduction or reliability theorem.*

Proof. Finite observations do not supply a universal tail bound and may reflect correlated aliasing or solver stagnation. $\square$

2.8 Discretized towers and Type IV transfer

Definition 2.50 (Discretized tower comparison morphism). For each monitored (i, W, τ, h) , a discretized Type IV record contains an actual persistence morphism

$$\phi_{i,W,\tau}^{(h)} : M_{i,W,\tau}^{(h)} \longrightarrow N_{i,W,\tau}^{(h)},$$

with immutable source and target identities. A scalar proxy or diagram caption is not the morphism.

Definition 2.51 (Discretized terminal-generic diagnostics). If the discretized source, target, kernel, and cokernel profiles are terminal-tame on a common declared tail, define

$$\mu_{i,W,\tau}^{(h)} := \text{tgdim}_W(\ker \phi_{i,W,\tau}^{(h)}), \quad \nu_{i,W,\tau}^{(h)} := \text{tgdim}_W(\text{coker } \phi_{i,W,\tau}^{(h)}).$$

If the morphism or terminal-tameness evidence is missing, the diagnostic status is undefined; no numerical pair is imputed.

Definition 2.52 (Continuum tower comparison morphism). The corresponding continuum record contains an actual morphism

$$\phi_{i,W,\tau} : M_{i,W,\tau} \longrightarrow N_{i,W,\tau}$$

and its own terminal-tameness evidence. A family of discretized morphisms does not constitute this artifact by itself.

Definition 2.53 (Type IV diagnostic transfer witness). A Type IV transfer witness supplies morphisms $a_h : M^{(h)} \rightarrow M$, $b_h : N^{(h)} \rightarrow N$ forming a commuting square with $\phi^{(h)}$ and ϕ , together with a common terminal tail on which the induced maps

$$\ker \phi^{(h)} \longrightarrow \ker \phi, \quad \text{coker } \phi^{(h)} \longrightarrow \text{coker } \phi$$

are isomorphisms, or an exact registered theorem giving the same terminal-fiber conclusion.

Proposition 2.54 (Discretized Type IV zero does not imply continuum zero). *At a fixed h , the equality*

$$(\mu_{\text{Collapse}}^{(h)}, \nu_{\text{Collapse}}^{(h)}) = (0, 0)$$

does not imply $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$ for the continuum tower without a Type IV transfer witness.

Proof. Discretization may omit terminal-cofinal kernel or cokernel defects, and kernel/cokernel formation is not automatically transported by profile-level metric proximity. $\square$

Theorem 2.55 (Terminal-fiber diagnostic transfer). *Under Definition 2.53, the discretized and continuum terminal-generic dimensions agree degreeewise:*

$$\mu_{i,W,\tau}^{(h)} = \mu_{i,W,\tau}, \quad \nu_{i,W,\tau}^{(h)} = \nu_{i,W,\tau}.$$

Consequently, discretized certified zero transfers to continuum certified zero on the witnessed scope.

Proof. Terminal-generic dimension is invariant under isomorphism on a common terminal tail. Apply this to the induced kernel and cokernel isomorphisms. $\square$

Proposition 2.56 (Missing continuum tower artifact is undefined). *If continuum Type IV is invoked but $\phi_{i,W,\tau}$ or its terminal-tameness evidence is missing, the target-side Type IV clause is undefined. Discretized diagnostics, even if zero for all tested meshes, do not repair the missing artifact.*

Proof. The missing morphism is a non-scalar proof obligation and cannot be manufactured by numerical repetition or a proxy. $\square$

2.9 Artifacts, manifest schema, and register parity

Definition 2.57 (Discretization artifact bundle). A reproducible bundle contains, as applicable:

- (i) target, surrogate, mesh, sample, and realization identities;
- (ii) coefficient field, numerical evidence mode, solver, libraries, environment, and seed;
- (iii) filtration, tie, boundary, window, threshold, and truncation policies;
- (iv) pre- and post-threshold profiles and barcode or module artifacts;
- (v) comparison matching or enclosure and closed-band evidence;
- (vi) component ledger, dependency graph, and aggregation witness;
- (vii) invoked Type IV morphisms and terminal-tameness artifacts;
- (viii) replay command and content digests.

Definition 2.58 (Discretization manifest entry). A manifest entry serializes the artifact bundle, comparison mode, target scope, statuses, source bindings, failure routes, and every theorem or computation used to convert discretized data into verifier-side evidence. Omitted fields are not supplied by convention.

Proposition 2.59 (Artifacts are necessary for audit-readable discretization evidence). *A target-directed certificate relying on discretized evidence is not audit-readable when an independent verifier cannot reconstruct the actual surrogate, realization, comparison profiles, numerical mode, discrepancy witness, threshold-band evidence, and ledger contribution.*

Proof. These data are required to identify the mathematical objects and replay the exact inference. Without them, the verifier cannot distinguish a valid comparison from a stale or mismatched computation. $\square$

Definition 2.60 (Human-readable and machine-readable equivalence policy). The mathematical appendix body is the source of theorem statements, proofs, and internal labels. The complete human-readable Claim Register is published as a separate audit sidecar, together with canonical JSON and CSV serializations. The sidecar forms must contain the same UID set and the same typing, source-binding, comparison-policy, permitted-use, forbidden-reading, failure-route, ancestry, and review fields. A mismatch blocks promotion, but the register is not reproduced inside the mathematical appendix.

2.10 Appendix I package theorem

Theorem 2.61 (Appendix I discretization-stability package). *Within the Technical Extension layer:*

- (i) *discretized and continuum objects are separate typed records;*
- (ii) *target claims use exact, metric-gap-certified, or not-compared mode;*
- (iii) *the actual windowed pre-threshold profiles are the objects of metric comparison;*
- (iv) *crop, sampling, numerical, model, truncation, and rounding errors enter only through evidenced ledger components;*
- (v) *hard threshold deletion is transported only by exact comparison or the sharp two-sided closed-band theorem;*
- (vi) *post-threshold metric closeness and gate-classification transfer are distinct obligations;*
- (vii) *$2\varepsilon_h < \tau$ transfers the zero/nonzero classification after threshold deletion;*

- (viii) *target B-Gate⁺ still requires every invoked non-topological clause;*
- (ix) *Cauchy behavior, refinement plateaus, and advisory convergence plots do not identify a continuum target;*
- (x) *positive target clearance plus certified convergence gives eventual two-sided gap readiness;*
- (xi) *aliasing and heuristic estimators remain non-verdict without reliability evidence;*
- (xii) *Type IV transfer requires actual morphisms and terminal-fiber comparison;*
- (xiii) *content-addressed artifacts and register parity are mandatory for promotion.*

Proof. Typed target and realization separation is established in Definitions 2.6–2.11. The ledger and numerical-evidence requirements are Definitions 2.15–2.20. Exact and metric-gap-certified threshold transfer are Proposition 2.27 and Theorem 2.24. Classification transfer is Theorem 2.30 and Corollary 2.32; the full-gate boundary is Theorem 2.34. Convergence and eventual readiness are controlled by Proposition 2.38 and Theorem 2.41. Type IV transfer is Theorem 2.55, while audit readability is Proposition 2.59. $\square$

2.11 Explicit exclusions

The following are not asserted.

- No grid, mesh, point sample, solver output, barcode image, or refinement plot is a continuum proof by itself.
- No source-level discrepancy bound automatically survives bounded-window passage without direct windowed evidence or crop accounting.
- No global non-expansiveness of positive-threshold hard deletion is assumed.
- No one-sided clearance record is a two-sided comparison certificate.
- No adjusted-budget pass alone transfers a zero profile or B-Gate⁺ verdict.
- No persistence distance manufactures an admissible representative, an Ext¹ bridge, a tower morphism, or terminal-tameness evidence.
- No finite refinement prefix proves convergence or a uniform tail bound.
- No Cauchy discrete family is identified with a declared target without reconstruction evidence.
- No aliasing warning or learned error predictor is a ledger component without a reliability theorem.
- No fixed-mesh Type IV zero, and no numerical pair, replaces the continuum comparison artifact.
- No normative Latin u denotes the cokernel diagnostic; v is canonical.
- No package closure or successful replay proves a stronger mathematical statement than the registered claims.

2.12 Provenance and status

Definition 2.62 (Artifact closure). The integrated mathematical appendix body and the separately released audit sidecars form one release package. The sidecars carry machine registers, reproducibility records, and repair metadata; they are not mathematical proof and are not embedded in the appendix body.

This appendix depends on:

- (i) Chapters 1–3, 5–7;

- (ii) Appendix A for sharp threshold-gap transfer;
- (iii) Appendices D, F, and G for diagnostics, status, and manifests;
- (iv) [Appendix H](#) for advisory noninterference and re-entry.

Its role is to provide the grid-to-continuum and discretization-stability discipline without enlarging the Core or asserting an external-domain theorem.

Technical Extension definitions: Definitions [2.1](#), [2.2](#), [2.3](#), [2.4](#), [2.6](#), [2.7](#), [2.8](#), [2.9](#), [2.10](#), [2.11](#), [2.13](#), [2.14](#), [2.15](#), [2.16](#), [2.17](#), [2.20](#), [2.22](#), [2.28](#), [2.36](#), [2.37](#), [2.39](#), [2.40](#), [2.43](#), [2.44](#), [2.46](#), [2.47](#), [2.48](#), [2.50](#), [2.51](#), [2.52](#), [2.53](#), [2.57](#), [2.58](#), [2.60](#), [2.62](#).

Technical Extension results: Theorems [2.24](#), [2.30](#), [2.34](#), [2.41](#), [2.55](#), [2.61](#). Propositions [2.5](#), [2.12](#), [2.18](#), [2.19](#), [2.21](#), [2.23](#), [2.25](#), [2.27](#), [2.29](#), [2.31](#), [2.33](#), [2.35](#), [2.38](#), [2.42](#), [2.45](#), [2.49](#), [2.54](#), [2.56](#), [2.59](#). Corollaries [2.26](#), [2.32](#).

Explicit exclusions: Section [2.11](#).

Final status. Document role: Technical Extension appendix. Evidence status: mixed proved, conditional where explicitly stated, and operational. Formal CR-level promotion remains pending until the complete Technical Extension sequence is audited.

3 Appendix J: Betti Integral and Measurement-Level Stability

3.1 Scope and relation to Appendices H and I

This appendix develops Betti curves, Betti integrals, numerical integration certificates, and measurement-level stability for repaired persistence profiles. These quantities summarize evidence but are not, by themselves, Core gates or metric-ledger elements.

The central rule is

A scalar measurement acquires verifier authority only through an explicit theorem-backed discharge or type conversion.

[Appendix H](#) supplies the advisory/non-verdict boundary, while [Appendix I](#) supplies the discretization and threshold-gap transfer required before a measured repaired profile can be consumed.

Definition 3.1 (Measurement sovereignty principle). Betti curves, Betti integrals, plots, quadrature values, confidence intervals, and aggregate persistence scores may summarize or route evidence. They do not determine, compensate for, mutate, or retroactively repair a Core verdict unless a separate verified construction produces the required verifier-side record.

Definition 3.2 (No scalar-to-verdict upgrade principle). A scalar measurement is not a persistence profile, representative, eligibility record, Type IV artifact, diagnostic pair, or metric-ledger element merely because it is exact, small, stable, or reproducible. Any change of semantic type requires an explicit theorem or artifact-backed conversion at the claimed scope.

Definition 3.3 (Immutable dependency baseline). The statements of this appendix are interpreted relative to:

- (i) AK-HPDST v19.0.0 Main Chapters 1–8;
- (ii) Foundation Appendices A–G;
- (iii) CR-v19.0.0-r3;
- (iv) [Appendix H](#) and [Appendix I](#), and the promoted CM/TB records, only at their registered strengths.

Only the registered mathematical strengths of these dependencies are imported here.

Definition 3.4 (Comparison policy and runtime-mode calculus). A target-directed comparison is consumed in exactly one of three runtime modes: *exact*, *metric-gap-certified*, or *not compared*. Exact mode requires an evidenced identity or isomorphism of the actual compared objects. Metric-gap-certified mode requires an actual discrepancy bound together with every threshold-gap and crop hypothesis used by the transfer theorem. Not-compared mode licenses statements only about the identified source or surrogate and cannot discharge a target-comparison obligation. The machine-readable register records the corresponding claim-level policy; that schema vocabulary is maintained in the audit sidecar.

Proposition 3.5 (Missing profile or witness blocks measurement interpretation). *If the repaired profile identity, measurement scope, numerical method, or a required conversion witness is missing or undefined, the stronger measurement-derived verifier conclusion is undefined. A raw scalar may remain an advisory observation about its identified computation.*

Proof. Each missing item is a typed dependency of the stronger statement. The v19 readiness calculus is conjunctive and forbids scalar compensation for a missing non-scalar obligation. $\square$

3.2 Repaired Betti curves and exact barcode mass

Definition 3.6 (Repaired-profile identity record). For a filtered object F , monitored degree i , bounded right-open window $W = [u, u')$, and $\tau > 0$, set

$$M_{i,W,\tau}(F) := \text{Eval}_{i,W,\tau}(F) = T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

A measurement record binds to the immutable identity of this actual repaired profile, including its source object, degree, window, threshold, barcode convention, and comparison mode.

Definition 3.7 (Windowed repaired Betti curve). For a pointwise finite-dimensional repaired profile $M = M_{i,W,\tau}(F)$, define

$$\beta_M(t) := \dim_k M(t), \quad t \in W.$$

The notation $\beta_{i,W,\tau}^F$ is an alias only after the profile identity record is fixed.

Theorem 3.8 (Measurability and finite integrability of constructible Betti curves). *If M is pointwise finite-dimensional and constructible on the bounded window W , with only finitely many interval summands meeting W , then β_M is a nonnegative integer-valued measurable step function and belongs to $L^1(W)$.*

Proof. A finite constructible barcode gives a finite decomposition into interval modules. The dimension function is the finite sum of their indicator functions on W , hence measurable, piecewise constant, bounded, and integrable on the finite-measure window. $\square$

Definition 3.9 (Raw Betti data). A Betti curve or aggregate computed before the fixed window-first repaired evaluation, or without an immutable repaired-profile identity, is *raw Betti data*. It is advisory by default and is not silently identified with $\beta_{M_{i,W,\tau}(F)}$.

Definition 3.10 (Extended Betti integral). For any measurable nonnegative Betti curve on W , define

$$\mathcal{B}_W(M) := \int_W \beta_M(t) dt \in [0, +\infty].$$

For a repaired profile, write $\mathcal{B}_{i,W,\tau}(F) := \mathcal{B}_W(M_{i,W,\tau}(F))$. A finite-valued claim requires the hypotheses of Theorem 3.8 or another registered integrability witness.

Theorem 3.11 (Barcode mass identity). *Let the finite barcode of M be the multiset $\{I_\alpha\}_{\alpha=1}^N$, with multiplicity. Then*

$$\mathcal{B}_W(M) = \sum_{\alpha=1}^N \lambda(I_\alpha \cap W),$$

where λ is Lebesgue length. Thus the Betti integral is exactly total clipped barcode mass, not merely a numerical analogy.

Proof. For interval modules, $\beta_M = \sum_\alpha \mathbf{1}_{I_\alpha \cap W}$. Integrating the finite nonnegative sum gives the stated identity. $\square$

Definition 3.12 (Normalized Betti integral). If $0 < \lambda(W) < \infty$, define

$$\overline{\mathcal{B}}_W(M) := \lambda(W)^{-1} \mathcal{B}_W(M).$$

Normalization changes units but does not add verifier authority.

Definition 3.13 (Multi-degree Betti measurement vector). For a finite declared degree set D , define

$$\mathbf{B}_{D,W,\tau}(F) := (\mathcal{B}_{i,W,\tau}(F))_{i \in D}.$$

Any scalarization of this vector must declare weights, units, treatment of infinite values, and its permitted use.

3.3 Exact zero detection and its sharp boundary

Definition 3.14 (Betti-faithful profile regime). A repaired profile M is *Betti-faithful on W* if every nonzero interval summand in its declared decomposition has strictly positive Lebesgue-length intersection with W , and no nonzero ephemeral or point-supported summand invisible to Lebesgue integration is admitted. Standard finite right-open nondegenerate barcode models satisfy this condition when recorded explicitly.

Theorem 3.15 (Zero Betti integral is equivalent to zero profile in the faithful regime). *For a finite Betti-faithful profile M ,*

$$\mathcal{B}_W(M) = 0 \iff M = 0.$$

Proof. If $M = 0$, the integral is zero. Conversely, the barcode mass identity is a finite sum of nonnegative terms, each strictly positive for every nonzero summand by Betti-faithfulness. A zero sum therefore has no nonzero summand. $\square$

Proposition 3.16 (Lebesgue-invisible nonzero profiles obstruct unconditional zero detection). *In an ambient persistence category admitting a point-supported module E_{t_0} with value k only at t_0 , one has $E_{t_0} \neq 0$ but*

$$\int_W \dim_k E_{t_0}(t) dt = 0.$$

Hence zero Betti integral does not imply zero profile without the Betti-faithful hypothesis.

Proof. The support is a singleton and therefore has Lebesgue measure zero, while the functor is nonzero at t_0 . $\square$

Theorem 3.17 (Exact degree-one Betti witness). *If the repaired degree-one profile $M_{1,W,\tau}(F)$ is finite and Betti-faithful, then*

$$\mathcal{B}_{1,W,\tau}(F) = 0 \iff \text{Eval}_{1,W,\tau}(F) = 0.$$

Proof. This is Theorem 3.15 applied to the identified repaired degree-one profile. $\square$

Proposition 3.18 (Admissible representative consequence). *If an admissible representative record supplies an exact identification*

$$\mathbf{P}_1(C_{\tau,W}F) \cong \text{Eval}_{1,W,\tau}(F),$$

then the hypotheses and zero conclusion of Theorem 3.17 imply $\text{PH}_1(C_{\tau,W}F) = 0$.

Proof. Transport zero across the registered profile isomorphism. $\square$

Proposition 3.19 (Eligible one-way Ext consequence). *Under the registered eligibility hypotheses of the one-way repaired PH_1 -to- Ext^1 bridge, the conclusion of Proposition 3.18 may discharge the corresponding Ext^1 -vanishing clause. No reverse implication is asserted.*

Proof. Apply the eligible one-way bridge at exactly its registered strength. The reverse direction is excluded by the registered counterexample boundary. $\square$

Proposition 3.20 (Positive exact Betti integral witnesses nonzero profile). *For any measurable nonnegative Betti curve,*

$$\mathcal{B}_W(M) > 0 \implies M \neq 0.$$

Proof. A zero profile has identically zero Betti curve and therefore zero integral. Take the contrapositive. $\square$

3.4 Discrete integration and numerical certification

Definition 3.21 (Quadrature record). A quadrature record contains a partition $P : u = t_0 < \dots < t_m = u'$, sample points $q_j \in [t_j, t_{j+1})$, weights, endpoint convention, arithmetic mode, the computed value

$$Q_P(\beta) := \sum_{j=0}^{m-1} \beta(q_j)(t_{j+1} - t_j),$$

and either an exactness witness or a certified error enclosure.

Theorem 3.22 (Breakpoint-resolving quadrature is exact). *If the partition contains every breakpoint of the constructible Betti curve and each q_j lies in the corresponding constant half-open cell, then*

$$Q_P(\beta_M) = \mathcal{B}_W(M).$$

Proof. The integrand is constant on every partition cell, so the integral on that cell equals its sampled value times cell length. Sum over cells. $\square$

Theorem 3.23 (Bounded-variation quadrature error). *Let β be a finite-variation step function on W , let $\|P\| := \max_j(t_{j+1} - t_j)$, and choose arbitrary $q_j \in [t_j, t_{j+1})$. Then*

$$\left| Q_P(\beta) - \int_W \beta(t) dt \right| \leq \|P\| \text{Var}_W(\beta).$$

Proof. On each cell, the absolute error is at most its length times the oscillation of β on that cell. The oscillation is bounded by the variation on the cell. Summing and using each cell length at most $\|P\|$ gives the result. $\square$

Proposition 3.24 (Sampled values alone do not reconstruct a Betti curve). *A finite set of sampled Betti values does not determine the intervening curve, its integral, or its exact zero status unless a breakpoint, variation, monotonicity, interpolation, or finite-reduction witness is supplied.*

Proof. Two constructible step functions may agree on the sampled set and differ on an unsampled subinterval of positive measure. $\square$

Definition 3.25 (Numerical evidence modes). A Betti computation uses exactly one numerical mode:

- (i) `exact`: exact endpoints and exact arithmetic, or a proof-equivalent symbolic computation;
- (ii) `interval_certified`: outward-rounded enclosures with a verified implementation and replay record;
- (iii) `heuristic`: floating-point, learned, visual, or empirically calibrated output without a proof-producing enclosure.

Reproducibility is required in every mode but does not upgrade heuristic evidence.

Proposition 3.26 (Heuristic quadrature is non-verdict). *A heuristic Betti integral, even if stable across reruns or refinements, is non-verdict data. It cannot discharge exact vanishing, nonvanishing, a metric budget, or a gate clause.*

Proof. The computation supplies no verified enclosure containing the true quantity and no theorem-backed type conversion. $\square$

3.5 Scalar enclosures and exact classification

Definition 3.27 (Scalar Betti enclosure record). A scalar enclosure record for $\mathcal{B}_W(M)$ contains exact or outward-rounded numbers $0 \leq L \leq U \leq +\infty$ and verified evidence that

$$L \leq \mathcal{B}_W(M) \leq U.$$

It also records the profile identity, units, method, numerical environment, and artifact hashes.

Theorem 3.28 (Exact classification from a nonnegative scalar enclosure). *For a valid scalar enclosure:*

- (i) if $U = 0$, then $\mathcal{B}_W(M) = 0$;
- (ii) if $L > 0$, then $\mathcal{B}_W(M) > 0$;
- (iii) if $L = 0 < U$, the exact zero/nonzero classification is unresolved without additional separation data.

Proof. These are immediate consequences of nonnegativity and interval inclusion. $\square$

Definition 3.29 (Minimum positive Betti-mass witness). For a declared admissible family $\mathcal{M}$, a minimum-mass witness is a certified number $\kappa > 0$ such that

$$M \in \mathcal{M}, M \neq 0 \implies \mathcal{B}_W(M) \geq \kappa.$$

The witness must state why all finite, essential, boundary-clipped, and multiplicity cases satisfy the same lower bound; $\kappa = \tau$ is not assumed by default.

Theorem 3.30 (Separated zero certification). *Let $M \in \mathcal{M}$ have scalar enclosure $[L, U]$, and let κ be a valid minimum-mass witness for $\mathcal{M}$. If $U < \kappa$, then*

$$M = 0 \quad \text{and consequently} \quad \mathcal{B}_W(M) = 0.$$

No separate Betti-faithfulness hypothesis is required on this route.

Proof. If $M \neq 0$, the minimum-mass witness gives $\mathcal{B}_W(M) \geq \kappa$, while the valid enclosure gives $\mathcal{B}_W(M) \leq U < \kappa$, a contradiction. Hence $M = 0$, and its Betti integral is zero. $\square$

Proposition 3.31 (Positive tolerance alone cannot certify exact vanishing). *Absent a minimum-mass witness, a statement $\mathcal{B}_W(M) \leq \eta$ with $\eta > 0$ does not imply $M = 0$. Indeed, the interval modules $k_{[0,a]}$ have positive Betti mass $a \rightarrow 0$.*

Proof. Choose $0 < a < \eta$. The module is nonzero while its Betti integral is a . $\square$

Theorem 3.32 (Positive lower enclosure certifies nonvanishing). *If a valid scalar enclosure satisfies $L > 0$, then $M \neq 0$. No Betti-faithfulness hypothesis is needed for this direction.*

Proof. The true integral is positive by Theorem 3.28, and Proposition 3.20 applies. $\square$

Definition 3.33 (Scalar measurement ledger record). A scalar measurement ledger records value-space uncertainty, such as $[L, U]$, an absolute error $\eta_{\mathcal{B}}$, or a quadrature bound. Its codomain is a declared ordered value space. It is not the Core metric ledger unless a separate certified conversion supplies a metric-ledger element.

3.6 Measurement-level stability

Definition 3.34 (Measurement-level stability modulus). For an identified class of repaired profiles, a function $\omega : [0, +\infty] \rightarrow [0, +\infty]$ is a Betti-measurement stability modulus if

$$d_B(M, N) \leq \varepsilon \implies |\mathcal{B}_W(M) - \mathcal{B}_W(N)| \leq \omega(\varepsilon)$$

under the declared finiteness and barcode conventions.

Definition 3.35 (Expanded-window local bar count). For $W = [u, u']$ and $\varepsilon \geq 0$, put $W^\varepsilon := [u - \varepsilon, u' + \varepsilon]$. For finite barcodes $\mathcal{D}(M), \mathcal{D}(N)$, define

$$R_{W, \varepsilon}(M, N) := \#\{I \in \mathcal{D}(M) : I \cap W^\varepsilon \neq \emptyset\} + \#\{J \in \mathcal{D}(N) : J \cap W^\varepsilon \neq \emptyset\},$$

counting multiplicity.

Theorem 3.36 (Finite-barcode bottleneck stability of Betti mass). *Let M, N have finite barcodes and suppose $d_B(M, N) \leq \varepsilon$. Then*

$$|\mathcal{B}_W(M) - \mathcal{B}_W(N)| \leq 2\varepsilon R_{W, \varepsilon}(M, N).$$

Proof. Choose an ε -matching. For a matched pair of intervals, each endpoint moves by at most ε , so the clipped lengths on W differ by at most 2ε . An unmatched interval has persistence at most 2ε , hence contributes at most 2ε to clipped mass. Intervals outside W^ε contribute neither before nor after matching. Summing over the locally counted occurrences gives the stated safe bound. $\square$

Corollary 3.37 (Normalized Betti-mass stability). *If $0 < \lambda(W) < \infty$, then under the hypotheses of Theorem 3.36,*

$$|\overline{\mathcal{B}}_W(M) - \overline{\mathcal{B}}_W(N)| \leq \frac{2\varepsilon R_{W, \varepsilon}(M, N)}{\lambda(W)}.$$

Proposition 3.38 (No cardinality-free Lipschitz constant for Betti mass). *There is no universal constant C such that*

$$|\mathcal{B}_W(M) - \mathcal{B}_W(N)| \leq C d_B(M, N)$$

for all finite barcodes without a local cardinality or mass bound.

Proof. Fix $0 < \varepsilon < 1$, $W = [0, 1)$, let M_n be n copies of $[0, 1)$, and N_n be n copies of $[\varepsilon, 1 + \varepsilon)$. Then $d_B(M_n, N_n) \leq \varepsilon$, while the Betti-mass difference on W equals $n\varepsilon$, which is unbounded in n . $\square$

Proposition 3.39 (Repaired-profile transport prerequisite). *When M and N arise by hard threshold deletion from compared pre-threshold profiles, the distance bound consumed by Theorem 3.36 is verifier-usable only after the exact or sharp two-sided threshold-gap transfer required by Appendix I. Appendix J does not restore global non-expansiveness of hard deletion.*

Proof. The measurement theorem begins with an actual bound between the repaired profiles. Producing that bound across hard deletion is a distinct upstream obligation governed by the v19 threshold-gap theorem. $\square$

Definition 3.40 (Measurement stability certificate). A measurement stability certificate contains the repaired profile identities, exact or metric-gap-certified comparison record, ε , the local bar count or another proved mass bound, the resulting modulus, numerical mode, and reconstructible matching or proof artifacts.

3.7 Scalar measurements, metric ledgers, and re-entry

Proposition 3.41 (Betti integral is not a complete invariant). *Equal Betti integrals do not imply isomorphic persistence profiles. For example, on $W = [0, 1)$, the barcodes*

$$\{[0, 1)\} \quad \text{and} \quad \{[0, \tfrac{1}{2}), [\tfrac{1}{2}, 1)\}$$

have the same Betti integral 1 but are not isomorphic as persistence modules.

Proof. The barcode multisets differ, although the sums of clipped lengths agree. $\square$

Definition 3.42 (Value-space and metric-ledger separation). Let S be the ordered value space of scalar Betti uncertainty and let $(V, \oplus, 0_V, \preceq)$ be the Core metric-ledger quantale. A scalar error $\eta_{\mathcal{B}} \in S$ and a profile-discrepancy element $\mathfrak{d}_{\text{meas}} \in V$ are different typed objects. They are never combined or compared without a declared conversion theorem.

Proposition 3.43 (Scalar Betti error is not a metric budget). *A bound*

$$|\widehat{\mathcal{B}} - \mathcal{B}_W(M)| \leq \eta_{\mathcal{B}}$$

does not by itself imply a bottleneck bound on the underlying profile and therefore does not enter $\text{val}_B(\mathfrak{d}_{\text{met}}) < \text{gap}_{\tau}$. Equal or close Betti mass can hide different barcodes.

Proof. The Betti integral is not injective by Proposition 3.41. Thus scalar accuracy alone cannot reconstruct or bound profile discrepancy. $\square$

Definition 3.44 (Certified measurement-to-metric conversion). A certified measurement-to-metric conversion creates a new content-addressed element $\mathfrak{d}_{\text{meas}} \in V$ and supplies actual source and target profile identities plus reconstructible evidence that

$$d_B(M, \widehat{M}) \leq \text{val}_B(\mathfrak{d}_{\text{meas}}).$$

The witness records all numerical, reconstruction, crop, and normalization hypotheses. A scalar error formula without this profile bound is not a conversion.

Theorem 3.45 (Measurement-attachment noninterference). *Two certificate records with identical verifier-side projections have identical Core atomic status and failure route even if their Betti curves, integrals, plots, scalar enclosures, or measurement attachments differ.*

Proof. The Core verdict factors through the verifier-side projection. Measurement attachments remain outside that projection unless a separate verified record is constructed. $\square$

Theorem 3.46 (Discretization-to-scalar error composition). *Suppose Appendix I supplies $d_B(M_h, M) \leq \varepsilon_h$, a registered modulus gives*

$$|\mathcal{B}_W(M_h) - \mathcal{B}_W(M)| \leq \omega(\varepsilon_h),$$

and a numerical enclosure gives

$$|\widehat{\mathcal{B}}_h - \mathcal{B}_W(M_h)| \leq \eta_h.$$

Then

$$|\widehat{\mathcal{B}}_h - \mathcal{B}_W(M)| \leq \eta_h + \omega(\varepsilon_h).$$

Proof. Apply the triangle inequality in the scalar value space. $\square$

Proposition 3.47 (Mixed-codomain error addition is forbidden). *The formal expression $\mathbf{d}_{\text{disc}} \oplus \eta_{\mathcal{B}}$ is ill-typed when $\mathbf{d}_{\text{disc}} \in V$ and $\eta_{\mathcal{B}} \in S$. One must either propagate metric error to scalar error through a measurement modulus, or convert measurement reconstruction to a metric bound through Definition 3.44.*

Proof. The operands live in different typed monoids and no cross-operation is defined. $\square$

Definition 3.48 (Dependency-aware measurement aggregation). A combined measurement record contains a dependency graph separating source approximation, profile reconstruction, hard-deletion transport, quadrature, rounding, and scalar reporting. Aggregation counts each primitive uncertainty once and records whether downstream bounds already include upstream components.

3.8 Measured topological discharge and B-Gate boundary

Definition 3.49 (Measured topological discharge record). A measured topological discharge record for degree one contains the repaired-profile identity, numerical and artifact records, and exactly one of the following route-specific witness packages:

- (i) an *exact-zero route*: an exact zero Betti integral together with a Betti-faithfulness witness, as in Theorem 3.17;
- (ii) a *separated-zero route*: a valid scalar enclosure $[L, U]$ and a minimum-positive-mass witness κ with $U < \kappa$, as in Theorem 3.30.

It records the resulting verifier-side statement $\text{Eval}_{1,W,\tau}(F) = 0$. The verifier-side discharge record is new and distinct from every scalar attachment.

Theorem 3.50 (Sound measured topological discharge). *Every measured topological discharge record satisfying Definition 3.49 soundly discharges the repaired topological clause*

$$\text{Eval}_{1,W,\tau}(F) = 0.$$

Proof. The exact-zero route uses Theorem 3.17, where Betti-faithfulness is load-bearing. The separated-zero route uses Theorem 3.30 directly, because its minimum-mass witness already excludes every nonzero member of the declared family below κ . The artifact record establishes that the consumed scalar statement and route-specific witness are bound to the identified repaired profile. $\square$

Theorem 3.51 (Measured Betti discharge does not imply full B-Gate⁺). *A sound measured topological discharge proves only the repaired topological clause. It does not by itself establish representative eligibility, the one-way Ext¹ clause, Type IV cleanliness, tower artifacts, comparison obligations, or the strict metric budget required by B-Gate⁺.*

Proof. These are distinct conjunctive verifier-side obligations and cannot be manufactured or compensated for by a scalar measurement. $\square$

Theorem 3.52 (Certified measured nonzero forces topological failure). *If a valid scalar enclosure for the repaired degree-one profile has $L > 0$, then*

$$\text{Eval}_{1,W,\tau}(F) \neq 0,$$

so the repaired topological clause fails.

Proof. Theorem 3.32 gives a nonzero repaired profile. $\square$

3.9 Type IV proxies and threshold bands

Definition 3.53 (Betti Type IV proxy). A Betti Type IV proxy is a Betti curve, integral, or derived scalar intended to correlate with terminal-generic kernel or cokernel defects. It does not define the actual comparison morphism or either defect profile.

Theorem 3.54 (Betti Type IV proxy is not a diagnosis). *A Betti Type IV proxy does not determine $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$ or its negation. Diagnosis requires the actual tower comparison morphism, terminal-tameness evidence, and terminal-generic dimensions of its kernel and cokernel, with canonical cokernel symbol ν .*

Proof. Local dimension mass of source or target profiles does not compute the kernel and cokernel of the required comparison morphism. $\square$

Definition 3.55 (Measurement-stable threshold band). A threshold interval $B \subset (0, +\infty)$ is measurement-stable for a declared Betti quantity if a universal artifact or theorem proves the prescribed scalar tolerance for every $\tau \in B$. A finite or countable sampled sweep is recorded separately.

Proposition 3.56 (Sampled measurement band is not a universal band certificate). *Stability on sampled thresholds does not imply stability at every threshold in the interval without a finite-reduction, monotonicity, continuity, or breakpoint theorem covering the unsampled values.*

Proof. An unsampled threshold can cross a retained bar length or another breakpoint and change the repaired profile and its integral. $\square$

Theorem 3.57 (Betti-stable band is not a Core stable band). *A measurement-stable Betti band is not a Core Type IV stable band unless actual monitored Type IV diagnostics are certified zero throughout the band by the required universal artifact family or reduction theorem.*

Proof. Betti stability is a scalar predicate; Core Type IV stability is a predicate on kernel and cokernel defect profiles of actual morphisms. Neither implies the other by definition. $\square$

3.10 Artifacts, manifest, and register parity

Definition 3.58 (Betti measurement artifact bundle). A Betti artifact bundle contains, as invoked: repaired-profile and barcode identities; exact endpoint data; Betti curves; breakpoint sets; partitions and sample rules; exact or interval arithmetic logs; matching artifacts; scalar enclosures; minimum-mass witnesses; conversion witnesses; dependency graph; generator version; environment; and source hashes.

Definition 3.59 (Betti measurement manifest entry). A manifest entry records the parent certificate, degree, window, threshold, repaired profile, measurement kind and units, numerical mode, comparison policy and resolved mode, scalar enclosure, metric conversion when any, ledger destinations, artifact references, permitted use, forbidden reading, lifecycle status, and failure behavior. Omission is not repaired by convention.

Proposition 3.60 (Stale measurement artifacts are non-consumable). *If any content-addressed profile, barcode, window, threshold, integration rule, numerical environment, normalization, or source dependency changes, the prior measurement artifact is stale for the changed run and cannot be silently reused.*

Proof. The changed dependency defines a different computation identity and requires a new immutable record. $\square$

Proposition 3.61 (Betti artifacts are necessary for audit readability). *A certificate relying on Betti-measurement evidence is audit-readable only when content-addressed artifacts permit independent reconstruction. The reconstructible data include the repaired profile, integral or enclosure, numerical method, error argument, type conversion, and ledger placement.*

Proof. Without those artifacts, the verifier cannot check either the scalar statement or its permitted use. $\square$

Definition 3.62 (Human-readable and machine-readable equivalence). The mathematical appendix body is the source of theorem statements, proofs, and internal labels. The complete human-readable Claim Register is published as a separate audit sidecar, together with canonical JSON and CSV serializations. The sidecar forms must contain the same UID set and the same typing, source-binding, comparison-policy, permitted-use, forbidden-reading, failure-route, ancestry, and review fields. A mismatch blocks promotion, but the register is not reproduced inside the mathematical appendix.

3.11 Appendix J package theorem

Theorem 3.63 (Appendix J Betti-measurement package). *Within the Technical Extension layer:*

- (i) *repaired Betti curves are typed functions of immutable repaired profiles;*
- (ii) *finite constructible Betti integrals equal total clipped barcode mass;*
- (iii) *zero integral detects the zero profile exactly only in a Betti-faithful regime;*
- (iv) *an explicit ephemeral counterexample forbids the unconditional converse;*
- (v) *breakpoint-resolving quadrature is exact and bounded-variation quadrature has a certified mesh error;*
- (vi) *scalar enclosures certify a zero profile only through an exact-zero Betti-faithfulness witness or a separated-zero minimum-mass witness, and certify positivity through a positive lower enclosure;*
- (vii) *finite-barcode Betti mass is bottleneck-stable with an explicit local-cardinality factor;*
- (viii) *no cardinality-free Lipschitz constant exists;*
- (ix) *scalar measurement error and metric-ledger error are different types;*
- (x) *discretization and scalar measurement errors compose through a measurement modulus, not by mixed-codomain addition;*
- (xi) *theorem-backed Betti evidence may discharge the repaired topological clause but not full B-Gate⁺;*
- (xii) *Betti Type IV proxies and Betti-stable bands do not replace actual diagnostics;*
- (xiii) *content-addressed artifacts and human/machine register parity are mandatory.*

Proof. The barcode interpretation is Theorems 3.8 and 3.11. The exact zero boundary and its counterexample are Theorem 3.15 and Proposition 3.16. Numerical integration is controlled by Theorems 3.22 and 3.23. Scalar classification is Theorems 3.28, 3.30, and 3.32. Measurement stability and its sharp limitation are Theorem 3.36 and Proposition 3.38. Typed error separation and composition are Proposition 3.43, Theorem 3.46, and Proposition 3.47. The gate and Type IV boundaries are Theorems 3.50, 3.51, 3.54, and 3.57. $\square$

3.12 Explicit exclusions

Definition 3.64 (Explicit exclusion boundary). Appendix J asserts none of the following:

- a Betti integral, plot, scalar enclosure, or quadrature value is a Core gate by itself;

- zero integral implies zero profile outside a Betti-faithful regime;
- a positive numerical tolerance certifies exact zero without a minimum-mass witness;
- equal or close Betti integrals imply equal or close persistence profiles;
- a scalar measurement error is a metric-ledger component without a profile-discrepancy conversion;
- global non-expansiveness of hard threshold deletion;
- a raw or sampled Betti curve is a repaired profile;
- a measured topological discharge establishes the eligible Ext¹, Type IV, artifact, or budget clauses;
- a Betti Type IV proxy computes $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}})$;
- a sampled or measurement-stable band is a Core stable band;
- heuristic, visual, AI-generated, or merely reproducible output is theorem evidence;
- package closure proves mathematical truth or an external-domain theorem.

3.13 Provenance and status

Definition 3.65 (Artifact closure). The integrated mathematical appendix body and the separately released audit sidecars form one release package. The sidecars carry machine registers, reproducibility records, and repair metadata; they are not mathematical proof and are not embedded in the appendix body.

This appendix depends on:

- (i) Chapters 1–3, 5–7;
- (ii) Appendices A–D, F, and G for threshold, representative, bridge, diagnostic, status, and manifest disciplines;
- (iii) [Appendix H](#) for advisory noninterference;
- (iv) [Appendix I](#) for discretization transfer.

Its role is to provide the Betti-integral and measurement-stability discipline without enlarging the Core or asserting an external-domain theorem.

Technical Extension definitions: Definitions 3.1, 3.2, 3.3, 3.4, 3.6, 3.7, 3.9, 3.10, 3.12, 3.13, 3.14, 3.21, 3.25, 3.27, 3.29, 3.33, 3.34, 3.35, 3.40, 3.42, 3.44, 3.48, 3.49, 3.53, 3.55, 3.58, 3.59, 3.62, 3.64, 3.65.

Technical Extension results: Theorems 3.8, 3.11, 3.15, 3.17, 3.22, 3.23, 3.28, 3.30, 3.32, 3.36, 3.45, 3.46, 3.50, 3.51, 3.52, 3.54, 3.57, 3.63. Propositions 3.5, 3.16, 3.18, 3.19, 3.20, 3.24, 3.26, 3.31, 3.38, 3.39, 3.41, 3.43, 3.47, 3.56, 3.60, 3.61. Corollaries 3.37.

Explicit exclusions: Section 3.12.

Final status. Document role: Technical Extension appendix. Evidence status: mixed proved, conditional where explicitly stated, and operational. Formal CR-level promotion remains pending until the complete Technical Extension sequence is audited.

4 Appendix K: AWFS and Controlled Commutation

4.1 Scope and relation to the Core

This appendix develops Algebraic Weak Factorization System (AWFS) and controlled-commutation tools for the Technical Extension layer. Its purpose is to regulate when two operational orders in an AK pipeline may be compared, replaced, or transported with a recorded defect.

The central rule is

Controlled commutation is not Core equality unless the relevant defect is zero, or unless a certified defect is entered into the correct ledger and the affected Core gate is rechecked.

The repaired window-first evaluation remains

$$\text{Eval}_{i,W,\tau}(F) := T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

No AWFS, approximate square, soft diagram, controlled 2-cell, or factorization package replaces $B\text{-Gate}^+$, the topological clause $\text{Eval}_{1,W,\tau}(F) = 0$, the monitored Type IV condition $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$, the tower comparison artifact $\phi_{i,W,\tau}$, required artifacts, or the strict metric-budget condition.

Definition 4.1 (Commutation sovereignty principle). Controlled commutation may organize a pipeline or justify a scoped replacement. It does not determine, compensate for, mutate, or retroactively repair a Core verdict.

Definition 4.2 (No AWFS-to-Core upgrade principle). An AWFS factorization, lifting operation, coalgebra, algebra, or functorial replacement does not become a Core collapse functor, admissible representative, Type IV comparison artifact, or bridge theorem merely by existing. Every Core use requires the corresponding registered verifier-side record.

Remark 4.3 (Technical extension status). The structures in this appendix are optional. They support implementation discipline, categorical bookkeeping, lift construction, and diagrammatic comparison. They do not restore the older exact-collapse interpretation excluded by the v19 repair.

Remark 4.4 (Why the appendix is needed). Many AK workflows compare alternative orders: window then collapse, collapse then window, realize then normalize, normalize then realize, factor then evaluate, or evaluate then factor. Such comparisons are useful, but v19 requires each replacement to be typed, scoped, and either exact or budgeted.

4.2 Pipeline diagrams and typed commutation claims

Definition 4.5 (Pipeline diagram). A pipeline diagram is a finite diagram in a declared category, bicategory, 2-category, or typed operational environment. Its vertices are typed AK objects and its arrows are typed operations, such as window restriction, barcode-level threshold deletion, admissible representative construction, realization, normalization, transfer, quotienting, truncation, or AWFS factorization.

Definition 4.6 (Strict square). A square

$$\begin{array}{ccc} A & \xrightarrow{f} & B \\ g \downarrow & & \downarrow g' \\ C & \xrightarrow{f'} & D \end{array}$$

is strict on the declared scope if the two composites are equal in the declared category,

$$g' \circ f = f' \circ g,$$

or are identified by a declared canonical isomorphism whose coherence data are part of the record.

Definition 4.7 (Commutation claim). A commutation claim records:

- (i) the diagram and its typed objects;
- (ii) the two composites being compared;
- (iii) the comparison mode;
- (iv) the defect semantics;
- (v) the proof, computation, or artifact supporting the comparison;
- (vi) the allowed use of the comparison in downstream certificates.

The comparison mode is one of *strict*, *exact-isomorphic*, *metric-controlled*, *two-cell-controlled*, *advisory-only*, or *undefined*.

Definition 4.8 (Core projection of K-local comparison modes and statuses). The comparison modes of Definition 4.7 are K-local typing tokens. When a commutation claim is consumed by a Core or CR comparison field, it resolves as follows.

- (i) *strict* and *exact-isomorphic* resolve to the Core mode *exact*, provided the equality, canonical isomorphism, and coherence data are registered.
- (ii) *metric-controlled* resolves to *metric_gap_certified* only when it supplies an actual profile discrepancy bound, the required threshold-gap or predicate-transport hypotheses, and a certified ledger destination. Without those data, it is not a verifier-side comparison.
- (iii) *two-cell-controlled* has no automatic Core projection. It resolves to *exact* only when the controlled 2-cell gives zero-cost coherent equality on the declared scope, and to *metric_gap_certified* only after conversion to an actual metric-ledger bound. Otherwise it is *not_compared* for Core comparison purposes.
- (iv) *advisory-only* resolves to *not_compared*; it may trigger construction of a later verifier-side record but is not itself one.
- (v) *undefined* has no valid resolved comparison mode. If an invoked obligation depends on it, the affected obligation is undefined.

The local commutation statuses of Definition 4.60 are likewise not Core atomic statuses. When an invoked commutation replacement is itself a verifier-side obligation, *strict* or *controlled* may satisfy only that obligation, *advisory*, *uncontrolled*, and *undefined* resolve to an undefined affected obligation, and *failed* resolves to rejection of the affected path. If commutation is not invoked, the corresponding Core status is *not_invoked*.

Definition 4.9 (Commutation defect). A commutation defect for a square is a typed discrepancy object

$$\Delta_{\square}$$

measuring the difference between the two composites. Its codomain may be a metric space, normed space, homotopy category, 2-cell cost object, ledger quantale, logical status lattice, or another declared target.

Definition 4.10 (Zero defect). A commutation defect is zero only when

$$\Delta_{\square} = 0$$

in its declared defect codomain, with all required identity, coherence, and endpoint conventions fixed. A numerically small defect is not zero unless the certified enclosure has upper bound zero.

Proposition 4.11 (Defect semantics are part of the theorem). *A statement of the form “the square commutes up to error” has no verifier-side meaning until the defect codomain, measurement rule, scope, comparison target, and ledger treatment are declared.*

Proof. Different codomains support different operations and different implications. A norm defect, an interleaving bound, a 2-cell, and a ledger element are not interchangeable. Without the declaration, no typed implication can be consumed by an audit record. $\square$

Proposition 4.12 (Strictness is not inherited by notation). *Writing two operational composites with the same informal name does not prove that they are equal, isomorphic, or interchangeable.*

Proof. Names do not supply a morphism equality, a natural isomorphism, or a coherence proof. The comparison remains undefined until a typed commutation claim is provided. $\square$

4.3 Soft and controlled commutation

Definition 4.13 (Soft commutation). A square is soft-commuting at radius $\delta_\square$ if its declared defect satisfies

$$\Delta_\square \leq \delta_\square$$

in the declared ordered defect codomain. This is a bounded-defect statement, not an equality statement.

Definition 4.14 (Controlled commutation). A square has controlled commutation when:

- (i) both composites are defined on the same declared scope;
- (ii) the defect semantics are fixed;
- (iii) a bound or exact vanishing theorem is supplied;
- (iv) the defect is either proved zero or mapped into the correct verifier-side ledger by a conversion witness;
- (v) every downstream use states which route is consumed.

Definition 4.15 (Commutation ledger component). Let $(V, \oplus, 0_V, \preceq)$ be the metric-ledger quantale. A commutation ledger component is an element

$$\mathfrak{d}_{\text{comm}} \in V$$

with a scalarization $\text{val}_B : V \rightarrow [0, +\infty]$ and a theorem-backed bound on an actual profile discrepancy. A scalar commutation score is not a ledger component unless such a conversion is proved.

Definition 4.16 (Commutation-ledger namespace). The symbols $\mathfrak{d}_{\text{comm}}$ and $\mathfrak{d}_{2\text{-cell}}$ do not name independent ledgers. They denote typed subcomponents of the declared metric ledger $\mathfrak{d}_{\text{met}}$, normally in the comparison/commutation slot of the active ledger schema. If a successor product-ledger appendix refines this slot, it must provide either the same target quantale V or an explicit inclusion or scalarization map into the Core metric-ledger quantale consumed by the strict budget test. Until such a refinement is invoked, Appendix K treats all commutation and 2-cell defects as components of the Chapter 6/Appendix A metric-ledger semantics.

Theorem 4.17 (Controlled commutation noninterference). *If two certificate records have identical verifier-side projections after deleting commutation annotations and advisory commutation indicators, then they have the same Core atomic status and failure route.*

Proof. Core verdicts depend on verifier-side records: repaired evaluations, eligibility records, Type IV artifacts, diagnostics, comparison records, ledgers, and failure statuses. Pure annotations and advisory indicators are not inputs to the verdict function. $\square$

Theorem 4.18 (Soft commutation affects Core only through certified ledgers). *A nonzero soft-commutation defect may affect a Core-adjacent certificate only through a certified verifier-side ledger component at the declared scope. The original soft-commutation statement remains non-verdict data.*

Proof. A bounded defect is not a gate verdict, diagnostic, or representative-compatibility record. The Core can consume it only after it has been converted into the declared ledger object whose scalarization bounds the actual discrepancy used by the certificate. $\square$

Proposition 4.19 (Controlled does not mean harmless). *A controlled defect may still defeat the affected certificate if its ledger contribution consumes the available threshold margin.*

Proof. The budget condition is strict. Once the defect enters $\Sigma\delta$, the certificate can pass only if the complete aggregate remains below the relevant gap. $\square$

4.4 Metric replacement and threshold-safe use

Definition 4.20 (Operational replacement). An operational replacement is a declared instruction to use one composite pipeline output in place of another. If the outputs are repaired persistence profiles M and N , the replacement record is either exact, with $M \cong N$, or metric-controlled, with a certified bound

$$d_B(M, N) \leq \varepsilon.$$

Theorem 4.21 (Metric replacement is not equality). *A certified bound $d_B(M, N) \leq \varepsilon$ does not permit replacing equality-based conclusions about M by equality-based conclusions about N , unless an additional theorem transports the specific predicate under that bound.*

Proof. Metric proximity is a quantitative relation. Equality, vanishing, admissibility, and Type IV cleanliness are predicates with their own transport conditions. No general predicate transport follows from a distance bound alone. $\square$

Theorem 4.22 (Threshold-separated commutation transfer). *Let E_L and E_R be two post-threshold repaired profiles arising from alternative operational orders at threshold τ . If*

$$d_B(E_L, E_R) \leq \varepsilon \quad \text{and} \quad 2\varepsilon < \tau,$$

then

$$E_L = 0 \iff E_R = 0.$$

Proof. After hard deletion at threshold τ , every nonzero finite bar has length strictly larger than τ , and infinite bars cannot be matched to the diagonal at finite cost. If one side is zero and the other contains a bar, that bar has diagonal matching cost greater than $\tau/2 > \varepsilon$, contradicting the bottleneck bound. The argument is symmetric. $\square$

Proposition 4.23 (Budget pass alone does not transfer commutation predicates). *A commutation defect satisfying the scalar budget inequality does not by itself transfer exact vanishing, admissible representatives, Ext¹-eligibility, or Type IV diagnostics.*

Proof. The budget inequality controls allowable discrepancy. It does not provide the non-scalar records required by those predicates. $\square$

Definition 4.24 (Replacement manifest entry). A replacement manifest entry records the source composite, target composite, comparison mode, distance or equality evidence, threshold-gap hypotheses when relevant, predicate-transport theorem when invoked, ledger destination, and failure behavior.

4.5 AWFS structures and their boundary

Definition 4.25 (Weak factorization system). A weak factorization system on a category C is a pair $(\mathcal{L}, \mathcal{R})$ such that every morphism factors as $r \circ l$, with $l \in \mathcal{L}$ and $r \in \mathcal{R}$, and the classes are determined by lifting orthogonality:

$$\mathcal{L} = {}^{\square}\mathcal{R}, \quad \mathcal{R} = \mathcal{L}^{\square},$$

where $l \square r$ means that every commutative square with left side l and right side r admits a diagonal filler.

Definition 4.26 (Algebraic weak factorization system). An algebraic weak factorization system on C is a functorial factorization together with a comonad on the left factor and a monad on the right factor, plus the distributive law data making left coalgebras and right algebras interact coherently.

Definition 4.27 (AWFS-controlled pipeline). An AWFS-controlled pipeline is a pipeline in which selected arrows are factored using a declared AWFS, and the factorization data are used to organize lifts, replacements, fibrant or cofibrant repairs, comparison morphisms, or representative construction.

Proposition 4.28 (AWFS data are organizational until consumed by a verifier record). *AWFS factorization data have no Core effect until a downstream verifier record consumes a specific factor, lift, or comparison with its registered hypotheses.*

Proof. The Core does not ask whether a pipeline was organized by an AWFS. It asks whether the required repaired evaluation, representative, bridge, diagnostic, artifact, and ledger obligations are satisfied. $\square$

Definition 4.29 (AWFS lifting certificate). An AWFS lifting certificate consists of a displayed lifting problem, an L-coalgebra structure on the left map, an R-algebra structure on the right map, the constructed diagonal filler, and the coherence proof or replay artifact supplied by the AWFS.

Proposition 4.30 (Lifting notation does not supply a lift). *Drawing a lifting square or naming an AWFS does not establish a diagonal filler. A lift used in a certificate must be supplied by an AWFS lifting certificate or an independent proof.*

Proof. A lifting conclusion depends on the algebraic structures and the particular square. The notation alone identifies neither the coalgebra/algebra structures nor the filler. $\square$

Proposition 4.31 (Functorial factorization does not commute with AK operations by default). *A functorial AWFS factorization does not automatically commute with window restriction, threshold deletion, realization, normalization, transfer, or terminal-tail passage.*

Proof. Functoriality is internal to the declared factorization functor. Commutation with an external AK operation requires an additional naturality, Beck-Chevalley, exact-admissibility, or controlled-defect theorem. $\square$

4.6 Admissible representatives and categorical eligibility

Definition 4.32 (AWFS-assisted representative). An AWFS-assisted representative is an admissible representative $C_{\tau, W}F$ whose construction, replacement, or comparison is organized using AWFS factorization or lifting data.

Theorem 4.33 (AWFS-assisted representative has no stronger Core status). *An AWFS-assisted representative is Core-usable only under the ordinary admissible-representative compatibility condition*

$$\mathbf{P}_i(C_{\tau, W}F) \cong \text{Eval}_{i, W, \tau}(F)$$

on the declared scope. The AWFS organization itself does not strengthen that status.

Proof. The Core consumes the compatibility isomorphism with the repaired profile. The construction method may support finding such a representative, but it is not a substitute for the compatibility record. $\square$

Theorem 4.34 (Realization commutation does not replace eligibility). *A controlled square comparing realization before and after an operation does not replace the eligibility hypotheses required for a PH_1 -to- Ext^1 bridge.*

Proof. Eligibility requires amplitude, edge-realization, and representative hypotheses. A commutation square involving realization does not establish those hypotheses unless its witness explicitly proves them. $\square$

Proposition 4.35 (Bridge discharge is not functorial bookkeeping). *No AWFS lift, functorial replacement, or commutative realization square discharges an Ext^1 -clause without the registered one-way bridge hypotheses.*

Proof. The categorical bridge is a mathematical theorem with typed hypotheses. Bookkeeping structures do not imply those hypotheses by default. $\square$

4.7 Controlled commutation with repaired collapse and windows

Definition 4.36 (Collapse-commutation square). A collapse-commutation square compares an operation Q with barcode-level threshold deletion:

$$\begin{array}{ccc} M & \xrightarrow{Q} & Q(M) \\ T_{\tau}^{\text{bar}} \downarrow & & \downarrow T_{\tau}^{\text{bar}} \\ T_{\tau}^{\text{bar}} M & \xrightarrow{Q_{\tau}} & Q_{\tau}(T_{\tau}^{\text{bar}} M). \end{array}$$

It is a profile-level comparison unless an exact-admissible categorical regime is separately proved.

Definition 4.37 (Collapse-commutation defect). The collapse-commutation defect $\Delta_{T, Q}$ measures the discrepancy between

$$T_{\tau}^{\text{bar}} Q(M) \quad \text{and} \quad Q_{\tau} T_{\tau}^{\text{bar}}(M)$$

under a declared comparison mode.

Theorem 4.38 (No exact-collapse restoration). *A controlled collapse-commutation square does not imply that T_{τ}^{bar} is a global exact functor, exact reflector, Serre localization, or strict filtered lift.*

Proof. The square compares two named composites in a declared profile regime. It supplies neither global functoriality nor exactness on all morphisms. $\square$

Definition 4.39 (Window-commutation square). A window-commutation square compares an operation Q with restriction to W :

$$\begin{array}{ccc} M & \xrightarrow{Q} & Q(M) \\ W_W \downarrow & & \downarrow W_W \\ W_W M & \xrightarrow{Q_W} & Q_W(W_W M). \end{array}$$

Definition 4.40 (Crop-collapse defect). The crop-collapse defect is the special defect comparing

$$W_W T_\tau^{\text{bar}}(M) \quad \text{and} \quad T_\tau^{\text{bar}}(W_W M).$$

Theorem 4.41 (Crop-collapse commutation is not default). *The identity*

$$W_W T_\tau^{\text{bar}} \cong T_\tau^{\text{bar}} W_W$$

is not available by default. If an alternative order is used, strict equality or a certified crop-collapse defect must be recorded.

Proof. Windowing can shorten bars and hard threshold deletion depends on bar length. Reordering the operations can therefore change the surviving profile. The repaired Core avoids the hidden assumption by fixing the order W_W then T_τ^{bar} . $\square$

Remark 4.42 (Exact crop-collapse criteria). Exact crop-collapse commutation may be recovered only under a separately proved strict criterion, such as the τ -window-adapted condition of Appendix A and its CM-v19.0.0-r2 micro-theorem formulation. Appendix K does not restate that criterion as a default rule; it records that any use of the alternative order must cite such a strict theorem or supply a controlled-defect certificate.

Proposition 4.43 (Endpoint bookkeeping is not a defect proof). *A nonzero window or crop-collapse defect cannot be dismissed as endpoint bookkeeping unless the endpoint convention proves the defect vanishes on the declared profile class.*

Proof. Endpoint and crop conventions can change interval lengths and hence threshold survival. A vanishing conclusion requires an actual theorem for the declared convention. $\square$

4.8 Controlled commutation with realization, normalization, and transfer

Definition 4.44 (Realization-commutation square). A realization-commutation square compares an operation Q before realization with an operation $\bar{Q}$ after realization:

$$\begin{array}{ccc} F & \xrightarrow{Q} & Q(F) \\ \mathcal{R} \downarrow & & \downarrow \mathcal{R} \\ \mathcal{R}(F) & \xrightarrow{\bar{Q}} & \bar{Q}(\mathcal{R}(F)). \end{array}$$

Definition 4.45 (Normalization-commutation defect). A normalization-commutation defect compares applying a normalization $\mathfrak{N}$ before an AK operation with applying the corresponding normalized operation afterward. Its value has no default relation to the metric ledger unless a conversion theorem supplies one.

Definition 4.46 (Transfer-commutation defect). A transfer-commutation defect compares a transfer operation with another pipeline operation, such as windowing, threshold deletion, realization, representative selection, or Type IV comparison. It must specify direction; a controlled transfer square is not automatically reversible.

Proposition 4.47 (Direction matters). *A controlled comparison from composite A to composite B does not automatically supply the reverse comparison from B to A , unless the defect semantics and proof are symmetric.*

Proof. Some comparison structures are directed: simulations, lax natural transformations, error enclosures, and reconstruction maps may have one-way meaning. Reversibility is an additional hypothesis. $\square$

Proposition 4.48 (Normalization defects are not harmless rescalings). *A normalization change may alter metric scales, thresholds, bar lengths, or ledger scalarizations. It cannot be ignored unless a normalization invariance or conversion theorem is supplied.*

Proof. The Core budget and threshold-gap conditions are quantitative. Rescaling or renormalizing data changes the quantities consumed by those conditions unless compatibility is proved. $\square$

4.9 Controlled 2-cells and coherence

Definition 4.49 (Controlled 2-cell). In a bicategorical or 2-categorical setting, a controlled 2-cell is a 2-morphism

$$\alpha : g' \circ f \Rightarrow f' \circ g$$

together with declared cost, orientation, coherence data, and allowed composition rule.

Definition 4.50 (2-cell ledger). A 2-cell ledger records costs $\delta(\alpha_j)$, their domains, and their aggregation into a typed element $\mathfrak{d}_{2\text{-cell}} \in V$ or another declared ledger object.

Theorem 4.51 (Pasting law for controlled 2-cells). *Controlled 2-cells may be pasted only under a declared aggregation law. If the ledger law is $\oplus$, the pasted defect is bounded by the $\oplus$ -aggregate of the component defects, provided the source and target scopes match.*

Proof. Pasting composes the underlying comparison witnesses. The defect bound follows only from the declared composition law for the defect codomain. Without that law, no numerical or ledger bound is determined. $\square$

Proposition 4.52 (Coherence defects cannot be silently suppressed). *If associativity, unit, or interchange coherence is required for a downstream comparison, the corresponding coherence data or coherence defect must be recorded.*

Proof. A downstream equality of composites may depend on how multiple 2-cells associate and interchange. Missing coherence is a missing non-scalar obligation. $\square$

Remark 4.53 (Relation to later transfer appendices). Later transfer or mirror appendices may specialize controlled 2-cells to particular geometric, mirror, or arithmetic operations. Appendix K supplies only the general typed control discipline.

4.10 Defect aggregation and dependency control

Definition 4.54 (Sequential defect aggregation). For a sequence of controlled replacements with ledger components $\mathfrak{d}_1, \dots, \mathfrak{d}_m \in V$, the sequential aggregate is admissible only when the declared ledger theorem gives

$$\mathfrak{d}_{\text{seq}} \succeq \mathfrak{d}_1 \oplus \dots \oplus \mathfrak{d}_m.$$

Definition 4.55 (Parallel defect aggregation). For a finite family of parallel controlled comparisons, the parallel aggregate is the declared finite $\oplus$ -aggregate, maximum, product, lexicographic tuple, or other stated quantale expression. No ordinary-addition convention is implicit.

Definition 4.56 (Dependency graph for commutation defects). A defect dependency graph identifies which commutation defects are independent primitive bounds, nested bounds, alternative bounds, or downstream consequences of other bounds. The same primitive discrepancy may not be counted twice under different labels.

Proposition 4.57 (Aggregation must match ledger semantics). *A certificate using multiple commutation defects is audit-readable only when the aggregate defect is computed using the declared ledger semantics and dependency graph.*

Proof. The audit verifier must know how the total budget was obtained. A hidden additive or maximum convention changes the meaning of $\Sigma\delta$ and is therefore not acceptable. $\square$

Proposition 4.58 (No double counting and no undercounting). *A commutation-ledger record is invalid if it counts the same primitive error twice or omits a primitive defect used by the replacement path.*

Proof. Double counting overstates the budget and may wrongly reject; undercounting understates the budget and may wrongly pass. Both violate the declared aggregation semantics. $\square$

4.11 Status, manifests, and audit readability

Definition 4.59 (Commutation manifest entry). A commutation manifest entry records: the diagram; the two composites; the comparison mode; defect semantics; exact or bounded proof; ledger element when any; predicate-transport theorem when any; artifact references; allowed scope; and failure behavior.

Definition 4.60 (Commutation status). A commutation record has one of the following statuses:

- (i) *strict*: zero defect or canonical equality is proved;
- (ii) *controlled*: a nonzero defect is certified and correctly ledgered;
- (iii) *advisory*: evidence suggests commutation but lacks a verifier-side bound;
- (iv) *uncontrolled*: a comparison is invoked without a declared bound;
- (v) *failed*: the invoked defect exceeds the available budget or violates required transport conditions;
- (vi) *undefined*: a required object, composite, or defect semantics is missing.

Proposition 4.61 (Uncontrolled commutation cannot support certification). *An uncontrolled commutation claim cannot be used to support a Core or Core-adjacent certificate.*

Proof. The verifier cannot reconstruct a bound, equality, or failure behavior. The comparison is therefore not audit-readable. $\square$

Proposition 4.62 (Failed commutation defeats the affected path). *If an invoked commutation replacement is failed, then every certificate path depending on that replacement fails unless an independent verified path is supplied.*

Proof. The affected path consumes the replacement. If its defect violates the required budget or transport condition, the path's obligations are not discharged. $\square$

Definition 4.63 (Advisory commutation indicator). An advisory commutation indicator is a heuristic, spectral, numerical, visual, AI-generated, or measurement-level signal suggesting that a diagram may commute or approximately commute.

Proposition 4.64 (Advisory commutation indicator is not commutation proof). *An advisory commutation indicator does not establish strict or controlled commutation.*

Proof. Strict or controlled commutation requires a typed equality or defect bound. An indicator may guide where to construct such a record, but it does not provide the record. $\square$

4.12 Appendix K package theorem

Theorem 4.65 (Appendix K controlled-commutation package). *Within the Technical Extension layer, Appendix K supplies the following package.*

- (i) *Pipeline diagrams and commutation claims are typed by objects, composites, modes, and defect semantics.*
- (ii) *K-local comparison modes and commutation statuses project to Core comparison modes and atomic statuses only through the declared projection rule.*
- (iii) *Soft commutation is bounded defect, not equality.*
- (iv) *Nonzero commutation affects Core-adjacent certification only through certified ledger conversion into the declared metric-ledger namespace.*
- (v) *Metric replacement does not transport equality-based predicates without a predicate-specific theorem.*
- (vi) *For post-threshold profiles, $2\epsilon < \tau$ transfers zero/nonzero classification.*
- (vii) *AWFS structures are optional organizational data and do not upgrade Core status.*
- (viii) *AWFS lifts require displayed coalgebra/algebra data and a filler certificate.*
- (ix) *AWFS factorization does not commute with windowing, collapse, realization, normalization, transfer, or terminal-tail passage by default.*
- (x) *AWFS-assisted representatives have exactly the ordinary admissible-representative status.*
- (xi) *Realization commutation does not replace categorical eligibility or the one-way PH_1 -to- Ext^1 bridge hypotheses.*
- (xii) *Crop-collapse and collapse-operation commutation must be proved strict or budgeted when used.*
- (xiii) *Controlled 2-cells require orientation, coherence, and ledger-aware pasting.*
- (xiv) *Defect aggregation must obey the declared ledger semantics and dependency graph.*
- (xv) *Uncontrolled, advisory, failed, or undefined commutation cannot support certification.*
- (xvi) *No controlled-commutation theorem enlarges the Core or restores global exact collapse.*

Proof. Item (i) follows from the typed commutation definitions. Item (ii) is Definition 4.8. Item (iii) is the bounded-defect content of Definition 4.13. Item (iv) is Theorem 4.18 together with Definition 4.16. Items (v)–(vi) are Theorems 4.21 and 4.22. Items (vii)–(xi) follow from the AWFS and eligibility boundary results. Items (xii)–(xiii) follow from the collapse/window and 2-cell sections, including Remark 4.42. Items (xiv)–(xv) follow from the aggregation and manifest results. Item (xvi) is the combined non-upgrade boundary of this appendix. $\square$

4.13 Explicit exclusions

Appendix K asserts none of the following:

- No AWFS is required by the Core.
- No AWFS factorization is a repaired collapse functor.
- No AWFS-assisted representative has stronger status than an ordinary admissible representative.
- No soft-commuting square is strict unless its defect is proved zero.
- No controlled commutation replaces $B\text{-Gate}^+$.
- No controlled commutation replaces $\text{Eval}_{1,W,\tau}(F) = 0$.
- No controlled commutation replaces the eligible Ext^1 -clause.
- No controlled commutation replaces Type IV comparison artifacts or diagnostics.
- No controlled commutation replaces the strict metric-budget condition.
- No global exactness, global non-expansiveness, or positive-threshold Serre localization of T_τ^{bar} is assumed.
- No crop-collapse commutation is assumed by default.
- No identity $C_{\tau,W}F = (C_\tau F)|_W$ is assumed.
- No realization-commutation square replaces eligibility.
- No normalization or transfer square is reversible unless reversibility is proved.
- No controlled 2-cell is harmless without ledger and coherence data.
- No advisory commutation indicator is proof.
- No problem-specific arithmetic, PDE, geometric, mirror, categorical, or cryptographic theorem is proved here.

4.14 Provenance and status

The material in Appendix K depends on:

- (i) the repaired window-first and threshold-gap discipline of Appendix A;
- (ii) the admissible representative discipline of Appendix B;
- (iii) the eligible realization and one-way bridge discipline of Appendix C;
- (iv) the monitored Type IV diagnostic calculus of Appendix D;
- (v) the audit, ledger, and non-compensation discipline of Chapter 6 and Appendix F;
- (vi) the manifest schema of Appendix G;
- (vii) the advisory noninterference discipline of Appendix H;
- (viii) the discretization and metric-transfer discipline of Appendix I;
- (ix) the scalar/metric separation discipline of Appendix J.

Its role is to permit categorical and diagrammatic control without allowing commutation claims to contaminate Core verdicts.

Status labels for Appendix K. *Technical extension principles:* Definitions 4.1 and 4.2. The local label prefix `prin`: is permitted here only for principle definitions and is counted in the definition inventory for promotion.

Technical extension definitions: Definitions 4.5, 4.6, 4.7, 4.9, 4.10, 4.8, 4.13, 4.14, 4.15, 4.16, 4.20, 4.24, 4.25, 4.26, 4.27, 4.29, 4.32, 4.36, 4.37, 4.39, 4.40, 4.44, 4.45, 4.46, 4.49, 4.50, 4.54, 4.55, 4.56, 4.59, 4.60, 4.63.

Technical extension propositions/theorems: Propositions 4.11, 4.12, Theorems 4.17, 4.18, Proposition 4.19, Theorems 4.21, 4.22, Propositions 4.23, 4.28, 4.30, 4.31, Theorems 4.33, 4.34, Proposition 4.35, Theorems 4.38, 4.41, Propositions 4.43, 4.47, 4.48, Theorem 4.51, Propositions 4.52, 4.57, 4.58, 4.61, 4.62, 4.64, Theorem 4.65.

Operational cautions: Remarks 4.3, 4.4, 4.42, 4.53.

Explicit exclusions: Appendix K, Section 4 explicit exclusion subsection above.

5 Appendix L: Mirror/Transfer 2-cell Ledger

5.1 Scope and relation to Appendices K, H, I, and J

This appendix specializes the controlled-commutation framework of Appendix K to Mirror/Transfer pipelines. Its role is to regulate when a mirror correspondence, transfer operation, or weak comparison cell may support an AK certificate, and how any nonzero defect is recorded.

The central rule is

Mirror/Transfer compatibility is not a Core theorem unless its hypotheses, comparison status, defects, and ledger effects are explicitly verified.

No Mirror/Transfer diagram, comparison functor, correspondence, transfer square, mirror map, weak equivalence candidate, or controlled 2-cell replaces $B\text{-Gate}^+$, the topological clause

$$\text{Eval}_{1,W,\tau}(F) = 0,$$

the monitored Type IV condition

$$(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0),$$

the tower comparison artifact $\phi_{i,W,\tau}$, required artifacts, or the strict metric-budget condition

$$\Sigma\delta < \text{gap}_\tau.$$

All Core-relevant persistence data remain governed by the repaired window-first evaluation

$$\text{Eval}_{i,W,\tau}(F) := T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

Definition 5.1 (Mirror/Transfer sovereignty principle). Mirror/Transfer data may organize comparison, suggest a transport route, or support construction of a new verifier-side record. They do not determine, compensate for, mutate, or retroactively repair a Core verdict.

Definition 5.2 (No Mirror/Transfer-to-Core upgrade principle). A mirror correspondence, transfer operation, controlled 2-cell, or comparison square does not become a Core bridge theorem, admissible representative, tower comparison artifact, Type IV diagnostic, or metric-ledger component merely by existing. Every Core use requires the corresponding registered verifier-side record and all declared hypotheses.

Remark 5.3 (Technical extension status). Mirror/Transfer machinery may compare mathematical presentations, dual descriptions, categorical models, geometric transforms, or computational pipelines. It remains Technical Extension material and is subordinate to the repaired Core, Appendix K controlled-commutation discipline, Appendix H non-verdict discipline, Appendix I discretization-transfer discipline, and Appendix J scalar/metric separation.

Remark 5.4 (No restoration of old exact-collapse language). This appendix does not restore a global exact collapse functor, a positive-threshold Serre localization, or a strict filtered lift. Every collapse-related Mirror/Transfer statement is read at the barcode or repaired-profile level unless a separately declared exact-admissible theorem is supplied.

5.2 Mirror and Transfer settings

Definition 5.5 (Mirror setting). A Mirror setting consists of two declared categories, bicategories, structured domains, or typed operational environments

$$\mathcal{A} \quad \text{and} \quad \mathcal{A}^\vee,$$

together with a declared mirror comparison datum

$$\text{Mir} : \mathcal{A} \dashrightarrow \mathcal{A}^\vee.$$

The datum may be a functor, contravariant functor, correspondence, span, adjunction-like datum, equivalence candidate, mirror map, kernel transform, or another explicitly typed comparison structure.

Definition 5.6 (Transfer setting). A Transfer setting consists of two declared domains

$$\mathcal{A} \quad \text{and} \quad \mathcal{B},$$

together with a declared transfer operation

$$\text{Tr} : \mathcal{A} \dashrightarrow \mathcal{B}$$

intended to move objects, filtrations, profiles, representatives, diagnostics, ledgers, or certificate-relevant data.

Definition 5.7 (Mirror/Transfer pipeline). A Mirror/Transfer pipeline is a typed diagram in which at least one path passes through a Mirror or Transfer operation and at least one other path represents a Core-adjacent operation such as window restriction, barcode-level threshold deletion, repaired evaluation, admissible representative construction, realization, tower comparison, normalization, quotienting, or interface transport.

Definition 5.8 (Variance and domain record). A Mirror/Transfer operation records its source, target, variance, partial domain, functoriality level, object-level rule, morphism-level rule when available, and failure behavior. The symbol $\dashrightarrow$ records that no total functoriality is assumed by default.

Proposition 5.9 (Raw Mirror/Transfer output is not a Core object). A raw Mirror/Transfer output is not automatically a filtered object, persistence profile, admissible representative, tower comparison artifact, Type IV diagnostic, ledger component, or certificate field.

Proof. Each listed Core use requires a typed realization target and verifier-side schema. A raw output supplies neither the realization nor the required compatibility and artifact records. $\square$

5.3 Realization targets and Core-testable outputs

Definition 5.10 (Mirror/Transfer realization target). A Mirror/Transfer realization target is one of the following declared outputs, with all source artifacts and scope data attached:

- (i) a filtered object in $\text{FiltCh}(k)$;
- (ii) a constructible persistence object in $\text{Pers}_k^{\text{cons}}$;
- (iii) a repaired windowed collapsed profile $\text{Eval}_{i,W,\tau}(F)$;
- (iv) an admissible representative $C_{\tau,W}F$;
- (v) a realized derived object in $D^b(k\text{-mod})$;
- (vi) a tower comparison artifact;
- (vii) a metric-ledger component;
- (viii) a certificate-record component;
- (ix) an advisory or search-side artifact.

Definition 5.11 (Core-testable Mirror/Transfer output). A Mirror/Transfer output is Core-testable only when its realization target allows the verifier to evaluate or reconstruct the relevant repaired profile, eligibility record, Type IV artifact, diagnostic, comparison record, or ledger component consumed by the Core gate.

Proposition 5.12 (Core-testable does not mean Core-passing). *A Core-testable Mirror/Transfer output does not by itself imply $\text{Eval}_{1,W,\tau} = 0$, $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$, a valid tower artifact, a budget pass, or a Core pass verdict.*

Proof. Core-testability is a typing condition. Pass conditions are separate predicates with their own evidence and failure routes. $\square$

Definition 5.13 (Surrogate-only Mirror/Transfer mode). A Mirror/Transfer run is in surrogate-only mode when its output is realized and testable only for the transformed or transferred surrogate, with no certified comparison to the original target-side object. Statements in this mode are explicitly about the surrogate.

Proposition 5.14 (Surrogate-only mode blocks target reading). *A surrogate-only Mirror/Transfer result cannot be read as a theorem about the original target, unless an exact or metric-certified target comparison record is added.*

Proof. The source and target objects are different records. Identifying them requires a verifier-side comparison record, not a shared basename or informal interpretation. $\square$

5.4 Comparison squares and local comparison typing

Definition 5.15 (Mirror comparison square). A Mirror comparison square compares the two composites in a diagram of the form

$$\begin{array}{ccc} X & \xrightarrow{\text{Mir}} & X^\vee \\ Q \downarrow & & \downarrow Q^\vee \\ QX & \xrightarrow{\text{Mir}_Q} & Q^\vee X^\vee. \end{array}$$

All four objects and all arrows are part of the typed record; no arrow is inferred from notation alone.

Definition 5.16 (Transfer comparison square). A Transfer comparison square compares the two composites in a diagram of the form

$$\begin{array}{ccc} X & \xrightarrow{\text{Tr}} & \text{Tr}(X) \\ \varrho \downarrow & & \downarrow \bar{\varrho} \\ QX & \xrightarrow{\text{Tr}_Q} & \bar{Q}\text{Tr}(X). \end{array}$$

The operation Tr_Q is declared data and is not automatically induced.

Definition 5.17 (Mirror/Transfer comparison mode). A Mirror/Transfer comparison has one local mode:

strict, zero-cost, metric-controlled, two-cell-controlled, advisory-only, unavailable, failed.

These are L-local tokens and do not by themselves equal CR comparison modes.

Definition 5.18 (Core projection of L-local comparison modes). When a Mirror/Transfer comparison is consumed by a Core or CR comparison field, the local mode projects as follows.

- (i) *strict* and *zero-cost* project to *exact* only when the zero-cost semantics identify the two composites with the required coherence.
- (ii) *metric-controlled* projects to *metric_gap_certified* only when it supplies an actual profile discrepancy bound, the relevant threshold-gap or predicate-transport hypotheses, and a certified ledger destination.
- (iii) *two-cell-controlled* projects to *exact* only for zero-cost coherent equality, to *metric_gap_certified* only after conversion to an actual metric-ledger bound, and otherwise to *not_compared*.
- (iv) *advisory-only* projects to *not_compared*.
- (v) *unavailable* has no resolved comparison mode; any invoked dependent obligation is undefined.
- (vi) *failed* rejects the affected Mirror/Transfer path when that path is invoked.

This projection is declared locally for Appendix L and prevents L-local tokens from being consumed as undeclared Core or CR comparison modes. It is consistent with the declared-enumeration discipline of the Technical Extension layer, but it does not rely on any unverified cross-appendix token import.

For comparison with Appendix K terminology, L refines the K controlled-commutation vocabulary as follows: *strict* agrees with K-strict comparison; *zero-cost* is the Mirror/Transfer specialization of exact-isomorphic or coherent zero-defect comparison; *metric-controlled* and *two-cell-controlled* are typed refinements of K-controlled comparison; *advisory-only* is non-verdict advisory comparison; *unavailable* covers the K uncontrolled/undefined comparison cases when no admissible comparison record exists; and *failed* agrees with a failed affected route. The absence of a primitive L-token *exact-isomorphic* is intentional: L separates strict equality from weak zero-cost equality with explicit coherence.

Definition 5.19 (Mirror/Transfer commutation defect). A Mirror/Transfer commutation defect is a typed discrepancy object

$$\Delta_{\text{MT}}$$

measuring the difference between the two composites of a Mirror or Transfer comparison square. Its codomain may be metric, categorical, homotopical, 2-cell-valued, scalar advisory, or ledger-valued, but that codomain must be declared.

Proposition 5.20 (Untyped defect notation is non-audit-readable). *The notation Δ_{MT} cannot be consumed by a certificate unless its codomain, scope, comparison rule, aggregation rule, and failure behavior are declared.*

Proof. A symbol without typed semantics does not tell the verifier whether it is an equality witness, a metric bound, a 2-cell, a scalar advisory value, or a ledger component. Those have different proof obligations. $\square$

5.5 Mirror/Transfer 2-cells

Definition 5.21 (Mirror 2-cell). A Mirror 2-cell is a 2-morphism or weak comparison

$$\alpha_{\text{Mir}} : Q^\vee \circ \text{Mir} \Rightarrow \text{Mir}_Q \circ Q$$

with declared orientation, source composite, target composite, scope, coherence requirement, and defect semantics.

Definition 5.22 (Transfer 2-cell). A Transfer 2-cell is a 2-morphism or weak comparison

$$\alpha_{\text{Tr}} : \overline{Q} \circ \text{Tr} \Rightarrow \text{Tr}_Q \circ Q$$

with the same orientation and coherence data required for a Mirror 2-cell.

Definition 5.23 (Mirror/Transfer 2-cell cost). A Mirror/Transfer 2-cell cost is a typed value

$$\delta(\alpha)$$

attached to a declared 2-cell or weak comparison. It is not automatically a metric-ledger element unless it is already valued in the active ledger quantale or converted by a certified map.

Definition 5.24 (Zero-cost 2-cell). A Mirror/Transfer 2-cell is zero-cost when $\delta(\alpha) = 0$ in its declared cost semantics and that zero is certified to identify the compared composites for the declared purpose.

Proposition 5.25 (Weak comparison is not equality). *A Mirror/Transfer 2-cell witnesses controlled comparison, not equality, unless it is zero-cost and its semantics explicitly identify zero cost with strict or coherent equality.*

Proof. A nonzero or merely weak 2-cell records a comparison object. Equality requires either strict equality or a zero-cost equivalence theorem with coherence. $\square$

Proposition 5.26 (Orientation and coherence are load-bearing). *A 2-cell with the wrong orientation, missing naturality, or missing pasting coherence cannot be used as a substitute for the required Mirror/Transfer comparison.*

Proof. Controlled commutation compares specific composites. Reversing direction or omitting coherence changes the data and therefore the theorem consumed by the certificate. $\square$

5.6 Mirror/Transfer ledger namespace

Definition 5.27 (Mirror/Transfer 2-cell ledger component). Let $(V, \oplus, 0_V, \preceq)$ be the active metric-ledger quantale. A Mirror/Transfer 2-cell ledger component is an element

$$\mathfrak{d}_{\text{MT}} \in V$$

constructed from a finite or summable family of declared Mirror/Transfer cells and certified conversions of their costs into V .

Definition 5.28 (Mirror/Transfer ledger namespace). The symbols $\mathfrak{d}_{\text{MT}}$, $\Delta_{\text{Eval}, \text{MT}}$, and $\Delta_{\phi, \text{MT}}$ do not name independent ledgers. When consumed by Core audit, they denote typed subcomponents of the declared metric ledger $\mathfrak{d}_{\text{met}}$. The aggregate $\mathfrak{d}_{\text{MT}}$ already includes every declared Mirror/Transfer 2-cell contribution after certified conversion into the active quantale; no separate orphan two-cell component symbol is used in this appendix. If a successor product-ledger appendix refines these slots, it must supply an inclusion, scalarization, or aggregation map into the Core metric-ledger quantale used by the strict budget test.

Definition 5.29 (Mirror/Transfer ledger entry). A Mirror/Transfer ledger entry records the diagram, compared composites, 2-cells, cost semantics, conversion theorem, aggregation rule, ledger slot, scope, artifact references, and failure behavior.

Definition 5.30 (Dependency-aware Mirror/Transfer aggregation). A Mirror/Transfer aggregation record separates primitive cell costs, collapse-transfer defects, window-transfer defects, realization-transfer defects, tower-artifact transfer defects, and scalar advisory measurements. It specifies which bounds are independent, nested, or already included downstream, and forbids double counting or omission.

Theorem 5.31 (Mirror/Transfer defects affect Core only through certified ledger entries). *A nonzero Mirror/Transfer defect may affect Core audit only after it has been converted into the declared verifier-side ledger at the declared scope. The raw Mirror/Transfer comparison remains non-verdict data.*

Proof. Core audit consumes repaired evaluations, eligibility records, Type IV artifacts and diagnostics, comparison records, required artifacts, and ledger components. A raw Mirror/Transfer cost is not one of these until a certified ledger entry is created. $\square$

Proposition 5.32 (Mixed-codomain aggregation is ill-typed). *An expression aggregating a scalar Mirror/Transfer score directly with a metric-ledger component is ill-typed unless a conversion theorem first maps the scalar or 2-cell cost into the active ledger quantale.*

Proof. The operands live in different codomains. The operation $\oplus$ is defined only in the declared ledger quantale or under a specified product-ledger map. $\square$

5.7 Mirror/Transfer with collapse, windows, and repaired evaluation

Definition 5.33 (Mirror-collapse comparison). A Mirror-collapse comparison compares the two routes

$$X \mapsto \text{Mir}(X) \mapsto T_\tau^{\text{bar}} \text{Mir}(X)$$

and

$$X \mapsto T_\tau^{\text{bar}} X \mapsto \text{Mir}_\tau(T_\tau^{\text{bar}} X),$$

whenever both expressions are defined in the declared profile regime.

Definition 5.34 (Transfer-collapse comparison). A Transfer-collapse comparison compares the corresponding routes through Tr and T_τ^{bar} .

Definition 5.35 (Collapse-transfer defect). The collapse-transfer defect is the Mirror/Transfer commutation defect associated with moving barcode-level threshold deletion across a Mirror or Transfer operation. It is denoted $\Delta_{T, \text{MT}}$.

Theorem 5.36 (No automatic collapse transfer). *Mirror/Transfer operations do not commute with T_τ^{bar} by default. A certificate may use such a replacement only under strict equality, zero-cost coherent equality, or a certified ledgered defect.*

Proof. Barcode-level threshold deletion is not a global exact functor, and Mirror/Transfer operations may change bar lengths, multiplicities, supports, or realization data. Hence commutation must be separately proved or budgeted. $\square$

Definition 5.37 (Mirror-window comparison). A Mirror-window comparison compares

$$W_W \text{Mir}(X) \quad \text{and} \quad \text{Mir}_W(W_W X).$$

The Transfer-window comparison is defined analogously with Tr .

Definition 5.38 (Window-transfer defect). The window-transfer defect is the declared discrepancy $\Delta_{W,MT}$ associated with moving window restriction across a Mirror or Transfer operation.

Proposition 5.39 (Window transfer is not endpoint bookkeeping). *A nonzero Mirror/Transfer window defect cannot be dismissed as endpoint bookkeeping unless a theorem proves that it vanishes under the declared endpoint convention.*

Proof. Window restriction can change bar lengths and supports. That can change threshold deletion and any subsequent repaired profile. $\square$

Definition 5.40 (Mirror/Transfer evaluation comparison). A Mirror/Transfer evaluation comparison compares evaluating after Mirror/Transfer with transporting the already repaired evaluation:

$$\text{Eval}_{i,W,\tau}(\text{MT}(X)) \quad \text{versus} \quad \text{MT}_{i,W,\tau}(\text{Eval}_{i,W,\tau}(X)),$$

where MT denotes the invoked Mirror or Transfer operation.

Definition 5.41 (Evaluation-transfer defect). The evaluation-transfer defect $\Delta_{\text{Eval},MT}$ measures the discrepancy between the two routes in Definition 5.40.

Theorem 5.42 (Evaluation-transfer defect must be budgeted). *If a certificate replaces one repaired evaluation route by another under nonzero Mirror/Transfer defect, then the defect must enter the declared metric ledger before the replacement is consumed.*

Proof. The repaired evaluation is the Core-relevant profile. Replacing it by another route under nonzero discrepancy changes the verifier-side evidence, so the discrepancy is a ledger obligation. $\square$

Remark 5.43 (Preferred Core route). The preferred route is to compute $\text{Eval}_{i,W,\tau}$ directly on the declared Core-readable output. Mirror/Transfer shortcuts require an explicit comparison status and ledger treatment.

5.8 Predicate transfer and threshold-separated use

Definition 5.44 (Mirror/Transfer metric replacement). A Mirror/Transfer metric replacement between repaired profiles M and N records a certified bound

$$d_B(M, N) \leq \varepsilon$$

plus the source theorem, profile identities, threshold when relevant, and ledger destination.

Theorem 5.45 (Metric Mirror/Transfer replacement is not equality). *A certified Mirror/Transfer bound $d_B(M, N) \leq \varepsilon$ does not transfer equality-based conclusions, zero/nonzero classification, admissibility, Ext¹-eligibility, or Type IV cleanliness unless a theorem transports the specific predicate under that bound.*

Proof. Metric closeness is quantitative. The listed conclusions are typed predicates with non-scalar hypotheses. $\square$

Theorem 5.46 (Threshold-separated Mirror/Transfer zero transfer). *Let E_L and E_R be post-threshold repaired profiles obtained from two Mirror/Transfer routes at threshold τ . If*

$$d_B(E_L, E_R) \leq \varepsilon \quad \text{and} \quad 2\varepsilon < \tau,$$

then

$$E_L = 0 \quad \Longleftrightarrow \quad E_R = 0.$$

Proof. After threshold deletion, any surviving finite bar has length strictly greater than τ , and an infinite bar cannot be matched to the diagonal at finite cost. If one side is zero and the other nonzero, a surviving bar has diagonal cost greater than $\tau/2 > \varepsilon$, contradicting the bottleneck bound. The argument is symmetric. $\square$

Proposition 5.47 (Budget pass alone does not transfer Mirror/Transfer predicates). *A Mirror/Transfer defect that passes the scalar budget test does not by itself transfer repaired vanishing, admissible representatives, Ext¹-discharge, Type IV diagnostics, or tower-artifact validity.*

Proof. The budget test controls discrepancy; the listed predicates require additional theorem or artifact records. $\square$

5.9 Realization, eligibility, and representatives

Definition 5.48 (Mirror-realization comparison). A Mirror-realization comparison compares

$$\mathcal{R}(\text{Mir}(F)) \quad \text{and} \quad \text{Mir}_D(\mathcal{R}(F)),$$

where Mir_D is a declared realized-level mirror operation. Transfer-realization comparisons are defined analogously.

Definition 5.49 (Realization-transfer defect). The realization-transfer defect $\Delta_{\mathcal{R}, \text{MT}}$ measures the discrepancy associated with moving realization across a Mirror or Transfer operation.

Theorem 5.50 (Mirror/Transfer realization does not replace eligibility). *A controlled Mirror/Transfer realization comparison does not replace the eligible realization requirements for the one-way PH₁-to-Ext¹ bridge.*

Proof. Eligibility requires the relevant realized object to lie in the declared amplitude regime and satisfy the edge-realization condition. A Mirror/Transfer comparison involving realization does not, by itself, prove those conditions. $\square$

Definition 5.51 (Mirror/Transfer representative support). A Mirror/Transfer diagram provides representative support when it helps construct or compare an admissible representative $C_{\tau, W}F$. It is valid for Core use only when the standard compatibility

$$\mathbf{P}_i(C_{\tau, W}F) \cong \text{Eval}_{i, W, \tau}(F)$$

is separately verified or transported by a registered theorem.

Proposition 5.52 (Representative support is not representative validity). *A Mirror/Transfer representative support diagram does not make a representative admissible merely by exhibiting a comparison route.*

Proof. Admissibility is a condition on the repaired profile compatibility of the representative. A support diagram may help construct evidence, but it is not the evidence unless it proves that compatibility. $\square$

5.10 Tower comparison artifacts and Type IV diagnostics

Definition 5.53 (Mirror/Transfer tower comparison support). A Mirror/Transfer diagram provides tower comparison support if it helps construct, compare, or transport a declared tower comparison artifact

$$\phi_{i, W, \tau} : \text{ColimProfile}_{i, W, \tau}(F_\bullet) \longrightarrow \text{ApexProfile}_{i, W, \tau}(F_\infty).$$

Definition 5.54 (Tower-artifact transfer defect). The tower-artifact transfer defect $\Delta_{\phi, \text{MT}}$ measures the cost of transporting or comparing tower comparison artifacts through a Mirror/Transfer pipeline.

Theorem 5.55 (Mirror/Transfer does not replace tower comparison artifacts). *A Mirror/Transfer diagram does not replace the required tower comparison artifact $\phi_{i,w,\tau}$. If Type IV is invoked, the artifact and its kernel/cokernel defect profiles must still be declared.*

Proof. Monitored Type IV diagnostics are computed from the kernel and cokernel defect profiles of the declared artifact. A Mirror/Transfer diagram may help construct or compare that artifact, but it is not the artifact unless it supplies the same typed data. $\square$

Definition 5.56 (Mirror/Transfer Type IV proxy). A Mirror/Transfer Type IV proxy is a Mirror/Transfer-derived signal suggesting possible monitored generic tower defect.

Theorem 5.57 (Mirror/Transfer proxy is not Type IV diagnosis). *A Mirror/Transfer Type IV proxy does not determine whether*

$$(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$$

or its negation. Diagnosis requires the actual tower comparison artifact and terminal-generic kernel/cokernel computation.

Proof. A proxy does not compute the required kernel and cokernel defect profiles unless it is itself realized as the diagnostic artifact with all required data. $\square$

Remark 5.58 (Canonical cokernel notation). The cokernel diagnostic is denoted by ν . No normative Latin u is used for Type IV cokernel data in this appendix.

5.11 Certificate attachments and statuses

Definition 5.59 (Mirror/Transfer certificate attachment). A Mirror/Transfer certificate attachment is an optional section of a certificate record documenting Mirror/Transfer diagrams, comparison status, 2-cell data, defect semantics, ledgers, and artifacts used to support or compare pipeline data.

Definition 5.60 (Mirror/Transfer manifest entry). A Mirror/Transfer manifest entry records source and target domains, operation identity, variance, realization target, compared composites, comparison mode, 2-cells, cost conversion, ledger aggregation, evaluation-transfer defect, tower-artifact defect, final status, artifacts, permitted use, forbidden reading, and failure behavior.

Definition 5.61 (Mirror/Transfer comparison status). A Mirror/Transfer comparison status is one of *strict*, *zero-cost*, *controlled*, *advisory*, *unavailable*, *failed*, or *undefined*. These are L-local statuses and project to Core statuses only through Definition 5.18 and the invoked gate contract. The status-to-mode convention is: *strict* uses local mode *strict*; *zero-cost* uses local mode *zero-cost*; *controlled* must declare either *metric-controlled* or *two-cell-controlled*; *advisory* uses *advisory-only*; *unavailable* uses *unavailable*; *failed* uses *failed*; and *undefined* has no resolved comparison mode and leaves the invoked affected obligation undefined.

Theorem 5.62 (Only strict, zero-cost, or controlled comparisons may support certification). *A Mirror/Transfer comparison may support a Core-adjacent certificate only if its status is strict, zero-cost, or controlled with ledgered cost. Advisory, unavailable, failed, and undefined comparisons cannot support the affected certified path.*

Proof. Strict and zero-cost comparisons supply no defect for the declared purpose. Controlled comparisons supply a defect and the ledger treatment required for audit. The remaining statuses lack certified support or explicitly fail. $\square$

Theorem 5.63 (Mirror/Transfer attachment noninterference). *Two certificate records with the same verifier-side projection have the same Core atomic status and failure route even if their Mirror/Transfer attachments differ.*

Proof. Mirror/Transfer attachments are outside the verdict projection unless they create new verifier-side records such as certified comparison records or ledger components. $\square$

5.12 Adjusted budget and failure behavior

Definition 5.64 (Mirror/Transfer adjusted budget). If a certificate consumes Mirror/Transfer comparison data, the adjusted metric ledger is the complete aggregate

$$\mathbf{d}_{\text{adj}} := \mathbf{d}_{\text{base}} \oplus \mathbf{d}_{\text{MT}} \oplus \mathbf{d}_{\text{Eval,MT}} \oplus \mathbf{d}_{\phi,\text{MT}},$$

with absent non-invoked components omitted and all invoked components aggregated only by the declared ledger law and dependency graph. Its scalarized budget is

$$\Sigma\delta_{\text{adj}} := \text{val}_B(\mathbf{d}_{\text{adj}}).$$

Theorem 5.65 (Mirror/Transfer evidence must pass the adjusted budget). *A certificate record relying on nonzero Mirror/Transfer comparison cost may pass Audit Mode only if the complete adjusted scalar budget satisfies*

$$\Sigma\delta_{\text{adj}} = \text{val}_B(\mathbf{d}_{\text{adj}}) < \text{gap}_{\tau}.$$

The symbol denotes the full adjusted aggregate, not the Mirror/Transfer component alone.

Proof. All admissible deviations consumed by a certificate enter the total budget. The strict gap condition is applied to that complete aggregate. $\square$

Proposition 5.66 (Failed Mirror/Transfer route rejects the affected path). *If the Mirror/Transfer defect required by an invoked replacement exceeds the available budget margin, then the affected Mirror/Transfer-supported path cannot pass unless an alternative verified path is provided.*

Proof. The invoked path requires including the defect. If that inclusion violates the strict budget condition, the path fails its audit obligation. $\square$

5.13 Interface status and problem-specific implications

Definition 5.67 (Problem-specific Mirror/Transfer implication). A problem-specific Mirror/Transfer implication is a statement of the form

$$\text{Mirror/Transfer compatibility} \implies \text{problem-specific conclusion}.$$

Theorem 5.68 (Problem-specific Mirror/Transfer implications are Spec by default). *Every arithmetic, PDE, geometric, mirror-symmetric, categorical, cryptographic, or transfer-theoretic conclusion derived from Mirror/Transfer language is [Spec] by default unless separately promoted and reproved under the Core guard-rails and the relevant interface protocol.*

Proof. Such conclusions import external semantics beyond the Technical Extension ledger. Chapter 8 interface discipline requires explicit hypotheses and transport theorems before any problem-specific claim can be promoted. $\square$

Remark 5.69 (Mirror/Transfer does not solve external problems). A controlled Mirror/Transfer diagram may be mathematically useful. It does not by itself prove an external theorem or solve a problem-specific target.

5.14 Appendix L package theorem

Theorem 5.70 (Appendix L Mirror/Transfer ledger package). *Within the Technical Extension layer:*

- (i) *Mirror and Transfer operations must have declared source, target, variance, realization target, and failure behavior.*
- (ii) *Raw Mirror/Transfer outputs are not Core objects.*
- (iii) *L-local comparison modes project to Core comparison modes only through the declared projection rule.*
- (iv) *Mirror/Transfer 2-cells witness controlled comparison, not equality, unless zero-cost coherent equality is proved.*
- (v) *Nonzero Mirror/Transfer defects affect Core only through certified metric-ledger entries.*
- (vi) *Collapse, window, repaired-evaluation, and realization transfer are not automatic.*
- (vii) *Metric Mirror/Transfer replacement is not equality, and zero/nonzero transfer requires a predicate-transport theorem such as the threshold-separated theorem.*
- (viii) *Mirror/Transfer realization does not replace eligibility.*
- (ix) *Mirror/Transfer representative support does not replace representative compatibility.*
- (x) *Mirror/Transfer diagrams do not replace tower comparison artifacts or Type IV diagnostics.*
- (xi) *Only strict, zero-cost, or controlled ledger-certified comparisons may support certification.*
- (xii) *The adjusted budget must satisfy the strict gap condition.*
- (xiii) *Problem-specific Mirror/Transfer implications are [Spec] by default.*

Proof. The items follow respectively from Sections L.2–L.13: typed settings and targets, comparison projection, 2-cell control, ledger namespace, collapse/window/evaluation/realization comparisons, threshold-separated transfer, eligibility and representative boundaries, tower and Type IV boundaries, status rules, budget rules, and interface discipline. $\square$

5.15 Explicit exclusions

The following are not asserted in Appendix L.

- No Mirror operation is assumed to be a strict equivalence unless explicitly declared and proved.
- No Transfer operation is assumed to preserve Core-relevant data unless explicitly declared and proved.
- No Mirror/Transfer comparison is treated as commuting without strict, zero-cost, or controlled status.
- No nonzero Mirror/Transfer defect is ignored when used in certification.
- No Mirror/Transfer 2-cell replaces $B\text{-Gate}^+$.
- No Mirror/Transfer 2-cell replaces $\text{Eval}_{1,W,\tau}(F) = 0$.
- No Mirror/Transfer 2-cell replaces $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$.
- No Mirror/Transfer 2-cell replaces the tower comparison artifact $\phi_{i,W,\tau}$.
- No scalar or 2-cell cost is a metric-ledger component without certified conversion into the active ledger quantale.

- No Mirror/Transfer realization comparison replaces eligibility.
- No Mirror/Transfer evaluation comparison replaces direct repaired evaluation unless strict, zero-cost, or controlled status is declared.
- No advisory Mirror/Transfer diagram supports Core acceptance.
- No global exactness of T_τ^{bar} , no positive-threshold Serre localization, and no global strict filtered lift C_τ is assumed.
- No problem-specific arithmetic, PDE, geometric, categorical, mirror-symmetric, cryptographic, or transfer-theoretic conclusion is proved here.

5.16 Provenance and status

The material in this appendix depends on the controlled-commutation framework of Appendix K, the repaired barcode-level collapse and window cautions of Appendix A, the admissible representative discipline of Appendix B, the eligible realization regime of Appendix C, the monitored Type IV diagnostics of Appendix D, the audit and ledger discipline of Chapter 6, the interface protocol of Chapter 8, the manifest discipline of Appendix G, and the non-verdict and measurement-separation principles of Appendices H–J.

Status labels for Appendix L. *Technical extension principles:* Definitions 5.1, 5.2.

Technical extension definitions: Definitions 5.5, 5.6, 5.7, 5.8, 5.10, 5.11, 5.13, 5.15, 5.16, 5.17, 5.18, 5.19, 5.21, 5.22, 5.23, 5.24, 5.27, 5.28, 5.29, 5.30, 5.33, 5.34, 5.35, 5.37, 5.38, 5.40, 5.41, 5.44, 5.48, 5.49, 5.51, 5.53, 5.54, 5.56, 5.59, 5.60, 5.61, 5.64, 5.67.

Technical extension propositions/theorems: Proposition 5.9, Proposition 5.12, Proposition 5.14, Proposition 5.20, Proposition 5.25, Proposition 5.26, Theorem 5.31, Proposition 5.32, Theorem 5.36, Proposition 5.39, Theorem 5.42, Theorem 5.45, Theorem 5.46, Proposition 5.47, Theorem 5.50, Proposition 5.52, Theorem 5.55, Theorem 5.57, Theorem 5.62, Theorem 5.63, Theorem 5.65, Proposition 5.66, Theorem 5.68, Theorem 5.70.

Operational cautions: Remarks 5.3, 5.4, 5.43, 5.58, 5.69.

Explicit exclusions: Section 5.15.

6 Appendix M: Quantale and Ledger Semantics

M.1. Scope and non-Core status

Definition 6.1 (Ledger sovereignty principle). A ledger semantics records how declared discrepancies, tolerances, costs, and controlled deviations are aggregated in a certificate record. It does not create, repair, or replace any Core verifier-side object. In particular, a ledger policy does not replace B-Gate⁺, the repaired topological condition

$$\text{Eval}_{1,W,\tau}(F) = 0,$$

canonical Type IV cleanliness

$$(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0),$$

the tower comparison artifact $\phi_{i,W,\tau}$, an admissible representative $C_{\tau,W}F$, or any invoked categorical eligibility condition.

Definition 6.2 (No hidden ledger-default principle). There is no global hidden default ledger semantics in the Technical Extension layer. Ordinary addition, maximum aggregation, weighted aggregation, product ledgers, lexicographic ledgers, Boolean/status ledgers, and hybrid ledgers are distinct choices. A certificate record must declare the chosen value domain, order, aggregation law, scalarization into the bottleneck budget when needed, and strict comparison convention before the budget verdict is evaluated.

Remark 6.3 (Technical extension status). This appendix is a catalog and governance layer for budget semantics. It supplies admissible ways to type and aggregate ledger components, but it does not choose a ledger globally and does not enlarge Chapters 1–8.

Remark 6.4 (Relation to Appendix G). Appendix G requires a ledger policy in the manifest. Appendix M specifies what such a policy must contain when numerical, qualitative, product, commutation, discretization, measurement, or Mirror/Transfer components are used.

M.2. Core metric-ledger contract

Definition 6.5 (Ledger value system). A ledger value system is a tuple

$$(V, \preceq, \oplus, 0_V)$$

where $(V, \oplus, 0_V)$ is a commutative monoid or a declared ordered aggregation algebra, and $\preceq$ is the order used to compare budget values. When arbitrary joins and distributivity are required, the value system may be strengthened to a commutative quantale.

Definition 6.6 (Commutative quantale). A commutative quantale is a tuple

$$(V, \preceq, \oplus, 0_V, \bigvee)$$

such that $(V, \preceq)$ is a complete lattice, $(V, \oplus, 0_V)$ is a commutative monoid, and $\oplus$ preserves arbitrary joins in each variable. Quantale structure is optional unless a certificate or theorem invokes infinite aggregation, supremum aggregation, or quantale-valued stability.

Remark 6.7 (Reading older quantale terminology). Earlier passages sometimes call a four-tuple $(V, \oplus, 0_V, \preceq)$ a metric-ledger quantale. In Appendix M this is read as a ledger value system unless the complete-lattice and join-distribution data of Definition 6.6 are explicitly supplied.

Definition 6.8 (Core metric-ledger scalarization). A Core metric ledger additionally supplies a bottleneck scalarization

$$\text{val}_B : V \longrightarrow [0, +\infty]$$

satisfying the three Core conditions:

$$d \preceq e \implies \text{val}_B(d) \leq \text{val}_B(e), \quad \text{val}_B(0_V) = 0,$$

and

$$\text{val}_B(d \oplus e) \leq \text{val}_B(d) + \text{val}_B(e).$$

Thus $\text{val}_B(d)$ is a monotone, normalized, subadditive upper bound, in the registered bottleneck convention, for the actual profile discrepancy represented by d . A component that lacks such a bound may belong to an auxiliary ledger, but it cannot be consumed in the strict Core metric budget.

Definition 6.9 (Strict Core budget comparison). For a declared gate scope and protected family $\mathcal{B}$, the Core metric-ledger element is denoted

$$\mathfrak{d}_{\text{met}} \in V.$$

The strict Core budget comparison is

$$\text{val}_B(\mathfrak{d}_{\text{met}}) < \text{gap}_\tau(\mathcal{B}).$$

The boundary case of equality is not an audited pass. The notation $\mathfrak{d}_{\text{tot}}$ is permitted only as a local alias for this gate-scoped $\mathfrak{d}_{\text{met}}$ after the scope, protected family, and component decomposition have been declared. If a non-scalar or product ledger is used, the certificate must either compare all coordinates by a declared product gap or provide an admissible scalarization into this strict comparison.

Proposition 6.10 (Ledger pass is typed and scope-bound). *A budget pass is valid only for the declared ledger value system, scalarization, components, aggregation law, safety margin, and monitored scope. The same numerical entries do not automatically pass under a different ledger semantics or a different monitored profile.*

Proof. The budget element is not just a number; it is a typed aggregate with a declared interpretation. Changing the value system, scalarization, or scope changes the proposition being checked. $\square$

Proposition 6.11 (Budget pass is necessary but not sufficient for B-Gate). *The inequality*

$$\text{val}_B(\mathfrak{d}_{\text{met}}) < \text{gap}_\tau(\mathcal{B})$$

does not imply B-Gate⁺ pass.

Proof. B-Gate⁺ also requires the repaired topological clause, invoked categorical clauses, Type IV artifacts and diagnostics, and required certificate artifacts. A budget comparison is one conjunctive audit condition, not the whole gate. $\square$

Proposition 6.12 (Structural gate pass is not audit pass without a ledger). *Even if the structural clauses of B-Gate⁺ pass, Audit Mode does not pass unless the declared metric ledger is reconstructed and the strict budget comparison succeeds.*

Proof. Audit Mode includes both structural verifier-side obligations and the strict metric-budget obligation. Omitting the ledger leaves the audited verdict undefined. $\square$

M.3. Ledger policies, components, and support records

Definition 6.13 (Ledger policy). A ledger policy declares: the value system; the order; the aggregation law; the unit; the scalarization or coordinatewise gap rule when Core metric comparison is invoked; the strict comparison convention; the component namespace; dependency and no-double-counting rules; the total budget construction; and artifact references supporting the component values.

Definition 6.14 (Ledger component). A ledger component is a typed value

$$\mathfrak{d}^\alpha \in V_\alpha$$

associated with a specific source of deviation, approximation, uncertainty, controlled comparison, representative discrepancy, realization discrepancy, artifact transport cost, or converted advisory datum.

Definition 6.15 (Metric-admissible component). A component $\mathfrak{d}^\alpha$ is metric-admissible for the Core budget only when it has a support record proving an actual bottleneck discrepancy contribution or a certified conversion into the declared metric ledger V . Its resulting image is denoted by the corresponding subcomponent of $\mathfrak{d}_{\text{met}}$.

Definition 6.16 (Standard component namespace). The Main canonical metric-ledger decomposition uses the slots

$$\mathfrak{d}_{\text{alg}}, \mathfrak{d}_{\text{disc}}, \mathfrak{d}_{\text{meas}}, \mathfrak{d}_{\text{rep,met}}, \mathfrak{d}_{\text{real,met}}, \\ \mathfrak{d}_{\text{cmp}}, \mathfrak{d}_{\text{crop}}, \mathfrak{d}_{\text{cont}}, \mathfrak{d}_{\text{ov}}, \mathfrak{d}_{\text{rst}}.$$

Appendix M also permits named subcomponents and aliases,

$$\mathfrak{d}_{\text{eval}}, \mathfrak{d}_{\text{rep}}, \mathfrak{d}_{\text{elig}}, \mathfrak{d}_{\phi}, \mathfrak{d}_{\text{comm}}, \mathfrak{d}_{\text{MT}}, \\ \mathfrak{d}_{\text{adv}}, \mathfrak{d}_{\text{src}}, \mathfrak{d}_{\text{ser}},$$

but each named value must be placed into, or explicitly mapped into, the Main canonical decomposition before it is consumed by the strict Core budget.

Definition 6.17 (Main decomposition and alias discipline). At a gate scope the canonical metric ledger has the form

$$\mathfrak{d}_{\text{met}} = \mathfrak{d}_{\text{alg}} \oplus \mathfrak{d}_{\text{disc}} \oplus \mathfrak{d}_{\text{meas}} \oplus \mathfrak{d}_{\text{rep,met}} \oplus \mathfrak{d}_{\text{real,met}} \\ \oplus \mathfrak{d}_{\text{cmp}} \oplus \mathfrak{d}_{\text{crop}} \oplus \mathfrak{d}_{\text{cont}} \oplus \mathfrak{d}_{\text{ov}} \oplus \mathfrak{d}_{\text{rst}},$$

up to the declared aggregation law and omitted zero components. The following alias rules are normative.

1. $\mathfrak{d}_{\text{rep}}$ is a readable alias for $\mathfrak{d}_{\text{rep,met}}$ only when the component is a metric representative discrepancy.
2. $\mathfrak{d}_{\text{elig}}$ denotes eligibility-related bookkeeping; any metric realization distortion entering the Core budget must be placed in $\mathfrak{d}_{\text{real,met}}$ or in a declared subcomponent mapped to it.
3. $\mathfrak{d}_{\text{cmp}}$ contains comparison-mode discrepancies, including certified exact/metric-gap comparison costs, controlled-commutation conversions, and comparison-artifact discrepancies when they are used as metric bounds.
4. $\mathfrak{d}_{\text{crop}}$ contains crop, truncation, endpoint, and window-edge contributions required by window-first claims.
5. $\mathfrak{d}_{\text{cont}}$ contains continuation, limiting, or continuity-route discrepancy not already recorded as restart, overlap, discretization, or crop error.
6. $\mathfrak{d}_{\text{eval}}$ is an umbrella label for repaired-evaluation route discrepancy; for Core consumption it must be decomposed into $\mathfrak{d}_{\text{cmp}}$, $\mathfrak{d}_{\text{crop}}$, $\mathfrak{d}_{\text{cont}}$, or another declared canonical slot.
7. $\mathfrak{d}_{\phi}$ records tower-artifact transport or comparison cost; it does not replace $\phi_{i,W,\tau}$, and its metric portion must be placed in $\mathfrak{d}_{\text{cmp}}$ or another declared canonical coordinate.

No second authority for component names is created by Appendix M; it only supplies aliases and refinements compatible with the Main decomposition.

Definition 6.18 (Total metric ledger). For invoked metric-admissible components, the total metric ledger is the gate-scoped aggregate

$$\mathfrak{d}_{\text{met}} = \mathfrak{d}^1 \oplus \dots \oplus \mathfrak{d}^m$$

or the declared finite, countable, sequential, parallel, windowwise, overlapwise, or product aggregation determined by the ledger policy and compatible with Definition 6.17. If no metric-admissible components are invoked, $\mathfrak{d}_{\text{met}} = 0_V$. The notation $\mathfrak{d}_{\text{tot}}$ is a non-normative local synonym only when it is explicitly identified with this $\mathfrak{d}_{\text{met}}$.

Definition 6.19 (Component support record). A component support record identifies the component source, mathematical scope, value, unit or type, profile or artifact identities, theorem or computation supporting the bound, aggregation placement, dependency edges, whether the component is verifier-side or converted, and failure behavior.

Definition 6.20 (Dependency-aware aggregation). A ledger aggregation is dependency-aware when it records which components are independent, nested, alternative, shared, or already included in other components. The aggregation law must not count the same primitive discrepancy twice and must not omit an invoked primitive discrepancy.

Proposition 6.21 (Missing component typing blocks audit readability). *If a certificate uses a ledger component without its type, source, support artifact, conversion witness when needed, or aggregation rule, then the budget comparison is not audit-readable.*

Proof. The verifier cannot reconstruct either the component value or its contribution to $\mathfrak{d}_{\text{tot}}$. Hence the strict comparison is not a checkable statement. $\square$

Proposition 6.22 (No silent component dropping). *An invoked discrepancy source may be omitted from the total ledger only if a theorem proves it is zero, already included in another declared component, or irrelevant to the monitored scope.*

Proof. Otherwise the displayed total is not an upper bound for the actual discrepancy being audited. $\square$

M.4. Additive, max, and weighted scalar ledgers

Definition 6.23 (Additive scalar ledger). The additive scalar ledger uses

$$V = [0, +\infty], \quad \oplus = +, \quad 0_V = 0,$$

with the usual order and identity scalarization. The pass condition is

$$\sum_{\alpha} \mathfrak{d}^{\alpha} < \text{gap}_{\tau}.$$

Definition 6.24 (Max scalar ledger). The max scalar ledger uses

$$V = [0, +\infty], \quad \oplus = \max, \quad 0_V = 0,$$

with the usual order and identity scalarization. The pass condition is

$$\max_{\alpha} \mathfrak{d}^{\alpha} < \text{gap}_{\tau}.$$

Definition 6.25 (Weighted scalar ledger). A weighted scalar ledger uses

$$\text{val}_B(\mathfrak{d}_{\text{met}}) = \sum_{\alpha} w_{\alpha} \text{val}_B(\mathfrak{d}^{\alpha}), \quad w_{\alpha} \geq 0,$$

with all weights declared before the budget verdict is evaluated. The support record must prove that the weighted aggregate is still a conservative upper bound for the actual combined discrepancy on the declared scope. In particular, a zero weight or a weight below one is admissible only when the corresponding primitive discrepancy is proved zero, irrelevant, already included elsewhere, or dominated by a separate declared theorem.

Definition 6.26 (Weight policy). A weight policy specifies the components weighted, the weights, their source, whether they are fixed or calibrated, the calibration artifacts if any, and the rule forbidding post-hoc retuning after a failed budget comparison.

Proposition 6.27 (Weighted ledgers require predeclared weights). *A weighted scalar ledger is audit-admissible only when the weights are declared before the pass/fail comparison and supported by the policy artifacts.*

Proof. Changing weights can change the verdict. Post-hoc tuning would change the audit semantics after observing the result. $\square$

Proposition 6.28 (Weighted ledgers require dominance). *A weighted scalar ledger is audit-admissible only when the weighted aggregate dominates the actual composed discrepancy it is claimed to bound, or when every down-weighted component is justified by zero, irrelevance, inclusion, or a separate domination theorem.*

Proof. A weight can reduce the displayed scalar below a primitive discrepancy. Without a domination theorem or an inclusion/irrelevance proof, the displayed value need not be an upper bound for the audited profile discrepancy. $\square$

Remark 6.29 (Choosing between additive and max ledgers). The additive ledger is appropriate for sequential discrepancies that accumulate by a triangle inequality. The max ledger is appropriate for uniform or parallel controls where the worst active component dominates. The choice is a theorem or policy matter, not a cosmetic notation change.

M.5. Product, lexicographic, and hybrid ledgers

Definition 6.30 (Product ledger). Given value systems $(V_j, \preceq_j, \oplus_j, 0_j)$, the product ledger has value domain

$$V = V_1 \times \cdots \times V_r$$

with componentwise aggregation unless another order is declared. A product gap is a tuple $(g_1, \dots, g_r)$ or a declared scalarization to a single gap.

Proposition 6.31 (Product pass is coordinatewise unless scalarized). *Under the componentwise product ledger, the budget passes only if every invoked coordinate satisfies its declared strict comparison. Collapsing a product ledger to one scalar requires an admissible conversion.*

Proof. Componentwise order defines pass coordinate by coordinate. A scalar collapse changes the order and must be justified separately. $\square$

Definition 6.32 (Lexicographic ledger). A lexicographic ledger uses a product value domain with lexicographic order. The priority order of coordinates must be declared, and the gap must be a compatible lexicographic gap or be supplied through an admissible scalarization.

Proposition 6.33 (Lexicographic priority is normative). *Changing the coordinate order of a lexicographic ledger changes the audit semantics and requires a new declared ledger policy.*

Proof. The first differing coordinate determines comparison. Reordering coordinates can reverse or change the verdict. $\square$

Definition 6.34 (Qualitative status ledger). A qualitative status ledger records states such as

strict, zero-cost, controlled, advisory, uncontrolled, failed, undefined.

Its aggregation law and projection to Core atomic status must be declared. It is not by itself a metric budget.

Remark 6.35 (Status vocabulary is local until projected). Status words used in Appendices K, L, or M are local vocabulary unless a register or manifest projects them to Core atomic states pass, reject, undefined, or not_invoked, and to the declared comparison mode. The examples above are therefore admissible only through their appendix-specific projection rules.

Definition 6.36 (Hybrid ledger). A hybrid ledger combines a metric component with one or more qualitative, provenance, availability, or status components. A hybrid pass requires both the strict metric comparison and every declared qualitative admissibility condition.

Proposition 6.37 (Status ledgers do not replace metric ledgers). *A status value such as strict, controlled, advisory, failed, or undefined does not by itself supply a bottleneck discrepancy bound. It can block or annotate a certificate, but it cannot discharge the strict metric budget without a metric component.*

Proof. The metric budget consumes values that scalarize to bottleneck bounds. A qualitative status has no such numerical interpretation unless a conversion theorem supplies one. $\square$

Remark 6.38 (Hybrid ledgers for K and L). Hybrid ledgers are often appropriate for Appendices K and L: a controlled commutation may have both a status, such as strict or controlled, and a metric cost, such as $\mathfrak{d}_{\text{comm}}$ or $\mathfrak{d}_{\text{MT}}$.

M.6. Ledger conversion and refinement

Definition 6.39 (Ledger conversion). A ledger conversion is a declared map

$$\kappa : V \longrightarrow W$$

from one ledger value system to another, together with its intended effect on aggregation, order, and pass/fail interpretation.

Definition 6.40 (Admissible ledger conversion). A ledger conversion is admissible for audit use only if it is monotone, has a declared aggregation inequality,

$$\kappa(v \oplus_V v') \preceq_W \kappa(v) \oplus_W \kappa(v'),$$

preserves or tightens the intended budget admissibility, and is declared before the converted budget verdict is evaluated.

Proposition 6.41 (No post-hoc conversion to force pass). *A ledger conversion may not be introduced after a failed comparison merely to obtain a pass.*

Proof. That would change the audit semantics after observing the outcome. The ledger policy must be part of the certificate being audited. $\square$

Definition 6.42 (Nonmonotone transformation). A transformation $\kappa : V \rightarrow W$ is nonmonotone if $v \preceq v'$ can hold while $\kappa(v) \not\preceq \kappa(v')$.

Proposition 6.43 (Nonmonotone transformations are not audit-admissible by default). *A nonmonotone transformation cannot be used in the audit budget unless a separate theorem proves that it preserves the specific pass/fail semantics at the declared scope.*

Proof. Budget audit is order-based. Nonmonotonicity can hide larger discrepancies as smaller transformed values. $\square$

Definition 6.44 (Ledger refinement). A ledger refinement replaces a coarse component by finer components with a proof that the refined aggregate is no larger than the original component in the declared order, or that the replacement is otherwise audit-equivalent.

Proposition 6.45 (Refinement improves transparency only when conservative). *Ledger refinement is audit-safe only when it preserves or tightens the total budget and exposes the component sources more precisely.*

Proof. A refinement that increases uncertainty or loosens the comparison changes the audited claim. A conservative refinement retains the old upper-bound meaning while improving traceability. $\square$

M.7. Compatibility with repaired evaluation and threshold transfer

Definition 6.46 (Evaluation-route ledger). An evaluation-route ledger records discrepancies affecting the repaired profile

$$\text{Eval}_{i,W,\tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

Its metric component is $\mathfrak{d}_{\text{eval}}$.

Proposition 6.47 (Evaluation-route discrepancies must be ledgered). *If a certificate uses an implemented, transported, discretized, mirrored, transferred, or alternate route in place of the direct repaired evaluation, every nonzero discrepancy between that route and the declared repaired profile must be included in the metric ledger.*

Proof. The repaired profile is a verifier-side object. Replacing its construction by an alternate route changes the evidence unless the discrepancy is zero; a nonzero discrepancy must therefore be budgeted. $\square$

Proposition 6.48 (Ledgered proximity does not by itself transfer vanishing). *A metric ledger bound on repaired profiles does not by itself imply zero/nonzero equivalence. Such equivalence requires an additional transfer theorem, such as a half-threshold reserve condition.*

Proof. Metric proximity and vanishing are different predicates. Short surviving bars can be matched to the diagonal unless the relevant reserve theorem excludes that possibility. $\square$

Theorem 6.49 (Ledgered half-threshold zero-transfer). *Let E and E' be barcodes with finitely many bars, possibly including infinite bars, arising as repaired post-threshold profiles at the same declared threshold τ . Suppose a metric ledger component proves*

$$d_B(E, E') \leq \varepsilon.$$

If $2\varepsilon < \tau$, then

$$E = 0 \iff E' = 0.$$

Proof. Because E and E' are post-threshold repaired profiles at threshold τ , every surviving finite bar has length strictly greater than τ , hence diagonal matching cost greater than $\tau/2$. Infinite bars cannot be matched to the diagonal at finite cost. Thus a barcode within bottleneck radius $\varepsilon < \tau/2$ of the empty barcode must itself be empty, and the argument is symmetric. $\square$

Remark 6.50 (Relation to Appendices I and J). Appendix I supplies discretization-to-profile metric transfer, and Appendix J supplies scalar measurement-to-profile restrictions. Appendix M records how the resulting metric-admissible components enter the ledger; it does not turn scalar measurement error into a metric component by itself.

M.8. Compatibility with representatives, eligibility, and Type IV artifacts

Definition 6.51 (Representative ledger). A representative ledger records discrepancies or costs associated with constructing, transporting, or comparing admissible representatives $C_{\tau,W}F$. Its metric component is $\mathfrak{d}_{\text{rep}}$.

Proposition 6.52 (Representative ledger does not replace admissibility). *A representative ledger component does not prove that $C_{\tau,W}F$ is admissible or compatible with $\text{Eval}_{i,W,\tau}(F)$.*

Proof. Admissibility is a structural compatibility condition. A ledger value may bound a discrepancy in a construction but cannot manufacture the structural record. $\square$

Definition 6.53 (Eligibility ledger). An eligibility ledger records discrepancy or verification tolerance associated with the realized object and edge-realization condition used when an Ext^1 -clause is invoked. Its metric component is $\mathfrak{d}_{\text{elig}}$ when such a component is meaningful.

Proposition 6.54 (Eligibility ledger does not replace eligibility). *An eligibility ledger component does not prove amplitude $[-1, 0]$, edge realization, or the one-way PH_1 -to- Ext^1 bridge hypotheses.*

Proof. Those requirements are structural and categorical, not merely numerical budget claims. $\square$

Definition 6.55 (Tower artifact ledger). A tower artifact ledger records discrepancies associated with constructing, transporting, approximating, or comparing the tower comparison artifact

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty).$$

Its metric component is $\mathfrak{d}_\phi$.

Proposition 6.56 (Tower artifact ledger does not replace the artifact). *A tower artifact ledger component does not replace $\phi_{i,W,\tau}$, its kernel and cokernel defect profiles, or the terminal-generic computations defining μ_{Collapse} and ν_{Collapse} .*

Proof. The Type IV diagnostic is computed from an actual morphism and its defect profiles. A cost of obtaining or transporting that morphism is a different object. $\square$

Remark 6.57 (Canonical cokernel symbol). The cokernel diagnostic is denoted by ν_{Collapse} . No Latin u_{Collapse} convention is introduced in Appendix M.

M.9. Window, overlap, restart, and summability ledgers

Definition 6.58 (Windowwise ledger). A windowwise ledger assigns a total budget $\mathfrak{d}_W$ and a gap $\text{gap}_{\tau,W}$ to each declared window W .

Definition 6.59 (Global window aggregation). For a family of windows $\{W_\alpha\}$, global aggregation is a declared operation that combines $\{\mathfrak{d}_{W_\alpha}\}$ and compares it with a declared global gap or coordinatewise gaps.

Proposition 6.60 (Windowwise pass does not imply global pass by default). *Even if every windowwise budget passes, the global budget does not pass unless a global aggregation law and global gap comparison are declared and satisfied.*

Proof. Windowwise inequalities are local. A global certificate requires a statement about the aggregated object. $\square$

Definition 6.61 (Overlap ledger). An overlap ledger records budgets used in pairwise overlap checks and their aggregation across declared overlaps.

Proposition 6.62 (Overlap ledger must match the overlap gate). *The overlap ledger is admissible only if its components match the budgets consumed by the overlap gate verdicts.*

Proof. If the ledger and gate use different values, the certificate is not reconstructible. $\square$

Definition 6.63 (Restart ledger). A restart ledger records continuation or restart costs between MECE windows or phases. Its component is $\mathfrak{d}_{\text{rst}}$.

Definition 6.64 (Summable ledger family). An infinite family of ledger components is summable when its declared aggregate exists and satisfies the relevant strict gap comparison.

Proposition 6.65 (Summability is ledger-relative). *A sequence that is summable under one ledger semantics may fail to be summable under another.*

Proof. For example, a bounded sequence may pass a max ledger while its ordinary sum diverges in an additive ledger. $\square$

M.10. Advisory and extension components

Definition 6.66 (Advisory-to-budget conversion). An advisory-to-budget conversion creates a metric-admissible component $\mathfrak{d}_{\text{adv}}$ from an advisory invariant, spectral indicator, warning flag, navigation quantity, proxy, or candidate score. It must include a theorem-backed discrepancy bound or conservative conversion.

Proposition 6.67 (Advisory quantities are budget-neutral unless converted). *An advisory quantity has no effect on the Core budget unless it is converted into a declared metric-admissible ledger component.*

Proof. The budget consumes ledger components, not advisory values. $\square$

Definition 6.68 (Discretization and measurement ledgers). Discretization and measurement ledgers record components $\mathfrak{d}_{\text{disc}}$ and $\mathfrak{d}_{\text{meas}}$, respectively. Discretization must be tied to an actual profile discrepancy or transfer witness. Measurement error must not be confused with scalar Betti or observational error unless a certified measurement-to-metric conversion is supplied.

Proposition 6.69 (Discretization and measurement components must aggregate consistently). *When discretized or measured evidence is used, all metric-admissible components must be aggregated according to the declared ledger policy.*

Proof. Otherwise the displayed total no longer represents the declared upper bound on the combined discrepancy. $\square$

Definition 6.70 (Controlled-commutation and Mirror/Transfer components). Appendix K controlled-commutation costs enter through $\mathfrak{d}_{\text{comm}}$. Appendix L Mirror/Transfer costs enter through $\mathfrak{d}_{\text{MT}}$. Both become metric-ledger subcomponents only after their defects are certified as bottleneck-relevant discrepancy bounds on the monitored scope.

Proposition 6.71 (K/L costs must aggregate consistently). *Controlled-commutation and Mirror/Transfer costs are audit-admissible only if they are typed, converted when needed, and aggregated under the declared ledger policy.*

Proof. K/L costs affect the audited pipeline by contributing to the total discrepancy. Without typing, conversion, and aggregation, their budget effect is not reconstructible. $\square$

Definition 6.72 (Source-data ledger). A source-data ledger records import, provenance, transcription, sampling-source, or external-data discrepancies through the component $\mathbf{d}_{\text{src}}$. It does not assert truth of the source data; it only records a declared and supported discrepancy contribution when source uncertainty is part of the audited scope.

Definition 6.73 (Serialization and schema ledger). A serialization and schema ledger records parsing, serialization, execution-schema, format-conversion, or replay-environment discrepancies through the component $\mathbf{d}_{\text{ser}}$. A serialization component may support audit readability, but it does not replace the mathematical artifact, schema conformance, or verifier-side record being serialized.

M.10bis. Integrated downstream defect catalog and local alias closure

Definition 6.74 (Defect registration record). A *defect registration record* is a ledger-governance record for a displayed defect symbol. It must declare:

1. the symbol and source appendix;
2. the primitive discrepancy or governance failure it denotes;
3. whether the component is metric-admissible, status-ledger, qualitative-governance, provenance, or purely advisory;
4. its target canonical slot in Definition 6.17, if it is consumed by the strict Core metric budget;
5. its support record or conversion witness;
6. its relation to neighboring symbols: alias, refinement, aggregate, disjoint contribution, or blocked overlap;
7. its failure route when missing, ill-typed, stale, or unsupported.

A displayed defect symbol is therefore not automatically a metric component merely because it is written with a δ or Δ .

Definition 6.75 (Defect relation declaration). A *defect relation declaration* records how two defect symbols are related. The admissible relation types are:

1. *alias*: two names denote the same primitive discrepancy under the declared scope;
2. *refinement*: one symbol decomposes a coarser component into finer supported components;
3. *aggregate*: one symbol is the declared aggregation of listed primitive components;
4. *disjoint*: two symbols denote independent or non-overlapping primitive discrepancies;
5. *blocked overlap*: possible overlap is detected but not resolved, so joint consumption is not audit-admissible.

The relation must be declared before budget evaluation. It is not inferred from visual similarity of names.

Definition 6.76 (Local ledger alias). A *local ledger alias* is an appendix-local aggregate notation such as

$$\mathbf{d}_{\text{search}}, \quad \mathbf{d}_{\text{bridge}}, \quad \mathbf{d}_{\text{asm}}.$$

Such an alias is admissible only at a declared gate scope and only after its primitive components, canonical slot placement, scalarization, and relation to $\mathbf{d}_{\text{met}}$ have been declared. A local alias is not a new global Core invariant.

Definition 6.77 (Status-governance defect). A *status-governance defect* is a qualitative or governance failure involving identity, artifact adequacy, synchronization, environment compatibility, replay, status, claim-use permission, or schema conformance. Such a defect may block consumption, route the affected record to undefined or reject, or require quarantine. It enters the strict metric budget only after a separate metric-admissible conversion proves an actual bottleneck discrepancy bound.

Definition 6.78 (Part IV problem-interface defect catalog). The following problem-interface and technical-extension defect names are locally registered for v19.0.0 audit governance. They remain subject to Definition 6.74 before strict Core consumption.

Arithmetic and Iwasawa-family defects:

$$\delta^{\text{arith}}, \delta^{\text{arith-comp}}, \delta^{\text{arith-trunc}}, \delta^{\text{arith-extract}}, \delta^{\text{cyc}}, \delta^{\text{Iw}}, \delta^{pL}, \\ \delta^{\text{ctrl}}, \delta^{\text{cong}}, \delta^{\text{trunc}}.$$

Reflection, Mirror, categorical, and Fukaya-family defects:

$$\delta^{\text{Mir}}, \delta^{\text{cat}}, \delta^{\text{ref-extract}}, \Delta_{\text{Eval,ref}}, \Delta_{T,\text{ref}}, \Delta_{\phi,\text{ref}}, \\ \Delta_{\text{Eval,MT}}, \Delta_{T,\text{MT}}, \Delta_{\phi,\text{MT}}, \delta^{\text{Fuk-ext}}, \delta^{\text{Fuk-norm}}, \delta^{\text{Fuk-cont}}, \\ \delta^{\text{Fuk-pert}}, \delta^{\text{Fuk-model}}, \delta^{\text{Fuk-Mir}}, \Delta_{\phi,\text{Fuk}}.$$

Normalization, PDE, and Navier–Stokes-family defects:

$$\delta^{\text{nor}}, \delta^{\text{res}}, \delta^{\text{scale}}, \delta^{\text{filt-align}}, \delta^{\text{nor-ext}}, \delta^{\text{rev}}, \\ \delta^{\text{chart}}, \delta^{\text{smooth}}, \delta^{\text{nor-trunc}}, \Delta_{\phi,\text{nor}}, \delta^{\text{PDE-scale}}, \\ \delta^{\text{NS-disc}}, \delta^{\text{NS-meas}}, \delta^{\text{NS-alg}}, \delta^{\text{NS-real}}, \delta^{\text{NS-adv}}, \\ \delta^{\text{obs}}, \delta^{\text{NS-param}}.$$

If two displayed names above may refer to the same primitive discrepancy, the consuming record must declare an alias, refinement, aggregate, or disjointness relation. Otherwise the joint budget use is blocked.

Definition 6.79 (Part V search/platform defect catalog). The following Search / Platform defect names and aliases are locally registered for v19.0.0 audit governance.

Hunter / Mapper / Lifter and re-entry defects:

$$\delta^{\text{hunt}}, \delta^{\text{map}}, \delta^{\text{lift}}, \delta^{\text{score}}, \delta^{\text{reentry}}, \Delta_{\phi,\text{search}}, \mathfrak{d}_{\text{search}}.$$

Bridge-program defects:

$$\delta^{\text{bridge}}, \delta^{\text{bridge-real}}, \delta^{\text{bridge-interp}}, \delta^{\text{bridge-formal}}, \\ \delta^{\text{bridge-disc}}, \delta^{\text{bridge-transport}}, \Delta_{\phi,\text{bridge}}, \mathfrak{d}_{\text{bridge}}.$$

Validity-map, proof-store, and execution defects:

$$\delta^{X\text{-edge}}, \delta^{X\text{-asm}}, \delta^{X\text{-art}}, \delta^{X\text{-interp}}, \Delta_{\phi,\text{DAG}}, \mathfrak{d}_{\text{asm}}, \\ \delta^{Y\text{-sync}}, \delta^{Y\text{-id}}, \delta^{Y\text{-adequacy}}, \delta^{Z\text{-adequacy}}, \delta^{Z\text{-env}}, \delta^{Z\text{-replay}}.$$

The aliases $\mathfrak{d}_{\text{search}}$, $\mathfrak{d}_{\text{bridge}}$, and $\mathfrak{d}_{\text{asm}}$ are gate-scoped aggregates only. They must be mapped to $\mathfrak{d}_{\text{met}}$, to a product/status ledger coordinate, or to a blocked non-consumed status before verifier-side use.

Proposition 6.80 (Unregistered downstream defect blocks strict budget consumption). *A downstream defect symbol from the problem-interface, search/platform, proof-store, or execution layer cannot be consumed by the strict Core metric budget unless a defect registration record supplies its type, support record, canonical placement, scalarization when needed, and relation to neighboring components.*

Proof. The strict budget compares $\text{val}_B(\mathbf{d}_{\text{met}})$ with the protected threshold gap. An unregistered symbol has no declared value system, no certified scalarization, and no declared relation to other components. Its consumption would therefore be neither typed nor reconstructible. $\square$

Proposition 6.81 (No cross-layer double counting). *The same primitive discrepancy may not be counted twice under two displayed downstream names.*

Proof. If two names are aliases, counting both inflates the aggregate. If one refines the other, consuming both counts the coarse and fine versions together. If they are disjoint, this must be declared. Without such a relation, the aggregate does not represent the declared primitive discrepancy family. $\square$

Proposition 6.82 (Status-governance defects are not metric slack). *Identity, adequacy, synchronization, environment, replay, status, and claim-use defects do not become scalar metric slack merely by being written as δ -symbols.*

Proof. These defects concern whether a record may be consumed at all, not how far two pre-threshold barcode profiles are in bottleneck distance. They may route a dependency to undefined, reject, quarantine, or repair, but they enter the metric budget only through an additional metric-admissible conversion witness. $\square$

Proposition 6.83 (Local aliases are gate-scoped). *The aliases $\mathbf{d}_{\text{search}}$, $\mathbf{d}_{\text{bridge}}$, $\mathbf{d}_{\text{asm}}$, and any analogous downstream aggregate are admissible only within the gate scope, protected profile family, component decomposition, and scalarization declared in the consuming certificate.*

Proof. An aggregate notation does not determine its scope or protected profiles. Since the strict comparison is scope-bound by Proposition 6.10, every alias must inherit the declared scope rather than becoming a global invariant. $\square$

Theorem 6.84 (Integrated downstream defect-catalog closure). *The defect symbols and local aliases introduced by the repaired Problem Interface and Search / Platform appendices are admissible in v19.0.0 only under the following closure discipline:*

1. *each symbol has a defect registration record;*
2. *every metric-admissible component maps into the gate-scoped $\mathbf{d}_{\text{met}}$ through a certified support record and scalarization;*
3. *every status-governance component remains outside the metric budget unless converted by theorem or verified computation into an actual bottleneck bound;*
4. *all alias, refinement, aggregation, and disjointness relations are declared before budget evaluation;*
5. *no local alias becomes a global Core invariant;*
6. *missing relation data yield undefined or a blocked consumption route, not pass.*

Proof. The first condition is Definition 6.74. Metric consumption is controlled by Definitions 6.8 and 6.15. Status-governance components are separated by Definition 6.77 and Proposition 6.82. Alias and no-double-counting discipline are supplied by Definition 6.75 and Proposition 6.81. Local scope is Proposition 6.83. Missing relation data cannot produce an auditable strict comparison, so the weaker safe result is undefined or blocked consumption. $\square$

Remark 6.85 (Metric versus status-ledger reading). The identity, adequacy, synchronization, environment, replay, permission, and status defects appearing in Appendices X–Z are typically status-ledger or qualitative-governance components. Their normal function is to block consumption, trigger repair, or route to undefined or reject. They are not metric-admissible unless an additional record proves a profile-level bottleneck discrepancy bound.

Remark 6.86 (Part IV registration boundary). Problem-interface defect names, including arithmetic, Mirror/categorical, Fukaya, normalization, and Navier–Stokes names, are registered here as namespace entries and governance hooks. This registration does not prove any arithmetic, PDE, geometric, categorical, mirror-symmetric, or Fukaya theorem, and it does not prove that any individual defect bound is small enough for a Core pass.

Remark 6.87 (Part V registration boundary). Search/platform defect names, including search, bridge, DAG, proof-store, and execution names, are registered here as workflow and audit-governance entries. This registration does not convert search, storage, execution, replay, or reproducibility into proof.

M.11. Ledger manifests and machine readability

Definition 6.88 (Ledger manifest entry). A ledger manifest entry records the value system, aggregation law, strict comparison convention, scalarization or coordinate gap rule, components, component support records, dependency graph, total budget, safety margin, pass/fail or undefined budget result, artifact references, and any conversion or refinement used.

Proposition 6.89 (Ledger manifest is required for audit readability). *A certificate relying on a nontrivial ledger is not audit-readable unless its ledger manifest entry is present and reconstructible.*

Proof. The verifier needs the manifest to reconstruct $\mathbf{d}_{\text{met}}$, scalarize it when appropriate, and compare it to the gap. $\square$

Definition 6.90 (Ledger verdict states). A ledger budget check has one of the following local states: pass, reject, undefined, or not_invoked. Missing ledger semantics, missing components, missing scalarization, or missing gap data yield undefined, not pass.

Proposition 6.91 (Failed budget is reject for the affected audit path). *If every required ledger input is defined and the strict budget comparison fails, then the affected audited path is rejected rather than undefined.*

Proof. Undefined records arise from missing or ill-typed obligations. Once the comparison is well-defined and false, the relevant budget obligation fails. $\square$

Remark 6.92 (Relation to H–L). Appendices H–L produce advisory conversions, discretization costs, measurement restrictions, controlled-commutation costs, and Mirror/Transfer costs. Appendix M is the common ledger semantics layer that determines how such components are admitted and aggregated.

M.12. Appendix M package theorem

Theorem 6.93 (Appendix M quantale-ledger package). *Within the Technical Extension layer, Appendix M provides the following package.*

1. *Ledger semantics are declared locally and do not enlarge the Core.*
2. *Core metric budget consumption requires a typed metric ledger and monotone, normalized, subadditive bottleneck scalarization.*

3. *Additive, max, weighted, product, lexicographic, qualitative, and hybrid ledgers are admissible only when explicitly declared.*
4. *Components must be typed, supported by artifacts, aligned with the Main canonical decomposition, and aggregated without double counting or silent omission.*
5. *Product and qualitative ledgers do not become scalar metric budgets without admissible conversion.*
6. *Ledger conversion and refinement must be monotone, conservative, declared, and non-post-hoc.*
7. *Repaired-evaluation, representative, eligibility, and tower-artifact discrepancies can be budgeted but do not replace the corresponding verifier-side records.*
8. *Window, overlap, restart, and infinite-family budget claims are ledger-relative.*
9. *Advisory, discretization, measurement, K/L commutation, and Mirror/Transfer quantities affect the Core budget only after certified metric-ledger admission.*
10. *Problem-interface and Search/Platform defect names are consumable only through defect registration records declaring metric/status type, support, canonical placement, and alias/refinement/aggregation/disjointness relations.*
11. *Status-governance defects may block or route certificate use, but they do not become scalar metric slack without certified metric conversion.*
12. *A ledger manifest is required for audit readability.*
13. *Budget pass does not imply B-Gate⁺ pass, and structural B-Gate⁺ pass does not imply Audit Mode pass without the ledger.*

Proof. The items are supplied respectively by the sovereignty principles, the Core metric-ledger contract, the catalog definitions, the component support and dependency-aware aggregation rules, the conversion and refinement rules, the compatibility sections M.7–M.10, the manifest requirements, and the non-replacement propositions. □

M.13. Explicit exclusions

The following are not asserted in Appendix M.

- No single ledger semantics is imposed globally.
- No hidden default aggregation law is assumed.
- No ordinary addition is assumed unless declared.
- No weak scalarization lacking monotonicity, normalization, or subadditivity is accepted for the Core metric budget.
- No product, lexicographic, qualitative, or hybrid ledger is collapsed to a scalar budget without an admissible conversion.
- No weight policy is accepted without declaration.
- No advisory quantity affects the budget unless converted.
- No scalar measurement error is a metric-ledger component without a certified measurement-to-metric conversion.

- No crop, continuation, or comparison-mode component is omitted when it is invoked by the monitored scope.
- No controlled-commutation or Mirror/Transfer defect is ignored when used in certification.
- No nonmonotone ledger transformation is accepted by default.
- No post-hoc conversion, weighting, or refinement may turn failure into pass.
- No representative ledger replaces admissible-representative compatibility.
- No eligibility ledger replaces eligibility.
- No tower-artifact ledger replaces $\phi_{i,W,\tau}$, its defect profiles, or Type IV diagnostics.
- No budget comparison replaces B-Gate⁺.
- No budget comparison replaces $\text{Eval}_{1,W,\tau}(F) = 0$.
- No budget comparison replaces $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$.
- No ledger catalog entry proves a problem-specific arithmetic, PDE, geometric, categorical, mirror-symmetric, or transfer-theoretic theorem.
- No downstream defect symbol enters the strict Core metric budget without a defect registration record.
- No identity, artifact adequacy, synchronization, environment, replay, status, or claim-use defect is repaired by numerical metric slack.
- No local alias such as $\mathfrak{d}_{\text{search}}$, $\mathfrak{d}_{\text{bridge}}$, or $\mathfrak{d}_{\text{asm}}$ is a global Core invariant.
- No primitive discrepancy may be counted twice under different displayed defect names.

M.14. Provenance and status

The material in this appendix depends on the metric-repair contract of Chapter 1, the overlap/restart and audit disciplines of Chapters 6 and 7, the manifest schema of Appendix G, the repaired evaluation and gap discipline of Appendix A, the representative and eligibility disciplines of Appendices B and C, the tower-artifact and Type IV discipline of Appendix D, the window/restart/overlap discipline of Appendix E, the Technical Extension ledgers of Appendices H–L, the repaired Problem Interface defect namespaces, and the repaired Search / Platform defect namespaces of Appendices U–Z. Its role is to provide budget semantics without allowing any ledger choice, defect name, status component, or platform alias to bypass Core verifier-side obligations.

Status labels for Appendix M. *Technical extension principles:* Principles 6.1, 6.2.

Technical extension definitions: Definitions 6.5, 6.6, 6.8, 6.9, 6.13, 6.14, 6.15, 6.16, 6.17, 6.18, 6.19, 6.20, 6.23, 6.24, 6.25, 6.26, 6.30, 6.32, 6.34, 6.36, 6.39, 6.40, 6.42, 6.44, 6.46, 6.51, 6.53, 6.55, 6.58, 6.59, 6.61, 6.63, 6.64, 6.66, 6.68, 6.70, 6.72, 6.73, 6.74, 6.75, 6.76, 6.77, 6.78, 6.79, 6.88, 6.90.

Technical extension propositions/theorems: Propositions and Theorems 6.10, 6.11, 6.12, 6.21, 6.22, 6.27, 6.28, 6.31, 6.33, 6.37, 6.41, 6.43, 6.45, 6.47, 6.48, 6.49, 6.52, 6.54, 6.56, 6.60, 6.62, 6.65, 6.67, 6.69, 6.71, 6.80, 6.81, 6.82, 6.83, 6.84, 6.89, 6.91, 6.93.

Operational cautions: Remarks 6.3, 6.4, 6.7, 6.29, 6.35, 6.38, 6.50, 6.57, 6.92, 6.85, 6.86, 6.87.

Explicit exclusions: Section M.13.

7 Appendix N: Derived, Sheaf, Stack, and Higher-Categorical Extension Discipline

N.1. Scope and non-Core status

Definition 7.1 (Higher-enhancement sovereignty principle). A derived, sheaf-theoretic, stack-theoretic, or higher-categorical enhancement is external infrastructure around the AK Core. It may organize objects, comparisons, descent data, representatives, or proof traces, but it does not create or modify any Core verifier-side obligation. In particular, it does not replace B-Gate⁺, the repaired topological clause

$$\text{Eval}_{1,W,\tau}(F) = 0,$$

canonical Type IV cleanliness

$$(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0),$$

the tower comparison artifact $\phi_{i,W,\tau}$, admissible representatives, invoked categorical eligibility, artifact declarations, or the strict metric-ledger budget.

Definition 7.2 (No higher-to-Core upgrade principle). No result, object, equivalence, descent datum, sheaf gluing theorem, derived edge map, stable ∞ -categorical enhancement, or higher coherence cell becomes a Core theorem merely by appearing in a Technical Extension appendix. Core import requires a separately declared statement, Core-compatible hypotheses, a proof under the repaired guard-rails, and updated status labels.

Remark 7.3 (Technical extension status). Appendix N closes the Technical Extension layer. Its purpose is to permit sophisticated external mathematical organization while preventing silent enlargement of Chapters 1–8. The default status of the material below is Technical Extension or [Spec], not Core.

Remark 7.4 (Core evaluation remains fixed). All Core-relevant persistence data remain governed by the repaired window-first, threshold-after-window evaluation

$$\text{Eval}_{i,W,\tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

No derived, sheaf, stack, or higher-categorical route changes this order unless it supplies a separate controlled comparison whose defect is typed, converted, and ledgered.

Remark 7.5 (Canonical Type IV notation). Throughout Appendix N the cokernel-side Type IV diagnostic is denoted by ν_{Collapse} . No alternative Latin-letter cokernel diagnostic is introduced.

N.2. External enhancements and verifier-side re-entry

Definition 7.6 (External enhancement). An external enhancement is a declared derived, sheaf-theoretic, stack-theoretic, dg-categorical, stable ∞ -categorical, enriched, indexed, descent, or higher-categorical structure placed around the AK Core to organize, compare, refine, or interpret Core-relevant data.

Definition 7.7 (Enhancement interface). An enhancement interface for an external enhancement $\mathcal{E}$ consists of: its ambient structure; source and target classes; variance; realization target; extraction route to Core-readable data; comparison cells or functors used; ledger treatment of nonzero defects; artifact references; and status labels for any problem-specific implications.

Definition 7.8 (Verifier-side re-entry). An enhancement re-enters verifier-side Core data only through one of the following declared outputs: a constructible one-parameter persistence object over the ground field; a filtered object from which such a persistence object is computed; repaired evaluation data $\text{Eval}_{i,w,\tau}$; an admissible representative $C_{\tau,w}F$; an eligible realized object for an invoked Ext^1 -clause; a tower comparison artifact $\phi_{i,w,\tau}$; a Type IV diagnostic record; a metric-ledger component with certified scalarization; or a gate verdict record.

Proposition 7.9 (Enhancement data do not update Core verdicts). *External enhancement data do not update a Core verdict unless the relevant data are explicitly realized as verifier-side Core data through Definition 7.8.*

Proof. Audit Mode reads repaired evaluations, representative and eligibility records when invoked, Type IV diagnostics, tower artifacts, ledgers, gate verdicts, and artifact references. An external enhancement by itself is none of these typed records. Therefore it has no Core verdict effect until it is re-entered through a declared verifier-side route. $\square$

Proposition 7.10 (Enhancement elegance is not proof). *Functoriality, naturality, descent, equivalence, or conceptual elegance of an enhancement is not a proof of a Core claim unless the claimed implication is proved under the Core guard-rails.*

Proof. The Core is governed by explicit verifier-side obligations. Conceptual organization may support a proof, but it does not itself discharge any obligation unless the proof supplies the required Core-readable data. $\square$

N.3. Derived and dg extensions

Definition 7.11 (Derived extension). A derived extension interprets AK-relevant objects in a declared derived category, dg-category, triangulated category, stable homotopical category, or stable ∞ -category.

Definition 7.12 (Derived realization target). A derived realization target is a declared functor, correspondence, span, or partial construction

$$\mathcal{R}_D : C_{\text{ext}} \dashrightarrow D$$

into a specified derived or stable target D , together with its domain of definition and extraction route back to Core-readable data.

Definition 7.13 (Core-compatible derived realization). A derived realization is Core-compatible only if it supplies the following data: a route to a constructible one-parameter persistence or filtered object; reconstructible repaired profiles; representative compatibility when representatives are used; and, when the Ext^1 -clause is invoked, an eligible realized object in the amplitude $[-1, 0]$ regime with the required edge-realization condition.

Proposition 7.14 (Derived enhancement does not replace eligibility). *A derived enhancement does not replace the eligible realization regime required for the Core Ext^1 -edge.*

Proof. The repaired edge bridge is a one-way Core statement in the declared amplitude $[-1, 0]$ setting over the ground field, with edge realization to the repaired residual degree-one persistence data. A general derived or dg target need not satisfy those hypotheses. Eligibility therefore remains a separate obligation. $\square$

Remark 7.15 (Beyond the field amplitude regime). Derived categories over rings, schemes, stacks, sheaves, motives, or dg-enhanced objects may be useful in interface appendices. They do not inherit the Core Ext^1 -edge theorem without new hypotheses and proof.

N.4. Derived Ext-edge extensions

Definition 7.16 (Derived Ext-edge proposal). A derived Ext-edge proposal is a proposed generalization of the Core edge identification

$$\text{Ext}^1(A, k) \cong \text{Hom}_k(H^{-1}(A), k)$$

to a different derived target, coefficient object, or amplitude regime.

Definition 7.17 (Admissible derived edge extension). A derived edge extension is admissible only if it declares: the ambient category; the coefficient object; the amplitude and truncation conditions; the splitting, injectivity, projectivity, or spectral-sequence degeneration hypotheses used; the exact edge map; the edge-realization link to repaired residual persistence data; and proof or artifact references supporting all hypotheses.

Proposition 7.18 (No automatic higher-degree bridge). *No higher-degree PH^n -to- Ext^n bridge is asserted in Appendix N.*

Proof. The Core uses a one-way degree-one bridge under a specific eligible realization regime. Higher-degree extensions require additional amplitude, spectral sequence, and edge-realization data. Those data are not supplied merely by adding a derived enhancement. $\square$

Proposition 7.19 (No automatic spectral-sequence collapse). *The existence of a spectral sequence or derived edge map does not by itself yield the edge isomorphism needed for a Core-style Ext-edge conclusion.*

Proof. A spectral sequence may have higher-page contributions, extension problems, or non-degenerate differentials. Without declared degeneration and vanishing hypotheses, an edge map need not be an isomorphism and need not realize the repaired persistence edge. $\square$

Remark 7.20 (Priority successor status). Higher-degree Ext-edge bridges may be studied as successor candidates, but Appendix N records only the governance conditions under which such candidates could later be imported. It does not promote them.

N.5. Sheaf and cosheaf extensions

Definition 7.21 (Sheaf-theoretic extension). A sheaf-theoretic extension represents AK-relevant data using sheaves, cosheaves, constructible sheaves, persistence sheaves, sheaf complexes, or sheaf-like objects on a declared space, site, or topos.

Definition 7.22 (Sheaf realization). A sheaf realization is a declared construction

$$\mathfrak{S} : C_{\text{ext}} \dashrightarrow \text{Sh}(X) \quad \text{or} \quad \mathfrak{S} : C_{\text{ext}} \dashrightarrow D^b(\text{Sh}(X)),$$

where X is declared, together with the functoriality and domain restrictions of $\mathfrak{S}$.

Definition 7.23 (Core-readable sheaf extraction). A sheaf realization is Core-readable only if it includes an extraction route to a constructible one-parameter persistence object or a filtered object whose repaired evaluation

$$T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(-))$$

is defined in the Core regime.

Proposition 7.24 (Sheaf realization does not replace persistence extraction). *A sheaf or cosheaf object is not Core-relevant merely by being present. It becomes Core-relevant only after a declared extraction route produces Core-readable persistence or filtered data.*

Proof. Core gates are stated in terms of repaired persistence profiles, representatives, eligibility records, tower artifacts, diagnostics, and ledgers. A raw sheaf object has a different type. A typed extraction is needed before the verifier can read it. $\square$

Remark 7.25 (Constructibility requirement). If sheaf data are used to produce persistence objects, the extracted persistence object must satisfy the constructibility, finiteness, windowing, and threshold conventions of the repaired Core.

N.6. Sheaf gluing, descent, and certificate gluing

Definition 7.26 (Sheaf gluing claim). A sheaf gluing claim asserts that local sheaf-theoretic objects, sections, complexes, or descent data glue to a global sheaf-theoretic object.

Definition 7.27 (Certificate gluing claim). A certificate gluing claim asserts that local certificate records assemble into a globally coherent certificate record with compatible windows, overlaps, thresholds, repaired profiles, ledgers, diagnostics, artifacts, and verdicts.

Proposition 7.28 (Sheaf gluing and certificate gluing are distinct). *A sheaf gluing theorem does not automatically imply a Core certificate gluing theorem, and a Core certificate gluing theorem does not automatically imply a sheaf gluing theorem.*

Proof. The two claims live in different typed regimes. Sheaf gluing concerns descent of sheaf-theoretic objects. Certificate gluing concerns verifier-side audit records. A bridge between them requires additional extraction, overlap, and ledger hypotheses. $\square$

Definition 7.29 (Audit-compatible descent). A descent datum is audit-compatible only if, beyond the descent conditions of its ambient stack or sheaf theory, it records all Core-relevant certificate data: windows, thresholds, repaired evaluations, representative and eligibility records when invoked, ledgers, diagnostics, tower artifacts, overlap budgets, gate verdicts, and artifact references.

Proposition 7.30 (Descent is not audit by default). *A stack-theoretic or sheaf-theoretic descent datum is not automatically an audit-compatible certificate record.*

Proof. Descent data may establish object-level gluing. Audit compatibility additionally requires the verifier-side data listed in Definition 7.29. These data are not included by default in descent theory. $\square$

N.7. Stack and higher-stack extensions

Definition 7.31 (Stack-theoretic extension). A stack-theoretic extension organizes AK-relevant objects, filtrations, certificates, local models, comparison data, or descent records in a stack, prestack, higher stack, or fibered category over a declared site.

Definition 7.32 (Stack realization target). A stack realization target declares the stack or higher stack, its site, its local charts or covers, its descent condition, and the extraction route by which stack-level data produce Core-readable verifier-side records.

Proposition 7.33 (Stack descent does not replace overlap gate). *Stack descent does not replace the overlap compatibility checks required for certificate assembly.*

Proof. Overlap gates compare repaired profiles, budgets, artifacts, and verdicts on overlaps. Stack descent compares local objects and transition data in the ambient stack. The latter may support the former only after the relevant verifier-side overlap data are extracted and checked. $\square$

Remark 7.34 (No automatic hyperdescent upgrade). Hyperdescent or higher descent may be mathematically strong, but it is not a Core audit theorem unless it is connected to the certificate-record gluing protocol by an explicit interface theorem.

N.8. Higher-categorical comparison cells

Definition 7.35 (Higher-categorical extension). A higher-categorical extension uses bicategories, 2-categories, double categories, enriched categories, derivators, $(\infty, 1)$ -categories, (∞, n) -categories, higher toposes, or coherent higher diagrams to organize AK-relevant structures.

Definition 7.36 (Higher comparison cell). A higher comparison cell is a 2-cell, higher cell, homotopy, equivalence, modification, coherent diagram, or higher morphism comparing two or more pipeline composites.

Definition 7.37 (Higher-cell defect). A higher-cell defect is a typed ledger value assigned to a higher comparison cell. It must declare its orientation, source composite, target composite, equivalence convention, cost semantics, and scalarization route if it is to affect the Core metric budget.

Proposition 7.38 (Higher cells affect audit only through certified ledger entry). *A nonzero higher-cell defect affects Core audit only through a declared ledger component that is compatible with the metric-ledger semantics of Appendix M.*

Proof. Higher cells are comparison data, not Core gate verdicts. If a certificate uses a higher cell to replace or compare pipeline routes, the induced discrepancy must be typed and aggregated into the declared ledger before it can affect Audit Mode. $\square$

Proposition 7.39 (Zero-cost higher equivalence is still typed). *A zero-cost higher equivalence may be treated as exact only for the declared purpose, orientation, and equivalence convention recorded in the certificate.*

Proof. Zero cost is a statement in a declared defect semantics. Without specifying what the cost measures and which composites it compares, the phrase does not determine a verifier-readable equality. $\square$

N.9. Higher functorial representatives and coherence

Definition 7.40 (Higher functorial representative system). A higher functorial representative system is a system of representatives $C_{\tau, W}F$ whose choices are functorial only up to declared coherent higher equivalences.

Definition 7.41 (Coherence defect). A coherence defect is a ledger-valued cost measuring failure of strict functoriality, associativity, unit compatibility, representative compatibility, naturality, or higher coherence.

Proposition 7.42 (Higher functoriality does not strengthen representative status). *A higher functorial representative system has the same Core status as ordinary admissible representative data unless its coherence data are imported and proved as Core-compatible.*

Proof. Representatives are consumed by the Core through compatibility with repaired profiles. Higher functoriality may organize choices, but it does not change the requirement that $\mathbf{P}_i(C_{\tau, W}F)$ agree with the repaired evaluation in the monitored degrees. Nonzero coherence defects must be ledgered if used. $\square$

Remark 7.43 (Coherence is not admissibility). Coherent representative choice may be valuable for implementation or categorical presentation. It does not convert a non-admissible representative into an admissible one.

N.10. Higher Type IV interpretations

Definition 7.44 (Higher Type IV interpretation). A higher Type IV interpretation is an external derived, sheaf, stack, or higher-categorical reading of monitored generic Type IV behavior.

Definition 7.45 (Higher Type IV proxy). A higher Type IV proxy is a higher-categorical, sheaf-theoretic, derived, or stack-theoretic signal that may correlate with possible tower comparison defect.

Definition 7.46 (Actual Type IV diagnostic in higher context). The actual monitored Type IV diagnostic in a higher context is still computed from the declared tower comparison artifact

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty)$$

by taking the window-relative terminal-generic dimensions tgdim_W of its kernel and cokernel defect profiles.

Proposition 7.47 (Higher Type IV proxy is not diagnosis). *A higher Type IV proxy does not determine whether*

$$(\mu_{\text{Collapse}}, \nu_{\text{Collapse}}) = (0, 0)$$

or not.

Proof. The diagnostic is defined by kernel and cokernel defect profiles of $\phi_{i,W,\tau}$. A proxy may guide where to compute those profiles, but it is not the computation unless a theorem identifies it with the same diagnostic data. $\square$

N.11. Local comparison modes and Core projection

Definition 7.48 (Higher-extension comparison mode). A higher-extension comparison has one of the following local modes: *strict*; *zero-cost equivalent*; *metric-controlled*; *higher-cell-controlled*; *advisory-only*; *unavailable*; or *failed*. The mode must be recorded for each comparison that supports a certificate.

Definition 7.49 (Core projection of higher-extension modes). Higher-extension modes project to Core comparison modes as follows. Strict and zero-cost equivalent comparisons project to exact for the declared purpose. Metric-controlled comparisons project to `metric_gap_certified` after a certified metric discrepancy bound and scalarization route into the declared Core metric ledger are supplied. The later strict gap comparison determines the resulting atomic verdict; a failed gap comparison yields reject rather than changing the comparison mode. Higher-cell-controlled comparisons project to exact only when zero-cost coherent equality is declared, to `metric_gap_certified` only after certified metric conversion is supplied, and otherwise to `not_compared`. Advisory-only and unavailable comparisons project to `not_compared`; if an invoked obligation depends on them, its atomic status is undefined. Failed comparisons force the affected path to reject.

Proposition 7.50 (Advisory higher comparison is non-verdict). *An advisory-only higher comparison cannot support Core acceptance.*

Proof. Advisory-only means that no exact equality and no certified metric-gap comparison has been supplied. The Core verifier cannot consume such a comparison as evidence for an invoked obligation. $\square$

N.12. Enhancement artifacts and manifest requirements

Definition 7.51 (Enhancement artifact). An enhancement artifact is a record supporting a derived, sheaf, stack, or higher-categorical extension. Examples include ambient category data, dg-models, sheaf or cosheaf data, descent records, stack charts, extraction functors, comparison cells, coherence diagrams, repaired evaluation records produced from the enhancement, tower artifacts, and ledger entries for higher defects.

Definition 7.52 (Enhancement manifest entry). An enhancement manifest entry records the enhancement type; ambient structure; realization target; extraction route; produced repaired profiles; representative and eligibility records when invoked; tower artifacts when Type IV is invoked; comparison cells; ledger costs; local comparison modes; problem-specific status; and artifact references.

Proposition 7.53 (Enhancement artifacts are required for audit readability). *A certificate relying on an enhancement is not audit-readable unless it includes enhancement artifact references sufficient to reconstruct the enhancement interface and its Core-readable outputs.*

Proof. Without artifact references, an auditor cannot reconstruct the ambient structure, extraction route, comparison cells, repaired evaluations, tower artifacts, or ledger entries. The manifest is therefore incomplete. $\square$

Remark 7.54 (Enhancement manifest is additional). An enhancement manifest entry is additional to the Minimal Manifest. It does not replace windows, thresholds, ledgers, diagnostics, gate verdicts, or artifact references required by the Core.

N.13. Ledger integration for higher extensions

Definition 7.55 (Higher-extension ledger namespace). The symbols

$$\mathfrak{d}_{\text{der}}, \quad \mathfrak{d}_{\text{sh}}, \quad \mathfrak{d}_{\text{desc}}, \quad \mathfrak{d}_{\text{stack}}, \quad \mathfrak{d}_{\text{higher}}$$

are readable aliases for higher-extension contributions. They are not independent Core ledgers. To affect Audit Mode they must be mapped into the declared gate-scoped metric ledger $\mathfrak{d}_{\text{met}}$ through a currently declared component namespace and support record. In the Appendix M review-closure namespace, typical target slots include comparison, crop/window-edge, continuation, realization-metric, representative-metric, source-data, serialization/schema, or algorithmic components; if that namespace is not available in a given assembly, the certificate must declare an equivalent mapping before consuming the higher-extension contribution.

Definition 7.56 (Higher-extension adjusted metric budget). If a certificate uses nonzero higher-extension defects, the adjusted metric ledger is

$$\mathfrak{d}_{\text{adj}} := \mathfrak{d}_{\text{base}} \oplus \mathfrak{d}_{\text{higher,conv}},$$

where $\mathfrak{d}_{\text{higher,conv}}$ is the certified conversion of all used higher-extension defects into the declared metric-ledger value system. The audited inequality is

$$\text{val}_B(\mathfrak{d}_{\text{adj}}) < \text{gap}_\tau(\mathfrak{B})$$

for the declared protected barcode family $\mathfrak{B}$.

Proposition 7.57 (Higher-extension evidence must pass adjusted budget). *A certificate record relying on nonzero higher-extension defects may pass Audit Mode only if the adjusted metric budget of Definition 7.56 satisfies the strict gap comparison.*

Proof. All nonzero defects used by a certificate must enter the metric ledger through an admissible conversion. Audit Mode then checks the strict scalarized budget against the declared gap. $\square$

Remark 7.58 (No budget-free higher coherence). Higher coherence data may be elegant, but nonzero coherence defects are not budget-free. If a certified pipeline depends on them, they must be typed, converted when necessary, and ledgered.

N.14. Problem-specific implications and Core import

Definition 7.59 (Problem-specific higher implication). A problem-specific higher implication is a statement of the form

$$\text{derived/sheaf/stack/higher structure} \implies \text{problem-specific mathematical conclusion.}$$

Proposition 7.60 (Problem-specific higher implications are [Spec] by default). *Every problem-specific higher implication is [Spec] by default unless separately promoted and reproved under the Core guard-rails and the relevant interface protocol.*

Proof. Such implications import external semantics. Chapter 8 requires problem-specific conclusions to remain interface-level unless explicitly internalized and proved under the relevant hypotheses. $\square$

Definition 7.61 (Admissible Core import). An admissible Core import of a technical extension result requires: a precise statement; Core-compatible hypotheses; a proof under the repaired guard-rails or a declared conservative extension; all realization, evaluation, representative, eligibility, diagnostic, artifact, and ledger data needed by the statement; consistency with the Core non-claim register; and updated status labels.

Proposition 7.62 (No silent Core import). *No derived, sheaf, stack, or higher-categorical result becomes part of the Core merely by appearing in Appendix N.*

Proof. Appendix N is Technical Extension material. By the no-upgrade principle, Core membership requires admissible Core import, not mere placement in an appendix. $\square$

N.15. Part III closure

Theorem 7.63 (Part III closure principle). *The Technical Extension Appendices H–N provide optional infrastructure around the AK Core. They do not enlarge the Core mathematical package, replace Core gates, replace repaired evaluation records, replace monitored Type IV diagnostics, replace required tower comparison artifacts, or weaken the audit budget condition.*

Proof. Each appendix in H–N states non-Core technical-extension status and explicit exclusions. Each extension is subordinated to Core sovereignty, manifest requirements, repaired evaluation discipline, representative and eligibility discipline, diagnostic discipline, tower artifact discipline, and metric-ledger audit. Therefore Part III is technical infrastructure, not a Core enlargement. $\square$

Remark 7.64 (Transition to problem interfaces). The natural next layer is the Problem Interface Appendices. By Chapter 8, domain connections to arithmetic, PDE, geometry, category theory, or other external programs are [Spec] by default unless explicitly promoted and reproved.

N.16. Package theorem

Theorem 7.65 (Appendix N higher-extension package). *Within the Technical Extension layer, Appendix N provides the following package.*

1. *Derived, sheaf, stack, and higher-categorical enhancements may organize external structures around the Core.*
2. *Such enhancements affect Core verdicts only through verifier-side re-entry.*
3. *Derived enhancements and derived edge proposals do not replace the eligible Ext^1 regime.*

4. *Sheaf and stack realizations require extraction routes to Core-readable data.*
5. *Sheaf gluing, descent, stack descent, and certificate-record gluing are distinct unless bridged by explicit interface theorems.*
6. *Higher comparison cells and higher coherence defects affect audit only through typed ledger entries.*
7. *Higher functoriality does not strengthen representative status.*
8. *Higher Type IV proxies do not replace $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}})$ or $\phi_{i,W,\tau}$.*
9. *Local higher-extension comparison modes project to Core comparison modes only through Definition 7.49.*
10. *Enhancement artifacts are required for audit readability.*
11. *Higher-extension defects must pass the adjusted metric-budget condition.*
12. *Problem-specific higher implications are [Spec] by default.*
13. *No technical extension result enters the Core without admissible Core import.*
14. *Part III closes as non-Core technical infrastructure.*

Proof. The items follow respectively from the sovereignty principles, verifier-side re-entry, derived edge restrictions, extraction and gluing propositions, higher-cell ledger propositions, representative and Type IV non-replacement results, Core projection rule, manifest requirements, adjusted metric-budget proposition, [Spec] default, no silent import, and the Part III closure principle. $\square$

N.17. Explicit exclusions

The following are not asserted in Appendix N.

- No derived, sheaf-theoretic, stack-theoretic, dg-categorical, or higher-categorical enhancement is Core by default.
- No general derived Ext-edge theorem beyond the eligible Core regime is asserted.
- No higher-degree PH^n -to- Ext^n bridge is asserted.
- No sheaf extraction, stack descent, or hyperdescent theorem replaces repaired persistence evaluation.
- No sheaf gluing theorem is inferred from certificate gluing, and no certificate gluing theorem is inferred from sheaf gluing.
- No higher comparison cell is treated as strict equality unless the declared zero-cost or exact regime proves it for the stated purpose.
- No higher coherence datum replaces an admissible representative.
- No higher Type IV proxy replaces $(\mu_{\text{Collapse}}, \nu_{\text{Collapse}})$ or the tower artifact $\phi_{i,W,\tau}$.
- No higher-extension comparison replaces B-Gate^+ , $\text{Eval}_{1,W,\tau}(F) = 0$, invoked eligibility, required artifacts, or the strict metric budget.
- No advisory-only higher comparison supports Core acceptance.
- No nonzero higher-extension defect is ignored when used in certification.
- No problem-specific arithmetic, PDE, geometric, categorical, motivic, sheaf-theoretic, stack-theoretic, or physical theorem is proved here.

- No Technical Extension appendix enlarges the Core without admissible Core import.

N.18. Provenance and status

Appendix N depends on the Core non-claim discipline of Chapter 7, the interface protocol of Chapter 8, the repaired evaluation discipline of Appendix A, the admissible representative discipline of Appendix B, the eligible realization regime of Appendix C, the Type IV and tower artifact discipline of Appendix D, certificate gluing in Appendix E, manifest requirements in Appendix G, and the technical-extension disciplines of Appendices H–M. Its role is to close the Technical Extension layer while preventing silent Core enlargement.

Status labels for Appendix N. *Technical extension principles:* Definitions 7.1, 7.2.

Technical extension definitions: Definitions 7.6, 7.7, 7.8, 7.11, 7.12, 7.13, 7.16, 7.17, 7.21, 7.22, 7.23, 7.26, 7.27, 7.29, 7.31, 7.32, 7.35, 7.36, 7.37, 7.40, 7.41, 7.44, 7.45, 7.46, 7.48, 7.49, 7.51, 7.52, 7.55, 7.56, 7.59, 7.61.

Technical extension propositions/theorems: Propositions 7.9, 7.10, 7.14, 7.18, 7.19, 7.24, 7.28, 7.30, 7.33, 7.38, 7.39, 7.42, 7.47, 7.50, 7.53, 7.57, 7.60, 7.62, Theorems 7.63, 7.65.

Operational cautions: Remarks 7.3, 7.4, 7.5, 7.15, 7.20, 7.25, 7.34, 7.43, 7.54, 7.58, 7.64.

Explicit exclusions: Section N.17.

Part closure: Theorem 7.63.